Contents

Chapter 1

Introduction

1.1 Prologue

Automation everywhere in CS, we want to set up boilerplates to help our development.

Examples:

- Continuous integration
- Macros are everywhere
- Artifitial intelligence

We delegate tasks to some tools.

expert system →

example plus frappant:

artificial intelligence: based on probability but not reproductible, not predicatable

several other examples in computer sciences:

...

1.2 Automation is everywhere

In the domain of formal proof automation is everywhere, and it is formally called elaboration [64]

In formal proof, proofs are verbose and have lot of details: types, coercions, canonical structures, type-classes.

In theorem provers the user benefit of notations, implicit inference and tactics to ease his job.

Simply typed lambda calculus is an exmaple of automation: an algo checks if ...

1.3 Contributions

In this manuscript we focus on an alternative type-class solver written in the elpi meta-language.

We also talk about a formalized tool. . .

Alternative type-class solver in *Rocq-Elpi*

***Rocq-Elpi*: Higher-order unification support**

Static determinacy checking for *Elpi*

First-order static determinacy checker formalization

CHAPTER 2

Context: languages and tools

At the very base of my PhD there are two softwares, the Rocq Proof assistant (formerly known as *Coq* [6]) and the *Elpi* [15].

2.1 The *Rocq* proof assistant

[6]

Theorem provers with HO logic programming languages: [17]
Mathematical components: [41]
Abella: [24]
SSReflect: [27]
Isabelle: [50]

2.1.1 The basis of the language

Function vs Propositions

2.1.2 Elaboration

As explained in [64], raw text is the mean bridging the communication between the and the theorem prover. This

What is a hole: missing information

In fig. 2.1 (taken from [64]), we have a skematical representation of the elaboration process. [64]

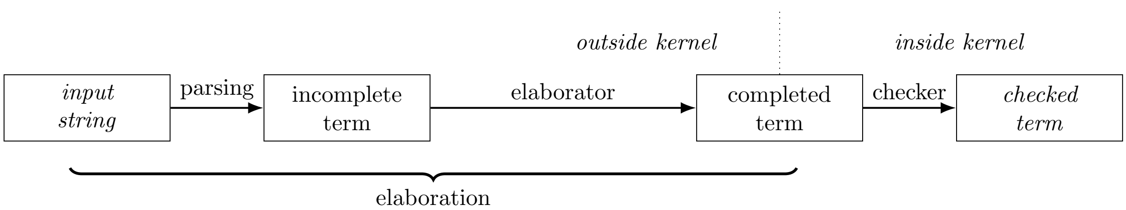


Figure 2.1: Elaboration skeme

Notation

Typechecking The rich type theory of the Coq system confers to type inference the power of computing values and proofs.

Coercions [4]: Used in general to forget information about an object

Collegare con Elaboration

Ad-hoc polymorphism Polymorphism is a fundamental feature of functional programming languages. In his lecture notes, Strachey [59] distinguishes between two main forms of polymorphism: parametric polymorphism and ad-hoc polymorphism. Parametric polymorphic functions are *uniform*, meaning that their behavior does not depend on the specific type at which they are instantiated. For example, the function *length*, with type `'a list -> int`, returns 7 when applied to any list of seven elements. In other words, the compiler reuses the same implementation of *length* for every call, regardless of the instantiation of `'a`.

Parametric polymorphism is widely used in functional programming. However, it is not suitable for modeling mathematical function overloading. In mathematics, the symbol + has a consistent, intuitive meaning, and no ambiguity arises in expressions such as $1 + 2$ or $\pi + e$. In the first expression, + denotes addition over integers; in the second, it denotes addition over real numbers. The same symbol is consistently used for operations that share common algebraic properties.

In contrast, parametric polymorphism does not allow to have multiple different type-dependent interpretations. This limitation appears in languages such as *OCaml*, where integer and floating-point additions are represented by distinct operators, + and +., respectively. Thus, one writes $1 + 2$ for integers and $\pi +. e$ * for floating-point numbers. In general, a different symbol must be introduced for each type implementing a given mathematical operation.

To address the need for a shared notation for operations and properties across types, Wadler and Blott introduced the notion of type classes in 1989 [68] and were soon implemented in *Haskell* [31] as a pervasive feature of the language. Then other languages, such as *Rocq* [58] also extended their type system to allow this kind of overloading.

2.1.3 Ad-hoc polymorphism: Type classes

Symbol overloading Type classes provide a principled form of ad-hoc polymorphism, enabling a single symbol to be associated with multiple type-specific implementations while preserving static type safety. They provide structure parameterized by one or more types t and declare the signatures of a set of functions, called methods, whose types may depend on t. An instance of a type class provides a concrete implementation of the type class for a specific choice of t, providing definitions for all of its methods.

Unlike parametric polymorphism, where a unique function is used for the execution of a call, type classes rely on instance resolution. Typeclasses and instances inhabit a database mapping classes to their instances. When compiling a call to a type class method, the elaborator finds the right implementation by selecting the instance of the type class associated with the given type in the given database.

*In *OCaml*, π is `Float.pi` and e is `Float.exp 1.0`

As an example, consider the type class `Add`, parameterized by a type `T` and equipped with a method `plus` to perform addition between two elements of type `T`. In *Rocq*, the definition of `Add` is given below.

```
Class Add T := {plus: T -> T -> T}.
```

The type of the method `plus` is `∀T, Add T → T → T → T`, where both the type parameter `T` and the instance argument `Add T` are implicit. The typechecking of the term `plus 3 4` gives:

```
@plus nat ?Add 3 4 : bool
where ?Add : [ |- Add nat]
```

The term has type `nat`, but the instance for `Add nat` cannot be inferred, leaving a metavariable `?Add`, this is beacuse in the type-class database, there is no instance associated to `Add`. To resolve this, we define the instance for natural numbers:

```
Instance addNat : Add nat := {plus := Nat.add}.
```

The declaration `addNat` has type `Add nat` and provides an implementation of the method `plus`. Registering this instance extends the instance database for `Add` with an entry for the type `nat`. Afterwards, the term `plus 3 4` can be elaborated successfully, with `?Add` instantiated to `addNat`.

Similarly, we may define an instance for booleans:

```
Instance addR : Add R := {plus := Rplus}.
```

The same symbol `plus` can then be applied to values of type `nat` or `R`. Therefore, extending the notation system, we are now allowed to write $1 + 2$ and $\pi + e$ knowing that the elaborator automatically selects the appropriate instance, `addNat` or `addR`.

A key feature of type classes is the ability to define *conditional* instances, whose availability depends on other instances. An example is the instance for the product type.

```
Instance addProd T1 T2 : Add T1 -> Add T2 -> Add (T1 * T2) :=
  {plus '(x1,y1) '(x2, y2) := (plus x1 x2, plus y1 y2)}.
```

The type of the instance `addProd` states that it implements `Add (T1 * T2)` provided that instances of `Add T1` and `Add T2` are available. In this way, addition on pairs is derived from addition on their components, that is, we can add $(\pi, 3)$ with $(e, 4)$, $(3, (\pi, 4))$ with $(7, (e, 0))$ ...

The type class `Add` can be defined in a similar style in *Haskell*, up to minor syntactic differences. In a proof assistant such as *Rocq*, however, type classes support not only function overloading but also *theorem overloading*.

For instance, one may enrich `Add` with the associativity specification:

```
Class Add T := { plus: T -> T -> T;
  assoc: ∀ a b c, plus a (plus b c) = plus (plus a b) c}.
```

An instance of this refined class must now provide both an implementation of the function `plus` and a proof that `plus` is associative. Then, if `addNat`, `addR` and `addProd` are implemented according the new definition of `Add`, the `assoc` symbol can be used as a proof that addition is associative over natural and real numbers and pairs. This feature of type classes has been adopted in several major *Rocq* libraries. Among them we can site, among others, *tlc*, *stdpp*, *iris* [35].

Class hierarchy Magma + Semigroup

Search strategy Source:[54]

Dire che la ricerca si base sui tipi delle istanze, l'implementazione dei metodi non è importante per la ricerca.

Instance search allow backtracking, in haskell default beahvior no overlapping instances, in coq they can.

Instance priority

Type class ambiguities and modes Source:[55]

Inline modes have been introduced in rocq v 9

Ground mode of rocq is recursively checked when searching for a class

Dire che i modi in stdpp hanno un peso importante: loro fanno ricerca di classi à la prolog, a un input si associa un output. Modi sbagliati implicano ricerche più lunghe e, potenzialmente sbagliate, dato che ci sono meno vincoli.

2.1.4 Ad-hoc polymorphism: Canonical structures

[40] Used to enrich the information about an object. In mathComp

Typeclasses vs Canonical structures

"Find me a structure that works" → typeclasses
"This type already carries its structure" → canonical structures

2.2 The *Elpi* language

Elpi [15] stands for *Embeddable Lambda-Prolog Interpreter*. It is a dialect of λ-*Prolog* [44, 45] that supports higher-order logic programming and integrates a goal suspension mechanism based on CHR (Constraint Handling Rules [23, 30]).

The term "embeddable" highlights that *Elpi* is designed to be used as an extension language. Its main application is the *Rocq-Elpi* plugin (see section 2.3), which enables communication between *Elpi* as the meta-language $\mathcal{M}$ and the *Rocq* proof assistant as the object language $\mathcal{O}$.

A comprehensive presentation of the *Elpi* language and its applications can be found in Tassi's "habilitation à diriger les recherches" [63]. Here we briefly introduce the aspects of *Elpi* that are relevant to this document.

Sections sections 2.2.1 and 2.2.2 describe the syntax and operational semantics of *Elpi*. Our presentation slightly differs from [63], as it follows the conventions adopted later in chapters 5 and 6. In particular, the syntax and semantics presented here exclude the *CHR* component of *Elpi*; we discuss it informally in section 2.2.6.

In section 2.2.3, we present a detailed example of the execution of an *Elpi* program. This example will be reused in chapter 6 when mechanizing the proof of equivalence with the tree-based semantics.

Section **??** outlines several key features of the *Elpi* language. Finally, section 2.2.8 reviews related work and compares our presentation of *Elpi* with existing works in the literature.

$$
\begin{array}{l}
\text{Program}\left\{
\begin{array}{rll}
\mathbb{K} & ::= f, g, \ldots & \text{datum} \\
\mathbb{P} & ::= p, q, \ldots & \text{predicate} \\
\mathbb{U} & ::= X, Y, \ldots & \text{unification variable} \\
\mathbb{N} & ::= x, y, \ldots & \text{nominal variable} \\
\mathbb{Tm} & ::= \mathbb{K} \mid \mathbb{P} \mid \mathbb{U} \mid \mathbb{N} \mid \mathbb{Tm}\ \mathbb{L} & \text{term} \\
\mathbb{L} & ::= \lambda \mathbb{N}.\mathbb{L} \mid \mathbb{Tm} & \lambda\text{–term} \\
\mathbb{A} & ::= \texttt{Cut} \mid \mathbb{Tm} \mid \texttt{pi}\ \mathbb{N}.\ \mathbb{A} \mid \mathbb{R} \Rightarrow \mathbb{A} & \text{atom} \\
\mathbb{R} & ::= \mathbb{Tm}\ \texttt{:-}\ \mathbb{A}^* & \text{rule}
\end{array}\right. \\
\text{Types}\left\{
\begin{array}{rll}
\mathbb{Kd} & ::= \texttt{kind}\ \mathbb{K}\ \mathbb{Ta} & \text{type declaration} \\
\mathbb{Ta} & ::= \texttt{type} \mid \mathbb{Ta} \to \mathbb{Ta} & \text{type arity} \\
\mathbb{Ks} & ::= \texttt{type}\ \mathbb{K}\ \mathbb{Ty} \mid \texttt{type}\ \mathbb{P}\ \mathbb{Ty} & \text{constructor declaration} \\
\mathbb{Tb} & ::= \mathbb{K} \mid \mathbb{N} \mid \mathbb{Tb}\ \mathbb{Ty} & \text{type} \\
\mathbb{Ty} & ::= \mathbb{Ta} \xrightarrow{?\mathrm{m}} \mathbb{Ta} \mid \mathbb{Tb} & \lambda\text{–type} \\
m & ::= \texttt{input} \mid \texttt{output} & \text{mode}
\end{array}\right.
\end{array}
$$

Figure 2.2: Abstract syntax of *Elpi* programs

2.2.1 Abstract syntax of *Elpi* (without *CHR*)

Syntax of *Elpi* programs Figure 2.2 presents the abstract syntax of *Elpi* programs.

The basic syntactic categories are data $\mathbb{K}$, predicates $\mathbb{P}$, unification variables $\mathbb{U}$, and nominal variables $\mathbb{N}$. Nominal variables represent binders introduced either by λ-abstractions or by `pi`-quantification.

A term $\mathbb{Tm}$ is either one of these basic constructs or an application. More precisely, terms are built by the binary application of a term to a λ-term $\mathbb{L}$. Lambda terms introduce abstractions; in the concrete *Elpi* syntax an abstraction is written `x\ t`. Function application follows a curried style: unlike *Prolog*, *Elpi* does not use parentheses between the functor and its arguments.

An atom $\mathbb{A}$ represents an executable unit. It can be:

1. the cut operator (`!` in the concrete syntax),
2. a predicate call,
3. a `pi`-quantification, written `pi x\ t` in the concrete syntax, which introduces a fresh nominal variable `x` visible in `t`, or
4. the dynamic insertion of a rule $\mathbb{R}$, written `r => t`, which makes the rule `r` locally available while solving `t`.

A rule $\mathbb{R}$ consists of a head and a (possibly empty) body, which is a list of atoms. The atoms in the body are called the premises of the rule.

For simplicity, we omit constraints from the syntax, since they do not play an immediate role in the presentation. They are briefly described in section 2.2.6.

Syntax of *Elpi* types In *Elpi*, users can define the algebraic data types required by their development as well as the signatures of predicates. A new algebraic data type is declared with the `kind` keyword, while constructors of that type are introduced using the `type` keyword. A type is ether a definition is either a type declaration, a variable, or the application of a type to another in the case of parametric types. The arrow symbol is used to type function space and in *Elpi* it take an

$$\frac{}{\texttt{runE}\ ((\sigma,\ \varnothing) :: a) \rightsquigarrow (\sigma, a)}\ \texttt{runE}_{\top}$$

$$\frac{}{\texttt{runE}\ \varnothing \rightsquigarrow \square}\ \texttt{runE}_{\bot}$$

$$\frac{\texttt{runE}\ ((\sigma,\ g) :: a) \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\_,\ \texttt{Cut}, a) :: g) :: \_) \rightsquigarrow r}\ \texttt{runE}_{!}$$

$$\frac{\mathcal{B}_E(\mathcal{P}, (\sigma\ t), \sigma, a, g) = a' \qquad \texttt{runE}\ (a'\ +\!\!+\ a) \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\mathcal{P},\ t,\ \_) :: g) :: a) \rightsquigarrow r}\ \texttt{runE}_{\mathcal{B}}$$

$$\frac{y \# \mathcal{P} \quad \texttt{runE}\ ((\sigma,\ (y +_{\forall} \mathcal{P},\ t[x/y], a) :: g) :: a') \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\ \mathcal{P}\ ,\ \texttt{pi}\ x.\ t, a) :: g) :: a') \rightsquigarrow r}\ \texttt{runE}_{\forall}$$

$$\frac{\texttt{runE}\ ((\sigma,\ (h +_{\Rightarrow} \mathcal{P},\ \ \mathrm{t}\ \ , a) :: g) :: a') \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\ \mathcal{P}\ ,\ h \Rightarrow t, a) :: g) :: a') \rightsquigarrow r}\ \texttt{runE}_{\Rightarrow}$$

Figure 2.3: Operational semantics of *Elpi*

optional mode information. The mode, `input` or `output`, tells the runtime which unification algorithm to use when unifying the arguments of two terms, we dealy this discussion in the next paragraph. By default, no mode information is compiled to `output`.

For example, the following declarations define Church naturals and polymorphic lists.

```
kind nat type.
type z nat.
type s nat -> nat.
```

```
kind list type -> type.
type nil list A.
type cons A -> list A -> list A.
```

The type `nat` contains the constant `z` representing zero and the constructor `s` representing the successor of another natural number. A `list` is either empty `nil` or the constructor `cons` that combines a head element with a tail list. Lists are polymorphic: the type of the head element must coincide with the type of the elements stored in the tail.

A predicate is a symbol whose type ends in `prop`. The type `o` is an equivalent notation following the convention of [45]. *Elpi* provides a dedicated syntax for declaring predicates, which also allows the programmer to specify the modes of their arguments.

The declaration `pred p i:t1, o:t2.` where `t1` and `t2` are types, is equivalent to the declaration `type p t1 -> t2 -> prop` but it also specifies that the first argument of `p` is an input (`i:`) and the second is an output (`o:`). These modes are used at runtime to determine which unification strategy should be applied when matching the head of a rule with the current goal.

Program type checking *Elpi* is an (almost) strongly typed language. When a program is interpreted, the type checker verifies that it is well typed. Any type inconsistency is reported as an error, which typically helps identify and correct implementation bugs early.

2.2.2 Operational semantics of *Elpi* (without *CHR*)

A substitution σ of type Σ is a partial mapping from unification variables $\mathbb{U}$ to terms $\mathbb{Tm}$. The application of a substitution to a term t is written $\sigma\ t$. The empty substitution is denoted by ε. The domain $\operatorname{dom}\sigma$ of a substitution σ is the set of variables that are assigned in σ.

Definition 2.2.1 (Substitution extension)**.** A substitution σ' is an extension of σ iff

$$\exists\sigma''.\ \sigma + \sigma'' = \sigma' \ \wedge\ \operatorname{dom}\sigma'' \cap \operatorname{dom}\sigma = \varnothing.$$

A typing environment $\mathbb{E}$ maps predicates $\mathbb{P}$ to their signatures.

A program $\mathcal{P}$ of type $\mathbb{P}$ is represented as a record with the following structure:

$$\mathbb{P}\ \texttt{:= \{index: list}\ \mathbb{R}\texttt{; nvar: set}\ \mathbb{U}\texttt{; sig:}\ \mathbb{E}\ \texttt{\}}$$

The field `index` is an ordered list of rules representing the rule database. The field `nvar` contains the set of nominal variables that have already been introduced, and `sig` is the signature environment.

We write $x\#\mathcal{P}$ to denote that the nominal variable x is fresh with respect to $\mathcal{P}$.`nvar`. The notation $x +_{\forall} \mathcal{P}$ denotes the program obtained by adding x to $\mathcal{P}$.`nvar`. Similarly, $h +_{\Rightarrow} \mathcal{P}$ denotes the program obtained by prepending the rule h to $\mathcal{P}$.`index`.

A goal g of type $\mathcal{G} = \mathbb{P} \times \mathbb{A} \times \overrightarrow{\mathcal{A}}$ is a triple consisting of a program, an atom, and a list of *cut-to alternatives*. An alternative a of type $\mathcal{A}$ is a pair $\Sigma \times \overrightarrow{\mathcal{G}}$ composed of a substitution and a list of goals.

Figure 2.3 presents the derivation rules of the *Elpi* interpreter, defined in terms of the function

$$\texttt{runE} : \overrightarrow{\mathcal{A}} \to (\Sigma, \overrightarrow{\mathcal{A}}) + \square.$$

This semantics is inspired by [12, Sec. 4.3] and [63]. Execution is modeled as the evaluation of a stack of alternatives $\mathcal{A}$. The interpreter processes alternatives sequentially. The evaluation of the alternative list either succeeds, producing a substitution and a new list of alternatives, or fails, returning $\square$.

In fig. 2.3, rule $\texttt{runE}_{\top}$ captures success: if the first alternative contains an empty list of goals, the interpreter returns its associated substitution together with the remaining alternatives. Conversely, rule $\texttt{runE}_{\bot}$ states that evaluation fails when the list of alternatives is empty.

The remaining rules of `runE`, namely $\texttt{runE}_{!}$, $\texttt{runE}_{\mathcal{B}_E}$, $\texttt{runE}_{\forall}$, and $\texttt{runE}_{\Rightarrow}$, apply when the first alternative contains a non-empty list of goals. Such alternative list has the form $((q :: g) :: a)$, where q is the current goal atom (or query). The derivation rule to use depends on the syntactic form of q, which can be one of the four forms of $\mathbb{A}$ listed in fig. 2.2.

If q is a `Cut`, rule $\texttt{runE}_{!}$ discards the current alternatives a and replaces them with the cut-to alternatives stored in q. Evaluation then continues with the remaining goals g.

If q is a `pi`-quantification $\texttt{pi}\,x.\,t$, the interpreter generates a fresh nominal variable y not occurring in $\mathcal{P}$, replaces all free occurrences of x in t with y, and continues evaluation with the resulting atom.

If q corresponds to the dynamic insertion of a rule h, evaluation continues with atom t in the program extended with the local rule h.

Finally, if q is a predicate call, the interpreter performs *backchaining* over the rules stored in $\mathcal{P}$.`index`. Backchaining retrieves the rules that can be used on the query q. In standard Prolog this is done by selecting rules whose head unifies with q. In *Elpi* the process depends on predicate modes, which determine if a argument should be "matched" or "unified".

Elpi provides two procedures, `unify` and `matching`. Both have type

$$\mathbb{Tm} \to \mathbb{Tm} \to \Sigma \to \Sigma + \square,$$

taking two terms and an initial substitution σ. They either fail (returning $\square$) or succeed with an updated substitution σ' that is an extension of σ.

The procedure `unify` implements higher-order unification restricted to the pattern fragment [44] and is used for arguments in `output` position. The procedure `matching`, inspired by *B-Prolog*'s "matching clauses", is used for arguments in `input` position.

Definition 2.2.2 (`unify`)**.** Two terms q and u *unify* under a substitution σ if there exists a substitution σ' extending σ such that $\sigma' q = \sigma' u$, and σ' is a most general unifier for the two terms.

Definition 2.2.3 (`matching`)**.** A query q *matches* a pattern u under a substitution σ if there exists a substitution σ' extending σ such that $\sigma' u = \sigma q$, where σ' assigns at most the free variables occurring in u.

Backchaining is defined by the function $\mathcal{B}$: var refreshing in bc def

Definition 2.2.4 (Backchain, $\mathcal{B} : \mathbb{P} \times \mathbb{Tm} \times \Sigma \to \overrightarrow{\Sigma \times \vec{\mathbb{A}}}$)**.**

$$\mathcal{B}(\mathcal{P}, q, \sigma) = \left[(\sigma', b) \text{ if } \mathcal{U}(\mathcal{S}(\mathcal{P}.\text{sig}, q), q, h, \sigma) = \sigma' \neq \square \;\middle|\; (h :\!- b) \in \mathcal{P}.\text{index} \right].$$

Intuitively, $\mathcal{B}$ selects the rules that can be applied to the current query q under substitution σ. For each usable rule it returns the updated substitution obtained by matching/unifying the rule head with q, together with the rule body.

The definition relies on two auxiliary functions:

$$\mathcal{S} : \mathbb{E} \to \mathbb{Tm} \to \mathbb{Ty} + \square \tag{2.1}$$
$$\mathcal{U} : (\mathbb{Ty} + \square) \to \mathbb{Tm} \to \mathbb{Tm} \to \Sigma \to \Sigma + \square. \tag{2.2}$$

The function $\mathcal{S}$ retrieves the signature of the predicate occurring in a term. It returns the signature if the predicate is present in the environment, and $\square$ otherwise.

The function $\mathcal{U}$ tests whether the query q matches/unifies the head h of a rule under an initial substitution σ. The predicate signature determines the mode of each argument, which in turn selects the appropriate procedure: `matching` for `input` arguments and `unify` for `output` arguments. If all arguments satisfy the unification condition, $\mathcal{U}$ returns the resulting substitution σ'; otherwise it returns $\square$.

Definition 2.2.5 (*Elpi* backchain, $\mathcal{B}_E : \mathbb{P} \times \mathbb{Tm} \times \Sigma \times \vec{\mathcal{A}} \times \vec{\mathcal{G}} \to \vec{\mathcal{A}}$)**.**

$$\mathcal{B}_E(\mathcal{P}, t, \sigma, a, g) = \left[\sigma', ([(\mathcal{P}, q, a) \mid q \in b] +\!\!+ g) \;\middle|\; (\sigma', b) \in \mathcal{B}(\mathcal{P}, t, \sigma) \right].$$

The function $\mathcal{B}_E$, used by rule $\texttt{runE}_{\mathcal{B}}$ in fig. 2.3, builds a new list of alternatives from the result of $\mathcal{B}$. For each filtered rule, the atoms in the rule body become new goals whose cut-to alternatives are a and whose continuation is g. The substitution associated with each alternative is the one returned by $\mathcal{B}$.

```
pred f i:int, o:int.
f 1 2.
f 2 3.
pred r i:int, o:int.
r 0 0.
r 0 1.
r 0 2 :- !.
pred g i:int, o:int.
g A C :- r A B, f B C, !.
g D F :- f D E, f E F.
```

Figure 2.4: Running example of a *Elpi* program

2.2.3 Example of program execution in *Elpi* with `Cut`

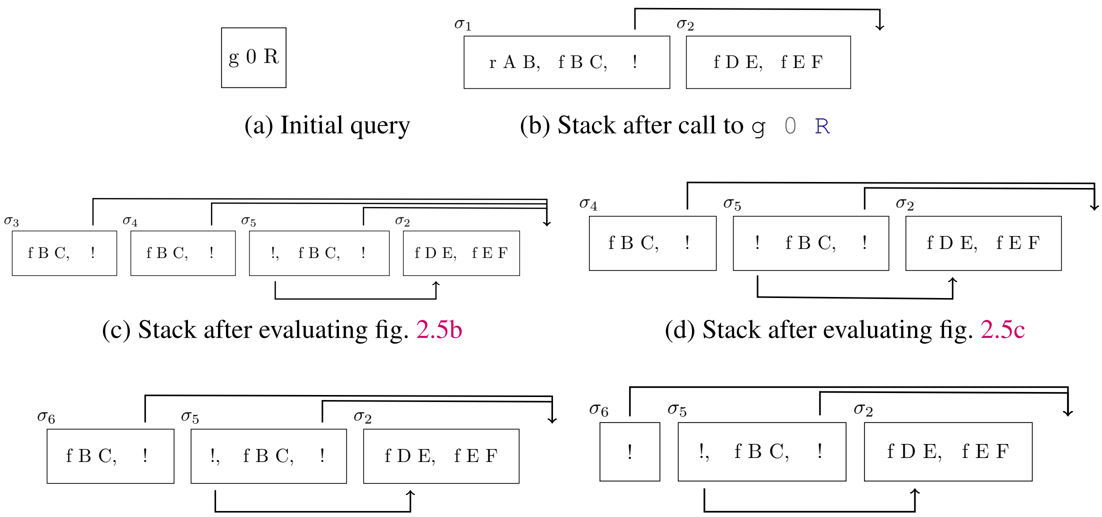
Consider the program in fig. 2.4 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\texttt{r A B, f B C, !}]),\ (\sigma_2, [\texttt{f D E, f E F}])]$$

with $\sigma_1 := \{\texttt{A} \mapsto \texttt{0}, \texttt{C} \mapsto \texttt{R}\}$ and $\sigma_2 := \{\texttt{D} \mapsto \texttt{0}, \texttt{F} \mapsto \texttt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_\mathcal{V}$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [\texttt{!}])]$ where $\sigma_3 := \sigma_1 + \{\texttt{B} \mapsto \texttt{0}\}$; $\sigma_4 := \sigma_1 + \{\texttt{B} \mapsto \texttt{1}\}$ and $\sigma_5 := \sigma_1 + \{\texttt{B} \mapsto \texttt{2}\}$.

The next step of interpretation continues with `f B C` under σ_3, that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [\texttt{!}])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 R`. This succeeds with $\sigma_6 := \sigma_4 + \{\texttt{R} \mapsto \texttt{2}\}$.



(a) Initial query

(b) Stack after call to `g 0 R`

(c) Stack after evaluating fig. 2.5b

(d) Stack after evaluating fig. 2.5c

(e) Stack after evaluating fig. 2.5d

(f) Stack after evaluating fig. 2.5e

Figure 2.5: Example of execution

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for `g` was selected. In contrast, a cut does not discard choice points created earlier; in this example, there are none. The final solution is: $\{\mathtt{A} \mapsto \mathtt{0}, \mathtt{B} \mapsto \mathtt{1}, \mathtt{C} \mapsto \mathtt{R}, \mathtt{R} \mapsto \mathtt{2}\}$.

2.2.4 Higher-order programs in *Elpi*

A notable higher-order program is the one turning relations into functions. [†]

```
func once (pred) -> . % a function taking a fully applied predicate
once P :- P, !.        % the ! commits to the first success
```

We say that the signature of the `once` predicate marks it as *unconditionally* a function: `once` leaves no choice points, no matter what it receives as the higher-order argument `P`.

An example of a *conditional* function is the predicate to map over a list:

```
func map (func A -> B), list A -> list B.
map _ [] [].
map F [X|XS] [Y|YS] :- F X Y, map F XS YS.
```

We say that the signature of `map` has a *precondition*, namely that `F` is a function, and a *post-condition*, namely that `map` is a function, i.e., the call `map F L L'` leaves no choice points. If the precondition is not met the postcondition is not guaranteed to hold and we say that `map` is *miscalled*. In this case we weaken its signature to[‡]:

```
pred map (func A -> B), list A -> list B.
```

Our static analysis tracks miscalled functions and considers them as relations for the analysis of the surrounding code. This is important to avoid code duplication, i.e., we can just have one copy of `map` in our library. We give an example of four scenarios involving calls and miscalls to `map`.

```
% a function                          % a relation
func incr int -> int.                 pred get_pdiv int -> int.
incr 1 2.                             get_pdiv 6 3. get_pdiv 6 2.

       % four predicates calling map or miscalling map
func p list int -> list int.          pred q list int -> list int.
p L R :- map incr L R. % OK           q L R :- map get_pdiv L R. % OK

func r list int -> list int.          func r1 list int -> list int.
r L R :- map get_pdiv L R.            r1 L R :- map get_pdiv L R, !. % OK
          % ^^^^^^^^ error: non-deterministic atom
```

Here `p` is a function because the precondition of `map` is respected (`incr` is a function). The body of `q` contains a miscall to `map`; as a consequence `q` may leave choice points. This is not an error because `q` is declared to be a relation. The body of `q` and `r` are identical, but `r` should be a function and hence the static analyzer complains. The predicate `r1` has the same signature of `r` but it compensates the miscall to `map` by discarding choice points via a cut. The static analyzer accepts this last declaration. We point out that, partial applications, like, `(map incr)`, are accepted in our system, see section 5.5.

[†]The syntax `(pred)`, equivalent to `(pred ->)`, is for fully applied predicates, i.e. for terms with neither inputs nor outputs

[‡]We explain in detail this weakining operation in section 5.5

2.2.5 Dynamic programs in *Elpi*

The domain where Elpi really shines is the manipulation of higher-order data represented via the so-called λ-tree syntax [46] also called HOAS [51]. Such data is pervasive in interactive provers based on Type Theory such as Rocq. In the paradigmatic example below `tm` is the data type for λ-terms.

```
kind tm type.                          % datatype of λ-terms in HOAS form
type app tm -> tm -> tm.               %  - binary application
type lam (tm -> tm) -> tm.             %  - λ-abstraction
```

In the HOAS encoding of syntax with binders we do not find a node for variables: we re-use the notion of bound variable of the programming language to encode the one of the syntax tree. At the level of types we see that the `lam` node carries an Elpi function over terms. It is by applying this function that one can recover the body of the λ-abstraction. A simple program manipulating higher-order data is the function performing a deep copy of a term:

```
func copy tm -> tm.
copy (app A B) (app C D) :- copy A C, copy B D.
copy (lam F) (lam G)      :- pi x\ copy x x => copy (F x) (G x).
```

The copy of an application is built from the copies of the subterms. What is more interesting is how one copies under a binder. The "`pi x\ copy x x =>`" instruction generates a fresh constant `x` and adds to the program the rule `copy x x` for the rest of the execution of copy. That rule explains how to copy the new constant `x`. Intuitively "`pi x\.. => ..`" amounts to a well-bracketed use of Prolog's `assert/1` and `retract/1`. Finally the code performs a recursive call over `F x`: by applying the Elpi function `F` to the fresh symbol `x` one obtains the body of the λ-abstraction where the bound variable is replaced by the new symbol `x`.

In Elpi, all predicates are *dynamic*, or open, hence the code of `copy` can be augmented with rules at runtime. Even so `copy` is a function because no two rules can apply to the same input: `lam` and `app` are different constants, and each constant `x` generated dynamically is guaranteed to be fresh by the `pi` operator, hence each `copy x x` rule is mutually exclusive with the other rules.

The `copy` predicate, as given, implements the identity and may seem rather useless, but it is not! Since the program is open, one can add new rules for `copy` and change its behavior. For example the code below computes the weak head normal form of a λ-term by contracting the application of a λ-abstraction to an argument and uses `copy` to perform the substitution:

```
func whd tm -> tm.
whd (app H A) R :- whd H (lam F), !,                  % we fire the redex
  pi x\ copy x A => copy (F x) S, whd S R.            %   and we continue
whd X X.                                              % or we stop
```

The argument `A` is put in place of the (previously) bound variable in `F` by replacing that variable with `x` and by running `copy` with the addition of a special rule copying `x` to `A`. Our static analysis understands this programming pattern and is able to recognize that `whd` is also a function. In particular the first rule, if taken, discard the second rule (due to the cut) and only calls functions.

2.2.6 *CHR*: goal suspension

[23, 22]

CHR elpi: [30, section 4.3]

CHR Pfenning: [43]

2.2.7 Rule names and insertion

```
:name XXXX
:before YYY
```

Miller: [44]
Huet: [34]
Modes: [71]
SWI prolog: [60]
Andrews: [2]
Vink: [11]

2.2.8 Related works

2.3 *Rocq-Elpi*: a logical framework for *Rocq*

particularly well suited for manipulating syntax trees with binders (see [44, 45, 15, 29]). It is used as an extension language for the Rocq Prover (formerly the Coq proof assistant) and has become an important piece of infrastructure: several projects and libraries depend on Elpi [36, 7, 9, 66, 19, 20]. Examples include the Hierarchy-Builder library-structuring tool [10], which supports the Mathematical Components ecosystem [41] and mechanizations such as the Odd Order and Four Color theorems [26, 25], and Derive [61, 62, 28], a program-and-proof synthesis framework with industrial applications at BlueRock systems.

Distinction between object and meta-language

2.3.1 HOAS: λ-terms representation

In order to implement such a search in Elpi, we shall describe the encoding of Coq terms and then the encoding of instances as rules. Elpi comes equipped with a Higher-Order Abstract Syntax (HOAS [51]) datatype of Coq terms, called `tm`, that includes (among others) the following constructors:

```
type lam  tm -> (tm -> tm) -> tm.      % lambda abstraction
type app  list tm -> tm.               % n-ary application
type all  tm -> (tm -> tm) -> tm.      % forall quantifier
type con  string -> tm.                % constants
```

2.3.2 Unification in $\mathcal{O}$ and $\mathcal{M}$

In order to reason about unification we provide a description of the $\mathcal{O}$ and $\mathcal{M}$ languages where unification variables are first-class terms, i.e. they have a concrete syntax as shown in fig. 4.2. Unification variables in $\mathcal{O}$ (`o-uva` term constructor) have no explicit scope: the "arguments" of a higher-order variable are given via the `o-app` constructor. For example the term «`P x`» is represented as «`o-app [o-uva N, x]`», where `N` is the memory address of `P` and `x` is a bound variable. In $\mathcal{M}$ the representation of «`P x`» is instead «`uva N [x]`», since unification variables are higher-order and come equipped with an explicit scope.

Coqelpi: [61]
Hierarchy Builder: [10]

Trackt: [7]
Equality test [28]
Derive: [62]

2.3.3 Concluding remarks

Overall, Elpi is good for elaboration

2.4 Notational conventions

In the following section we note $\mathbb{B}$ for the boolean type, and $\top$ and $\bot$ for `true` and `false`. The constructors of the option type are written $\cdot t$ for `Some` t and $\square$ for `None`. Following the Mathematical Components library, on which we depend, we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`. List concatenation is the infix `++`, while list comprehension (i.e. mapping a function on a list) is written `[seq f x | x <- l]`. The function `odflt d o` extracts the value from the option `o` and returns the default `d` if `o` is $\square$.

Chapter 3

Architecture of a Type-Class Solver in *Rocq-Elpi*

Type classes provide an elegant mechanism for implementing ad-hoc polymorphism [67]. They allow the same symbol to be reused in different contexts while preserving the same mathematical properties. From a Prolog-like perspective, a type-class problem corresponds to resolving a query to a predicate that associates its input with an instance solving the type-class constraint. In *Rocq*, instances act as rules stored in a database, which are used to backchain and resolve type-class queries.

To address type-class unification problems, we extended *Rocq* with an API called `Class_tactics.register_solver`. This API allows developers to register custom type-class solvers within the *Rocq* elaborator. In this way, type-class resolution can be delegated to a meta-language: the custom solver computes a solution (if it exists) to the type-class constraint and returns it to the *Rocq* elaborator. We refer to such a custom solver as a *meta-solver*.

To support this mechanism, we also introduced another API, `Classes.register_observer`. This API provides a systematic way to notify the meta-language whenever a new class or instance is declared in *Rocq*. The meta-language can then translate these declarations into its own representation and store them in its knowledge base.

Concretely, a meta-language integration must provide two components: (1) a compiler that translates *Rocq* class and instance declarations into the meta-language representation, and (2) a type-class solver that animates the search by interacting with the resulting database.

In our setting, the meta-language used to implement the meta-solver is *Elpi*. Consequently, the main task is to implement the class and instance compiler, which incrementally builds the database on which the *Elpi* program operates.

The type-class solver itself typically follows a *Prolog*-like resolution strategy: instances are tried according to their priority, and backtracking may reconsider previous choices when a branch fails. In *Elpi*, this behavior comes for free, since the resolution mechanism of the type-class solver strongly imitates the standard execution model of the *Elpi* runtime.

In the following sections (sections 3.1 to 3.3), we present the compiler by separating its three main components: the class compiler, the instance compiler, and the goal compiler.

In section 3.4.1, we describe our work on an alternative rule indexing mechanism for the *Elpi* runtime, designed to speed up clause retrieval during resolution.

```
Class Add T := {plus: T -> T -> T}.

Instance addNat : Add nat := {plus := Nat.add}.
Instance addR : Add R  := {plus := Rplus}.

Instance addProd T1 T2 : Add T1 -> Add T2 -> Add (T1 * T2) :=
  {plus '(x1,y1) '(x2, y2) := (plus x1 x2, plus y1 y2)}.
```

Figure 3.1: Add class and instances

Finally, in section 3.5, we discuss practical aspects of using the type-class solver. In particular, we explore how users can control and customize the solver to better fit their needs. We also show the application of our solver to existing libraries, such as *tlc*, *stdpp*, and *iris*.

3.1 Class encoding

A type-class goal is a *Rocq* type* T whose head symbol is a type-class C. Resolving such a goal amounts to finding a *Rocq* instance whose type is an instantiation of T. In general, T contains n arrows, corresponding to the n declared parameters of the class C.

Conceptually, a type-class goal can be translated into a query with $n + 1$ arguments. The first n arguments correspond to the parameters of the class, while the final argument is a term representing the concrete instance that solves the query.

As an example, consider the `Add` type-class introduced in section 2.1.3. For convenience, the full definition of the class and its instances is reproduced in fig. 3.1.

When the class `Add` is declared, the *Elpi* class compiler is triggered and receives the *Rocq* symbol `indt «Add»` as input. From this symbol we can retrieve the type of the class and determine the number of its parameters. In this example, the HOAS representation of the type of `Add` in *Elpi* is

```
prod `T` (sort (typ «Add.u0»)) c0 \ sort (typ «Add.u0»)
```

which indicates that the class has a single parameter.

From this information we generate the predicate associated with the class:

```
pred tc-Add o:term, o:term.
```

By default, the arguments of the predicate are considered outputs. As anticipated in the last paragraph of section 2.1.3, however, *Rocq* allows users to specify modes for the parameters of a class. These modes guide unification when matching instances against goals and help orient the search performed by the *Prolog*-like interpreter.

The modes are specified using the attribute `#[mode="..."]` attached to the class declaration. The *Elpi* compiler can retrieve this attribute and use it to determine the modes of the corresponding predicate.

This translation is not straightforward, however, because the mode systems of *Rocq* and *Elpi* do not coincide. The *Rocq* mode `-` corresponds to the output mode of *Elpi*, but neither `+` nor `!`

*or term since, in *Rocq*, types are terms.

directly matches the *Elpi* input mode. Moreover, a *Rocq* class may be associated with multiple mode declarations, whereas an *Elpi* predicate admits only a single mode specification.

The first issue, is addressed in the following way: + and ! are mapped to the *Elpi* input mode. The input mode is a viable alternative to the two modes. The + mode impose a very strong condition on the term, which should be ground. In the *stdpp* library

3.2 Instance encoding

register hook

3.2.1 From term to clause

In a first version we could be naif and have: tc-class TERM, but this is not good for X reasons:

We cannot benefit from modes

We cannot benefit from indexing

We cannot benefit from determinacy checker

3.2.2 Conditional instances: encoding premises

3.2.3 Rule priorities

3.3 Goal encoding

Devo compilare il contesto

3.4 Rule indexing

3.4.1 Discrimination trees

Discrimination trees: Pfenning notes [†]

TODO: do some benchmarks

Discrimination trees: opt for lists and tree depths.

3.4.2 Some ideas on tabling

[56, 52]

3.5 Discussion

3.5.1 User defined rules

Hint extern etc

[†] https://www.cs.cmu.edu/~fp/courses/99-atp/lectures/lecture28.html

3.5.2 Accessing the database

[19]

3.5.3 Experimental results

3.5.4 Unification problems

$\eta\beta$-unification

δ-unification, coercions and canonical structures TODO: ci sono dei comandi che iterano il database di classi e istanze per aggiungerle tutte in un botto

TODO: Quali sono i comandi che settano, unsettano la search?

CHAPTER 4

$\eta\beta$-unification support for $\mathcal{O}$ and $\mathcal{M}$

We aim to implement a form of proof search known as type-class resolution [68, 58] for Coq's type system [65]. Type-class solvers are unification based proof search procedures reminiscent of Prolog, which back-chain lemmas taken from a database of "type-class instances". Given this analogy with Logic Programming we want to leverage the Elpi meta programming language [61], a dialect of λProlog, already used to extend Coq in various ways [61, 62, 28, 19].

The use of a meta language facilitates the implementation of a proof system in two key ways. The first and most well-know one is that variable binding and substitution come for free. The second one is that these meta languages come equipped with some form of unification, a cornerstone of proof construction and proof search. In this paper we focus on this last aspect of the implementation of a Prolog-like proof system, precisely *how to reuse the higher-order unification procedure of the meta language in order to simulate a higher-order logic program for the object language*.

We take as an example the `Decision` and `Finite` type classes from the Stdpp library [35]. The class `Decision` identifies predicates equipped with a decision procedure, while `Finite` identifies types whose inhabitants can be enumerated in a (finite) list. The following three type-class instances state that: 1) the type of natural numbers smaller than `n`, called `fin n`, is finite; 2) the predicate `nfact n nf`, relating a natural number `n` to the number of its prime factors `nf`, is decidable; 3) the universal closure of a predicate has a decision procedure if its domain is finite and if the predicate is decidable.

```
Instance fin_fin: ∀n, Finite (fin n).                (* r1 *)
Instance nfact_dec: ∀n nf, Decision (nfact n nf).  (* r2 *)
Instance forall_dec: ∀A P, Finite A →               (* r3 *)
  (∀x:A, Decision (P x)) → Decision (∀x:A, P x).
```

Given this database, a type-class solver is expected to prove the following statement automatically:

```
Decision (∀x: fin 7, nfact x 3)                      (* g *)
```

The proof found by the solver back-chains on rule 3 (the only rule about the ∀ quantifier), and then solves the premises with rules 1 and 2 respectively. Note that rule 3 features a second-order parameter `P` that represents a function of type `A → Prop` (a 1-argument predicate over `A`). The solver infers a value for `P` by unifying the conclusion of rule 3 with the goal, and in particular, it has to solve the unification problem `P x = nfact x 3`. This higher-order problem falls in the so-called pattern fragment and admits a unique solution:

```
P = λx.nfact x 3
```

```
finite   (app [con"fin", N]).                    % (r1)
decision (app [con"nfact", N, NF]).              % (r2)
decision (all A x\ app [P, x]) :- finite A,      % (r3)
  pi w\ decision (app [P, w]).
```

Figure 4.1: Rules in elpi

In the rest of this paper, we refer to the pattern fragment as $\mathcal{L}$ [44].
Following the standard syntax of λProlog [45], the meta-level binding of a variable `x` in an expression `e` is written as «`x\ e`», while square brackets delimit a list of terms separated by comma. For example, the term «`∀y:t, nfact y 3`» is encoded as follows:

```
all (con"t") y\ app [con"nfact", y, con"3"]
```

We now illustrate a naive encoding of the three instances above as higher-order logic programming rules: capital letters denote rule parameters; `:-` separates the rule's head from the premises, and «`pi w\ p`» introduces a fresh nominal constant `w` for the premise `p`.
Unfortunately this encoding of rule *r3* in 4.1 does not work since it uses the predicate `P` as a first order term: for the meta language its type is `tm`. If we try to back-chain the rule *r3* in 4.1 on the encoding of the goal below given here

```
decision (all (app [con"fin", con"7"]) x\ app [con"nfact", x, con"3"]).
```

we obtain an unsolvable unification problem below: the two lists of terms have different lengths!

```
app [con"nfact", x, con"3"] = app [P, x]
```

The problem above has no solution in a first-order language where `P` can only stand for a predicate symbol, while in a higher-order language `P` can stand for any 1-argument predicate.

In this paper we study a more sophisticated encoding of Coq terms and rules that, on a first approximation, would reshape rule r3 in 4.1 as follows:

```
decision (all A x\ Pm x) :- link Pm P A, finite A,     % (r3')
  pi x\ decision (app [P, x]).
```

Since `Pm` is a higher-order unification variable of type `tm -> tm`, with `x` in its scope, the unification problem admits one solution:

```
app [con"nfact", x, con"3"] = Pm x
Pm = x\ app [con"nfact", x, con"3"]
```

Once the head of rule *r3'* unifies with the goal, the premise «`link Pm A P`» brings the assignment for *Pm* back to the domain `tm` of Coq terms, obtaining the expected solution:

```
P = lam A x\ app [con"nfact", x, con"3"]
```

This simple example is sufficient to show that the encoding we seek is not trivial and does not only concern the head of rules, but the entire sequence of unification problems that constitute the execution of a logic program.

In fact the solution for `P` above generates a (Coq) β-redex in the second premise (the predicate under the `pi w\`). In turn, this redex prevents rule *r2* in 4.1 from backchaining properly since the following unification problem has no solution:

```
app [ lam A (a\ app [con"nfact", a, con"3"]) , x] =
app [ con"nfact"                             , N, NF]
```

The root cause of the problems we outlined in this example is a subtle mismatch between the equational theories of the meta language and the object language, which in turn makes the unification procedures of the meta language weak. The equational theory of the meta-language Elpi encompasses $\beta\eta$-equivalence and its unification procedure can solve higher-order problems in the pattern fragment. Although the equational theory of Coq is much richer, for efficiency and predictability reasons automatic proof search procedures typically employ a unification procedure that only captures a $\beta\eta$-equivalence and only operates in $\mathcal{L}$. The similarity is striking, but one needs to exercise some caution in order to simulate a higher-order logic program in Coq using the unification of Elpi.

In this paper we identify a minimal language $\mathcal{O}$ in which the problems sketched in the introduction can be formally described. We detail an encoding of a logic program in $\mathcal{O}$ to a strongly related logic program in $\mathcal{M}$, the language of the meta language, and we show that the higher-order unification procedure of the meta language $\simeq_m$ can be used to simulate a higher-order unification procedure $\simeq_o$ for the object language that features $\beta\eta$-conversion. We show how $\simeq_o$ can be extended with heuristics to deal with problems outside the pattern fragment.
Section 4.1 states the problem formally and gives the intuition behind our solution; section 4.2 sets up a basic simulation of first-order logic programs, section 4.3 and section 4.4 extend it to higher-order logic programs in the pattern fragment while section 4.6 goes beyond the pattern fragment. Section 4.7 discusses the implementation in Elpi.
The λProlog code discussed in the paper can be accessed at the address `https://github.com/FissoreD/ho-unif-for-free`.

4.1 Problem statement and solution

Even if we encountered the problem working on Coq's Calculus of Inductive Constructions, we devise a minimal setting to ease its study. In this setting, we have a (first order) language $\mathcal{O}$ with a rich equational theory and a $\mathcal{M}$ language with a simpler one.

Notational conventions When we write $\mathcal{M}$ terms outside code blocks we follow the usual λ-calculus notation, reserving f, g, a, b for constants, x, y, z for bound variables and X, Y, Z, F, G, H

```
kind to type.                              kind tm  type.
type o-app list to -> to.                  type app list tm -> tm.
type o-lam (to -> to) -> to.               type lam (tm -> tm) -> tm.
type o-con string -> to.                   type con string -> tm.
type o-uva addr -> to.                     type uva addr -> list tm -> tm.
```

Figure 4.2: The $\mathcal{O}$ and $\mathcal{M}$ languages

for unification variables. However, we need to distinguish between the "application" of a unification variable to its scope and the application of a term to a list of arguments. We write the scope of unification variables in subscript while we use juxtaposition for regular application. Here are few examples:

$f \cdot a$	`app [con "f", con "a"]`
$\lambda x.\lambda y.F_{xy}$	`lam x\ lam y\ uva F [x, y]`
$\lambda x.F_x \cdot a$	`lam x\ app [uva F [x], con "a"]`
$\lambda x.F_x \cdot x$	`lam x\ app [uva F [x], x]`

When it is clear from the context, we shall use the same syntax for $\mathcal{O}$ terms (although we never use subscripts for unification variables).
We use s, $s_1, \ldots$ for terms in $\mathcal{O}$ and t, $t_1, \ldots$ for terms in $\mathcal{M}$.

4.1.1 Equational theories and unification

Term equality: $=_o$ and $=_m$ For both languages, we extend the equational theory over ground terms to the full language by adding reflexivity for unification variables, i.e. a unification variable is equal to itself but different to any other term.

The first four rules are common to both equalities and define the usual congruence over terms. Since we use an HOAS encoding, they also capture α-equivalence. In addition to that, $=_o$ has rules for η and β-equivalence.

```
type (=o) to -> to -> o.
o-con X =o o-con X.
o-app A =o o-app B :- forall2 (=o) A B.
o-lam F =o o-lam G :- pi x\ x =o x => F x =o G x.          % rλλ
o-uva N =o o-uva N.
o-lam F =o T :-                                              % rηl
  pi x\ beta T [x] (T' x), x =o x => F x =o T' x.
T =o o-lam F :-                                              % rηr
  pi x\ beta T [x] (T' x), x =o x => T' x =o F x.
o-app [o-lam X|L] =o T :- beta (o-lam X) L R, R =o T.       % rβl
T =o o-app [o-lam X|L] :- beta (o-lam X) L R, T =o R.       % rβr
```

Figure 4.3: FO eq

```
type (=m) tm -> tm -> o.
con C =m con C.
app A =m app B :- forall2 (=m) A B.
lam F =m lam G :- pi x\ x =m x => F x =m G x.
uva N A =m uva N B :- forall2 (=m) A B.
```

Note that the symbol `|` separates the head of a list from the tail; that `forall2` applies a binary predicate to two lists (of equal length); and that `=>` is λProlog syntax for implication and is used to augment the current program with an extra rule. Both procedures use implication to assume that the fresh nominal constants introduces by `pi` are equal to themselves.

The main point in showing these equality tests is to remark how weaker $=_m$ is, and to identify the four rules that need special treatment in the implementation of $\simeq_o$. For brevity, we omit the code of `beta`: it is sufficient to know that «`beta F L R`» computes in `R` the weak-head normal form of «`o-app [F|L]`».

Substitution: ρs and σt We write $\sigma = \{ X \mapsto t \}$ for the substitution that assigns the term t to the variable X. We write σt for the application of the substitution to a term t, and $\sigma X = \{ \sigma t \mid t \in X \}$ when X is a set of terms. We write $\sigma \subseteq \sigma'$ when σ is more general than σ'. We shall use ρ for $\mathcal{O}$ substitutions, and σ for the $\mathcal{M}$ ones. For brevity, in this section, we consider the substitution for $\mathcal{O}$ and $\mathcal{M}$ identical. We defer to section 4.2.1 a more precise description pointing out their differences.

Term unification: $\simeq_o$ vs. $\simeq_m$ $\mathcal{M}$'s unification signature is:

```
type (≃m) tm -> tm -> subst -> subst -> o.
```

We write $\sigma t_1 \simeq_m \sigma t_2 \mapsto \sigma'$ when σt_1 and σt_2 unify with substitution σ'. Note that σ' is a refined (i.e. extended) version of σ; this is reflected by the signature above that relates two substitutions. We write $t_1 \simeq_m t_2 \mapsto \sigma'$ when the initial substitution σ is empty. We write $\mathcal{L}$ as the set of terms that are in the pattern-fragment, i.e. every unification variable is applied to a list of distinct names.

The specification of a "good" unification procedure restricted to a domain $\mathcal{D}$ is the following:

Proposition 4.1.1 (Good unification). *A good unification $\simeq$ for equality $=$ in the domain $\mathcal{D}$ satisfies the following properties:*

$$\{ t_1,\ t_2 \} \subseteq \mathcal{D} \Rightarrow t_1 \simeq t_2 \mapsto \rho \Rightarrow \rho t_1 = \rho t_2 \tag{4.1}$$
$$\{ t_1,\ t_2 \} \subseteq \mathcal{D} \Rightarrow \rho t_1 = \rho t_2 \Rightarrow \exists \rho', t_1 \simeq t_2 \mapsto \rho' \wedge \rho' \subseteq \rho \tag{4.2}$$

The meta language of choice is expected to provide an implementation of $\simeq_m$ that is a good unification for $=_m$ in $\mathcal{L}$.

Even if we provide an implementation of the object-language unification $\simeq_o$ in section 4.2.6, our final goal is to simulate the execution of an entire logic-program.

4.1.2 The problem: logic-program simulation

We represent a logic program *run* in $\mathcal{O}$ as a sequence of n *steps*. Each step p consists in unifying two terms, $\mathbb{P}_{p_l}$ and $\mathbb{P}_{p_r}$, taken from the list of all unification problems $\mathbb{P}$. We write $\mathbb{P}_p = \{ \mathbb{P}_{p_l}, \mathbb{P}_{p_r} \}$ and $s \in \mathbb{P} \Leftrightarrow \exists p, s \in \mathbb{P}_p$. The composition of these steps starting from the empty substitution ρ_0 produces the final substitution ρ_n, which is the result of the logic program execution.

$$\mathrm{step}_o(\mathbb{P}, p, \rho) \mapsto \rho' \stackrel{def}{=} \rho \mathbb{P}_{p_l} \simeq_o \rho \mathbb{P}_{p_r} \mapsto \rho'$$
$$\mathrm{run}_o(\mathbb{P}, n) \mapsto \rho_n \stackrel{def}{=} \bigwedge_{p=1}^{n} \mathrm{step}_o(\mathbb{P}, p, \rho_{p-1}) \mapsto \rho_p$$

In order to simulate a logic program we start by compiling $\mathbb{P}$ to $\mathbb{Q}$, i.e. each $\mathcal{O}$-term $s \in \mathbb{P}$ is translated to a $\mathcal{M}$-term $t \in \mathbb{Q}$. We write this translation as $\langle s \rangle \mapsto (t, m, l)$. The implementation of the compiler is detailed in sections 4.2, 4.4 and 4.6, here we just point out that it additionally produces a variable map m and a list of links l. The variable map connects unification variables in $\mathcal{M}$ to variables in $\mathcal{O}$ and is used to "decompile" the substitution, that is written $\langle \sigma, m, l \rangle^{-1} \mapsto \rho$. Links are an accessory piece of information whose description is deferred to section 4.1.3.

Then we simulate each run in $\mathcal{O}$ with a run in $\mathcal{M}$ as follows:

$$
\begin{aligned}
&\text{step}_m(\mathbb{Q}, p, \sigma, \mathbb{L}) \mapsto (\sigma'', \mathbb{L}') \stackrel{def}{=\!=} \\
&\quad \sigma\mathbb{Q}_{p_l} \simeq_m \sigma\mathbb{Q}_{p_r} \mapsto \sigma' \wedge \text{progress}(\mathbb{L}, \sigma') \mapsto (\mathbb{L}', \sigma'') \\
&\text{run}_m(\mathbb{P}, n) \mapsto \rho_n \stackrel{def}{=\!=} \\
&\quad \mathbb{Q} \times \mathbb{M} \times \mathbb{L}_0 = \{(t, m, l) | s \in \mathbb{P}, \langle s \rangle \mapsto (t, m, l)\} \\
&\quad \textstyle\bigwedge_{p=1}^{n} \text{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \mapsto (\sigma_p, \mathbb{L}_p) \\
&\quad \langle \sigma_n, \mathbb{M}, \mathbb{L}_n \rangle^{-1} \mapsto \rho_n
\end{aligned}
$$

By analogy with $\mathbb{P}$, we write $\mathbb{Q}_{p_l}$ and $\mathbb{Q}_{p_r}$ for the two $\mathcal{M}$ terms being unified at step p, and we write $\mathbb{Q}_p$ for the set $\{ \mathbb{Q}_{p_l}, \mathbb{Q}_{p_r} \}$.
The step_m procedure is made of two sub-steps: a call to the meta-language unification and a check for progress on the set of links, that intuitively will compensate for the weaker equational theory honored by $\simeq_m$. Procedure run_m compiles all terms in $\mathbb{P}$, then executes each step, and finally decompiles the solution. We claim:

Proposition 4.1.2 (Run equivalence)**.** $\forall \mathbb{P}, \forall n,$ *if* $\mathbb{P} \subseteq \mathcal{L}$

$$\text{run}_o(\mathbb{P}, n) \mapsto \rho \wedge \text{run}_m(\mathbb{P}, n) \mapsto \rho' \Rightarrow \forall s \in \mathbb{P}, \rho s =_o \rho' s$$

That is, the two executions give equivalent results if all terms in $\mathbb{P}$ are in the pattern fragment. Moreover:

Proposition 4.1.3 (Simulation fidelity)**.** *In the context of* run_o *and* run_m*, if* $\mathbb{P} \subseteq \mathcal{L}$ *we have that* $\forall p \in 1 \ldots n,$

$$\text{step}_o(\mathbb{P}, p, \rho_{p-1}) \mapsto \rho_p \Leftrightarrow \text{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \mapsto (\sigma_p, \mathbb{L}_p)$$

In particular, this property guarantees that a *failure* in the $\mathcal{O}$ run is matched by a failure in $\mathcal{M}$ *at the same step*. We consider this property very important from a practical point of view since it guarantees that the execution traces are strongly related, and in turn, this enables a user to debug a logic program in $\mathcal{O}$ by looking at its execution trace in $\mathcal{M}$.

We also claim that run_m handles terms outside $\mathcal{L}$ in the following sense:

Proposition 4.1.4 (Fidelity recovery)**.** *In the context of* run_o *and* run_m*, if* $\rho_{p-1}\mathbb{P}_p \subseteq \mathcal{L}$ *(even if* $\mathbb{P}_p \notin \mathcal{L}$*) then*

$$\text{step}_o(\mathbb{P}, p, \rho_{p-1}) \mapsto \rho_p \Leftrightarrow \text{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \mapsto (\sigma_p, \mathbb{L}_p)$$

In other words, if the two terms involved in a step re-enter $\mathcal{L}$, then step_m and step_o are again related, even if $\mathbb{P} \notin \mathcal{L}$ and hence proposition 4.1.3 does not apply. Indeed, the main difference between proposition 4.1.3 and proposition 4.1.4 is that the assumption of the former is purely static, it can be checked upfront. When this assumption is not satisfied, one can still simulate a logic program and have guarantees of fidelity if, at run time, decidability of higher-order unification is restored.

This property has practical relevance since in many logic programming implementations, including Elpi, the order in which unification problems are tackled does matter. The simplest example is the sequence $F \simeq \lambda x.a$ and $F \cdot a \simeq a$: the second problem is not in $\mathcal{L}$ and has two unifiers, namely $\sigma_1 = \{ F \mapsto \lambda x.x \}$ and $\sigma_2 = \{ F \mapsto \lambda x.a \}$. The first problem picks σ_2, making the second problem re-enter $\mathcal{L}$.

4.1.3 The solution (in a nutshell)

A term s is compiled to a term t where every "problematic" subterm e is replaced by a fresh unification variable h with an accessory *link* that represents a suspended unification problem $h \simeq_m e$. As a result, $\simeq_m$ is "well behaved" on t, in the sense that it does not contradict $=_o$ as it would otherwise do on the "problematic" sub-terms.
We now define "problematic" and "well behaved" more formally. We use the $\Diamond$ symbol since it stands for "possibly" in modal logic and all problematic terms are characterized by some uncertainty.

Definition 4.1.1 ($\Diamond\beta$)**.** $\Diamond\beta$ is the set of terms of the form $F \cdot x_1 \ldots x_n$ such that $x_1 \ldots x_n$ are distinct names (of bound variables).

An example of a $\Diamond\beta$ term is the application $F \cdot x$. This term is problematic since the application node of its syntax tree cannot be used to justify a unification failure, i.e. by properly instantiating F, the term head constructor may become a λ, or a constant, or a name, or remain an application.

Definition 4.1.2 ($\Diamond\eta$)**.** $\Diamond\eta$ is the set of terms $\lambda x.s$ such that $\exists \rho, \rho(\lambda x.s)$ is an η-redex.

An example of a term s in $\Diamond\eta$ is $\lambda x.\lambda y.F \cdot y \cdot x$ since the substitution $\rho = \{F \mapsto \lambda x.\lambda y.f \cdot y \cdot x\}$ makes $\rho s =_o \lambda x.\lambda y.f \cdot x \cdot y$, which is the eta-long form of f. This term is problematic since its leading λ abstraction cannot justify a unification failure against a constant f.

Definition 4.1.3 ($\Diamond\mathcal{L}$)**.** $\Diamond\mathcal{L}$ is the set of terms of the form $F \cdot t_1 \ldots t_n$ such that $t_1 \ldots t_n$ are not distinct names.

These terms are problematic for the very same reason terms in $\Diamond\beta$ are, but they cannot be handled directly by the unification of the meta language, which is only required to handle terms in $\mathcal{L}$. Still, given $s \in \Diamond\mathcal{L}$ there may exist a substitution ρ such that $\rho s \in \mathcal{L}$.

We write $\mathcal{P}(t)$ for the set of sub-terms of t, and we write $\mathcal{P}(X) = \bigcup_{t \in X} \mathcal{P}(t)$ when X is a set of terms.

Definition 4.1.4 (Well behaved set)**.** Given a set of terms X,

$$\mathcal{W}(X) \Leftrightarrow \forall t \in \mathcal{P}(X), t \notin (\Diamond\beta \cup \Diamond\eta \cup \Diamond\mathcal{L})$$

We write $\mathcal{W}(t)$ as a short for $\mathcal{W}(\{t\})$. We claim our compiler validates the following properties:

Proposition 4.1.5 ($\mathcal{W}$-enforcing)**.** *Given a term s in $\mathcal{O}$,*

$$\langle s \rangle \mapsto (t, m, l) \Rightarrow \mathcal{W}(t)$$

Proposition 4.1.6 (compiler-useful)**.** *Given two terms s_1 and s_2, if $\exists \rho, \rho s_1 =_o \rho s_2$, then*

$$\langle s_i \rangle \mapsto (t_i, m_i, l_i) \text{ for } i \in \{1,2\} \Rightarrow t_1 \simeq_m t_2 \mapsto \sigma$$

In other words the compiler outputs terms in $\mathcal{W}$ even if its input is not. And if two terms can be unified in the source language then their images can also be unified in the target language. Note that the property holds for any substitution, not necessarily a most general one: when applied to terms in $\mathcal{W}$ the meta-language unification $\simeq_m$ agrees with the object-language equality $=_o$. Note that a compiler that translate all terms to unification variables would satisfy this property, but wound certainly fail to verify proposition 4.1.3.

Proposition 4.1.7 ($\mathcal{W}$-preservation)**.** $\forall\mathbb{Q}, \forall\mathbb{L}, \forall p, \forall\sigma, \forall\sigma'$

$$\mathcal{W}(\sigma\mathbb{Q}) \wedge \sigma\mathbb{Q}_{p_l} \simeq_m \sigma\mathbb{Q}_{p_r} \mapsto \sigma' \Rightarrow \mathcal{W}(\sigma'\mathbb{Q})$$
$$\mathcal{W}(\sigma\mathbb{Q}) \wedge \text{progress}(\mathbb{L}, \sigma) \mapsto (\_, \sigma') \Rightarrow \mathcal{W}(\sigma'\mathbb{Q})$$

Proposition 4.1.7 is key to proving propositions 4.1.2 and 4.1.3. Informally, it says that the problematic terms moved on the side by the compiler are not reintroduced by step_m, hence $\simeq_m$ can continue to operate properly. In sections 4.3, 4.4 and 4.6 we describe how to recognize terms in $\Diamond\beta$, $\Diamond\eta$ and $\Diamond\mathcal{L}$ and how progress takes care of links and how it preserves $\mathcal{W}$ and ensures propositions 4.1.2 to 4.1.4.

We prove proposition 4.1.3 (SIMULATION FIDELITY) in sections 4.4 to 4.6 and proposition 4.1.2 (RUN EQUIVALENCE) only in section 4.3, where links are not present yet. In order to prove proposition 4.1.2 in the presence of links one needs to refine proposition 4.1.3 by relating the two substitutions ρ_p and σ_p: they are equivalent only by using decompilation to take into account terms in $\mathbb{L}$. Then, by induction on the number of steps, proposition 4.1.3 implies proposition 4.1.2. We discuss proposition 4.1.4 (FIDELITY RECOVERY) in section 4.6.

4.2 Basic compilation and simulation

4.2.1 Memory map ($\mathbb{M}$) and substitution (ρ and σ)

Unification variables are identified by a (unique) memory address. The memory and its associated operations are described below:

```
type new mem A -> addr -> mem A -> o.
```

If a memory cell is `none`, then the corresponding unification variable is not set. `assign` sets an unset cell to the given value, while `new` finds the first unused address and sets it to `none`.

Since each $\mathcal{M}$ unification variable occurs together with a scope, its assignment needs to be abstracted over it to enable the instantiation of the same assignment to different scopes. This is expressed by the `inctx` container, and in particular its `abs` binding constructor.

```
kind inctx type -> type.
type abs (tm -> inctx A) -> inctx A.
type val A -> inctx A.
typeabbrev assignment (inctx tm).
typeabbrev subst (mem.mem assignment).
```

A solution to a $\mathcal{O}$ variable is, instead, a plain term, that is, `fsubst` is an abbreviation for `mem to`. The compiler establishes a mapping between variables of the two languages.

```
kind ovariable type.
type ov addr -> ovariable.
kind mvariable type.
type mv addr -> arity -> mvariable.
kind mapping type.
type (<->) ovariable -> mvariable -> mapping.
typeabbrev mmap (list mapping).
```

Each `mvariable` is stored in the mapping together with its arity (a number) so that the code of 4.4 below can preserve the following property:

Invariant 4.2.1 (Unification-variable arity). *Each variable* `A` *in* $\mathcal{M}$ *has a (unique) arity* `N` *and each occurrence* `(uva A L)` *is such that* `L` *has length* `N`*.*

```
type m-alloc ovariable -> mvariable -> mmap -> mmap ->
  subst -> subst -> o.
m-alloc Ov Mv M M S S :- mem M (Ov <-> Mv), !.
m-alloc Ov Mv M [Ov <-> Mv|M] S S1 :- Mv = mv N _, new S N S1.
```

Figure 4.4: Malloc code

When a single `ovariable` occurs multiple times with different numbers of arguments, the compiler generates multiple mappings for it, on a first approximation, and then ensures the mapping is bijective by introducing η`-link`; this detail is discussed in section 4.5.

It is worth examining the code of `deref`, that applies the substitution to a $\mathcal{M}$ term. Notice how assignments are moved to the current scope, i.e. the `abs`-bound variables are renamed with the names in the scope of the unification variable occurrence.

```
type deref subst -> tm -> tm -> o.
deref _ (con C) (con C).
deref S (app A) (app B) :- map (deref S) A B.
deref S (lam F) (lam G) :-
  pi x\ deref S x x => deref S (F x) (G x).
deref S (uva N L) R :- set? N S A,
  move A L T, deref S T R.
deref S (uva N A) (uva N B) :- unset? N S,
  map (deref S) A B.
```

Note that `move` strongly relies on invariant 4.2.1: the length of the arguments of all occurrences of a unification variable is the same. Hence, they have the same simple type for the meta-level, and therefore the number of `abs` nodes in the assignment matches that length. This guarantees that `move` never fails.

```
type move assignment -> list tm -> tm -> o.
move (abs Bo) [H|L] R :- move (Bo H) L R.
move (val A)  [] A :- !.
```

We write $\sigma = \{ A_{xy} \mapsto y \}$ for the assignment «`abs x\abs y\y`» and $\sigma = \{ A \mapsto \lambda x.\lambda y.y \}$ for «`lam x\lam y\y`».

4.2.2 Links ($\mathbb{L}$)

As mentioned in section 4.1.3, the compiler replaces terms in $\Diamond\eta$, $\Diamond\beta$, and $\Diamond\mathcal{L}$ with fresh variables linked to the problematic terms. Terms in $\Diamond\beta$ do not need a link since $\mathcal{M}$ variables faithfully represent the problematic term thanks to their scope.

```
kind baselink type.
type link-eta  tm -> tm -> baselink.
type link-llam tm -> tm -> baselink.
typeabbrev links (list (inctx baselink)).
```

The right-hand side of a link, the problematic term, can occur under binders. To accommodate this situation, the compiler wraps `baselink` using the `inctx` container.

Invariant 4.2.2 (Link left hand side)**.** *The left-hand side of a suspended link is a variable.*

New links are suspended by construction. If the left-hand side is assigned during a step, then the link is considered for progress and possibly eliminated. This is discussed in section 4.4 and section 4.6.

In the examples we represent links as equations under a context. The equality sign is subscripted with the kind of `baselink`. For example $x \vdash A_x =_{\mathcal{L}} F_x\ a$ corresponds to:

```
abs x\ val (link-llam (uva A [x]) (app[uva F [x],con "a"]))
```

4.2.3 Compilation

The simple compiler described in this section serves as a base for the extensions in sections 4.3, 4.4 and 4.6. Its main task is to beta normalize the term and map one syntax tree to the other. In order to bring back the substitution from $\mathcal{M}$ to $\mathcal{O}$ the compiler builds a "memory map" connecting the $\mathcal{O}$ variables to the $\mathcal{M}$ ones.

The signature of the `comp` predicate below allows for the generation of links (suspended unification problems), which plays no role in this section but plays a major role in sections 4.3, 4.4 and 4.6.

```
comp (o-app A) (app A1) M1 M2 L1 L2 S1 S2 :-        % r@
  fold6 comp A A1 M1 M2 L1 L2 S1 S2.
```

Figure 4.5: Comp FO

```
type compile to -> tm -> mmap -> mmap ->  links -> links ->
  subst -> subst -> o.
compile F G M1 M2 L1 L2 S1 S2 :-
  beta-normal F F', comp F' G M1 M2 L1 L2 S1 S2.
```

With respect to section 4.1, the signature also allows for updates to the substitution. The code above uses that possibility in order to allocate space for the variables, i.e. it sets their memory address to `none` (a detail not worth mentioning in the previous sections).

```
type comp-lam (to -> to) -> (tm -> tm) ->
  mmap -> mmap -> links -> links -> subst -> subst -> o.
comp-lam F G M1 M2 L1 L3 S1 S2 :-
  pi x y\ (pi M L S\ comp x y M M L L S S) =>
    comp (F x) (G y) M1 M2 L1 (L2 y) S1 S2,
  close-links L2 L3.
```

Figure 4.6: Comp FO

In the code above, the syntax `pi x y\..` is syntactic sugar for iterated `pi` abstraction, as in `pi x\ pi y\..`. The auxiliary predicate `fold6` folds a predicate with three accumulators (the memory map, the links and the substitution) over a lists to obtain a new list and the final values of the accumulators.

The auxiliary function `close-links` tests if the bound variable `v` really occurs in the link. If it does, the link is wrapped into an additional `abs` node binding `v`. In this way links generated deep inside the compiled terms can be moved outside their original context of binders.

```
type close-links (tm -> links) -> links -> o.
close-links (v\[X v|L v]) [abs X|R] :- close-links L R.
close-links (_\[]) [].
```

4.2.4 Execution

A step in $\mathcal{M}$ consists in unifying two terms and reconsidering all links for progress. If either of these tasks fails, we consider the entire step to fail. It is at this granularity that we can relate steps in the two languages.

```
type hstep tm -> tm -> links -> links -> subst -> subst -> o.
hstep T1 T2 L1 L2 S1 S3 :-
  (T1 ≃m T2) S1 S2,
  progress L1 L2 S2 S3.
```

Note that the infix notation `((A ≃m B) C D)` is syntactic sugar for `((≃m) A B C D)`.

Reconsidering links is a fixpoint process because the progress of a link can update the substitution, which may then enable another link to progress.

```
progress L L3 S S3 :-
  progress1 L L1 S S1,
  occur-check-links L1,
  scope-check L1 L2 S1 S2
  if (L = L2, S = S2) (L3 = L2, S3 = S2)
     (progress L2 L3 S2 S3).
```

Progress In the base compilation scheme, `progress1` is the identity function on both the links and the substitution, so the fixpoint trivially terminates. Sections 4.4 and 4.6 add rules to `progress1` and explain why the do not hinder termination.

Scope check The predicate `scope-check` replaces each link of the form $\Gamma \vdash X_{x_1 \ldots x_n} =_\eta t$ with a link $x_1 \ldots x_n \vdash X_{x_1 \ldots x_n} =_\eta t'$ whenever $\{x_1 \ldots x_n\} \subset \Gamma$. The term $t' = \lambda x.Y_{x_1 \ldots x_n\ x}$ is obtained after executing $\lambda x.Y_{x_1 \ldots x_n\ x} \simeq_m t$ using a fresh Y. This unification ensures that t' cannot contain variables in $\Gamma / \{x_1 \ldots x_n\}$, and if it fails then progress hence step$_m$ fails. In turn this grants fidelity in the case where the lhs of an η−`link` is pruned of a name that occurs (rigidly) in t: links represent suspended unification problems *that may succeed*.

Occur check Since compilation moves problematic terms out of the sight of $\simeq_m$, that procedure can only perform a partial occur check. For example, the unification problem $X \simeq_m f\ Y$ cannot generate a cyclic substitution alone, but should be disallowed if $\mathbb{L}$ contains a link like $\vdash Y =_\eta \lambda z.X_z$: we don't know yet if Y will feature a lambda in head position, but we surely know it contains X. The procedure `occur-check-links` is in charge of performing this check that is needed in order to guarantee proposition 4.1.3 (SIMULATION FIDELITY).

4.2.5 Substitution decompilation

Decompiling the substitution involves three steps.

First and foremost, problematic terms stored in $\mathbb{L}$ have to be moved back into the game: a suspended link must be turned into a valid assignment. This operation is possible thanks to invariant 4.2.2 (LINK LEFT HAND SIDE), and that no link causes an occur-check (4.2.4) and the fact that $\mathbb{L}$ is duplicate-free (see section 4.5).

The second step allocates in the memory of $\mathcal{O}$ new variables used to implement the pruning of higher-order variables. For example, $F \cdot x \cdot y = F \cdot x \cdot z$ requires allocating a variable G in order to express the assignment $F_{ab} \mapsto G_a$.

The final step is to decompile each assignment. The `val` node carries a term that is easy to decompile since $\mathbb{M}$ is a bijection. Since the `o-lam` node carries no additional information (other than the function body), each `abs` node can be decompiled to a `o-lam` one.

However, if the `o-lam` node was carrying more information, as is the case for Coq where it holds the type of the bound variable, one would need to store this piece of information in the memory map, were we store the arity (that is a very simple function type).

Proposition 4.2.3 (Compilation round trip). *If $\langle s \rangle \mapsto (t, m, l)$ and $l \in \mathbb{L}$ and $m \in \mathbb{M}$ and $\sigma = \{A \mapsto t\}$ and $X \mapsto A^n \in \mathbb{M}$ then $\langle \sigma, \mathbb{M}, \mathbb{L} \rangle^{-1} \mapsto \rho$ and $\rho X =_o \rho s$.*

We omit to sketch the proof of this property for brevity.

4.2.6 Definition of $\simeq_o$ and its properties

We already have all the ingredients to show the code of $\simeq_o$.

```
type (≃o) to -> to -> fsubst -> o.
(A ≃o B) F :-
  compile A A' [] M1 [] L1 [] S1,
  compile B B' M1 M2 L1 L2 S1 S2,
  hstep A' B' L2 L3 S2 S3,
  decompile M2 L3 S3 [] F.
```

So far the compiler is very basic. It does not really enforce that the terms passed to step_m are in $\mathcal{W}$, and indeed makes no use of the higher-order capabilities of the meta language (all generated variables have an empty scope). Still, we can prove that $\simeq_o$ is a good "first-order" unification algorithm if the input already happens to be in $\mathcal{W}$. Later, when the compiler will ensure proposition 4.1.5 ($\mathcal{W}$-ENFORCING), the proof will be refined to cover for the new cases.

Theorem 4.2.4 (Properties of $\simeq_o$ in $\mathcal{W}$). *The implementation of $\simeq_o$ above is a good unification for $=_o$ (as per proposition 4.1.1) in the domain $\mathcal{W}$.*

Proof sketch.

In this setting, $=_m$ is as strong as $=_o$ on ground terms. What we have to show is that whenever two different $\mathcal{O}$ terms can be made equal by a substitution ρ, we can produce this ρ by finding a σ via $\simeq_m$ on the corresponding $\mathcal{M}$ terms and by decompiling it. If we look at the syntax of $\mathcal{O}$, the only interesting case is «`o-uva X` $\simeq_o$ `s`». In this case, after compilation, we have «`uva Y []` $\simeq_m$ `t`» that succeeds with $\sigma = \{Y \mapsto t\}$ and σ is decompiled to $\rho = \{X \mapsto s\}$ by proposition 4.2.3.

□

Theorem 4.2.5 (Fidelity in $\mathcal{W}$)**.** *Proposition 4.1.2 (*RUN EQUIVALENCE*) and proposition 4.1.3 (*SIMULATION FIDELITY*) hold if* $\mathcal{W}(\mathbb{P})$.

Proof sketch.

Since `progress1` is a no-op then step_o and step_m are the same. By theorem 4.2.4, $\simeq_m$ is equivalent to $\simeq_o$ in $\mathcal{W}$.

□

4.2.7 Notational conventions

In the following sections we use the following notation for input and output of the compiler. $\mathbb{P}$ represents the input unification problems in $\mathcal{O}$ while $\mathbb{Q}$ their corresponding compiled version with memory mapping $\mathbb{M}$ and links $\mathbb{L}$. For example:

$$\begin{array}{llllllll}
\mathbb{P} = \{ & p_1 & \simeq_o & p_2 & p_3 & \simeq_o & p_4 & \} \\
\mathbb{Q} = \{ & t_1 & \simeq_m & t_2 & t_3 & \simeq_o & t_4 & \} \\
\mathbb{M} = \{ & X_1 \mapsto A_1^n & & & X_2 \mapsto A_2^m & & & \} \\
\mathbb{L} = \{ & \Gamma \vdash a =_\eta b & & & & & & \}
\end{array}$$

We index each sub-problem, sub-mapping, and sub-link with its position starting from 1 and counting from left to right, top to bottom. For example, $\mathbb{Q}_2$ corresponds to the $\mathcal{M}$ problem $t_3 \simeq_m t_4$.

As we deal with each category of problematic terms we weaken the assumptions of theorem 4.2.5 using the following definitions:

Definition 4.2.1 ($\mathcal{W}_\beta$)**.** $\mathcal{W}_\beta(X) \Leftrightarrow \forall t \in \mathcal{P}(X), t \notin (\Diamond\eta \cup \Diamond\mathcal{L})$

Definition 4.2.2 ($\mathcal{W}_{\beta\eta}$)**.** $\mathcal{W}_{\beta\eta}(X) \Leftrightarrow \forall t \in \mathcal{P}(X), t \notin \Diamond\mathcal{L}$

4.3 Handling of $\Diamond\beta$

The basic compiler given in the previous section is unable to make the following higher-order unification problem succeed.

$$\begin{array}{lllll}
\mathbb{P} = \{ & \lambda x.(f \cdot (X \cdot x) \cdot a) & \simeq_o & \lambda x.(f \cdot x \cdot a) & \} \\
\mathbb{Q} = \{ & \lambda x.(f \cdot (A \cdot x) \cdot a) & \simeq_m & \lambda x.(f \cdot x \cdot a) & \} \\
\mathbb{M} = \{ & X \mapsto A^0 & & & \}
\end{array}$$

The unification problem $\mathbb{Q}_1$ fails while trying to unify $A \cdot x$ and x, which is equivalent to «`app [uva A [], x]`» versus `x`. In order to exploit the higher-order unification algorithm of the meta language, we compile the $\mathcal{O}$ term $X \cdot x$ into the $\mathcal{M}$ term A_x.

4.3.1 Compilation and decompilation

We add the following rule before rule $r_@$ in 4.5, where `pattern-fragment` is a predicate checking if a list of terms is a list of distinct names.

```
comp (o-app [o-uva A|Ag]) (uva B Ag1) M1 M2 L L S1 S2 :-
  pattern-fragment Ag, !,
    fold6 comp Ag Ag1 M1 M1 L L S1 S1,
    len Ag Arity,
    m-alloc (ov A) (mv B (arity Arity)) M1 M2 S1 S2.
```

Note that compiling `Ag` cannot create new mappings nor links, since `Ag` is made of bound variables, and the hypothetical rule in 4.6 loaded by `comp-lam` grants this property.

Decompilation No change to `commit-link` since no link is added.

Progress No change to `progress` since no link is added.

Lemma 4.3.1 (Properties of $\simeq_o$ in $\mathcal{W}_\beta$). *Given the updated compilation scheme, the $\simeq_o$ of section 4.2.6 is a good unification (as per proposition 4.1.1) for $=_o$ in the domain $\mathcal{W}_\beta$.*

Proof sketch.

If we look at the $\mathcal{O}$ terms, there is one more interesting case, namely `o-app[o-uva X|W]` $\simeq_o$ `s` when `W` are distinct names compiled to $\vec{w}$. In this case the $\mathcal{M}$ problem is $Y_{\vec{w}} \simeq_m t$ that succeeds with $\sigma = \{Y_{\vec{y}} \mapsto t[\vec{w}/\vec{y}]\}$, which in turn is decompiled to $\rho = \{Y \mapsto \lambda\vec{y}.s[\vec{w}/\vec{y}]\}$. Thanks to rule r_{β_l} in 4.3 we have $(\lambda\vec{y}.s[\vec{w}/\vec{y}])\ \vec{w} =_o s$.

□

Lemma 4.3.2 ($\mathcal{W}$-partial-enforcement). $\forall s, \langle s\rangle \mapsto (t,m,l)$, *if* $\mathcal{W}_\beta(s)$ *then* $\mathcal{W}(t)$.

In other words the compiler eliminates all subterms that are in $\Diamond\beta$.

Theorem 4.3.3 (Fidelity in $\mathcal{W}_\beta$). *Proposition 4.1.2 (RUN EQUIVALENCE) and proposition 4.1.3 (SIMULATION FIDELITY) hold if* $\mathcal{W}_\beta(\mathbb{P})$.

Proof sketch.

Thanks to lemma 4.3.2, $\simeq_m$ is as powerful as $\simeq_o$ in $\mathcal{W}_\beta$, as well as in $\mathcal{W}$ by theorem 4.2.5.

□

4.4 Handling of $\Diamond\eta$

A term $\lambda x.t\cdot x$ is said to be the η-redex of t if x does not occur free in t, and conversely we say that t is the η-contraction of $\lambda x.t\cdot x$. The equational theory of $\mathcal{O}$ identifies these terms, but the current compilation scheme does not, as shown by the following example: while $\lambda x.X\cdot x \simeq_o f$ does admit the solution $\rho = \{\ X \mapsto f\ \}$, the corresponding problem in $\mathbb{Q}$ does not.

$$\begin{array}{lccc}
\mathbb{P} = \{ & \lambda x.(X\cdot x) & \simeq_o & f \quad \} \\
\mathbb{Q} = \{ & \lambda x.A_x & \simeq_m & f \quad \} \\
\mathbb{M} = \{ & X \mapsto A^1 & & \quad \}
\end{array}$$

The reason is that «`lam x\ uva A [x]`» and «`con"f"`» start with different term constructors.

In order to guarantee proposition 4.1.2, we detect lambda abstractions that can disappear by η-contraction (section 4.4.1), and we modify the compiler so that it generates fresh unification variables in their place and moves the problematic term from $\mathbb{Q}$ to $\mathbb{L}$ (section 4.4.2). The compilation of the problem $\mathbb{P}$ above is refined to:

$$\begin{array}{lllll}
\mathbb{P} = \{ & \lambda x.(X\cdot x) & \simeq_o & f & \} \\
\mathbb{Q} = \{ & A & \simeq_m & f & \} \\
\mathbb{M} = \{ & X \mapsto B^1 & & & \} \\
\mathbb{L} = \{ & \vdash A =_\eta \lambda x.B_x & & & \}
\end{array}$$

As per invariant 4.2.2 the η-`link` left-hand side is a variable while the right-hand side is a term in $\Diamond\eta$ that has the following property:

Invariant 4.4.1 (η-`link` rhs)**.** *The rhs of any η-`link` has the shape $\lambda x.t$ and t is in $\mathcal{W}$.*

Each η-`link` is kept in the link store $\mathbb{L}$ during execution and is activated under some conditions. Activation is implemented in section 4.4.3 by extending the `progress1` predicate defined in section 4.2.4.

4.4.1 Detection of $\Diamond\eta$

When compiling a term t, we need to determine if any subterm $s \in \mathcal{P}(t)$ that is of the form $\lambda x.r$ can be an η-redex, i.e. if there exists a substitution ρ such that $\rho(\lambda x.r) =_o s$. The detection of lambda abstractions that can "disappear" is not as trivial as it may seems. Here a few examples:

$$\begin{array}{lll}
\lambda x.f\cdot(A\cdot x) & \in \Diamond\eta & \rho = \{\ A \mapsto \lambda x.x\ \} \\
\lambda x.f\cdot(A\cdot x)\cdot x & \in \Diamond\eta & \rho = \{\ A \mapsto \lambda x.a\ \} \\
\lambda x.f\cdot x\cdot(A\cdot x) & \notin \Diamond\eta & \\
\lambda x.\lambda y.f\cdot(A\cdot x)\cdot(B\cdot y\cdot x) & \in \Diamond\eta & \rho = \{\ A \mapsto \lambda x.x\ ;\ B \mapsto \lambda y.\lambda x.y\ \}
\end{array}$$

The first two examples are easy, and show how a unification variable can expose or erase a variable in its scope, turning the resulting term into an η-redex or not.
The third example shows that when a variable occurs outside the scope of a unification variable, it cannot be erased and can hence prevent a term from being an η-redex.
The last example shows the recursive nature of the check we need to implement. The term starts with a spine of two lambdas, hence the whole term is in $\Diamond\eta$ iff the inner term $\lambda y.f\cdot(A\cdot x)\cdot(B\cdot y\cdot x)$ is in $\Diamond\eta$ itself. If it is, it could η-contract to $f\cdot(A\cdot x)$ making $\lambda x.f\cdot(A\cdot x)$ a potential η-redex.
We can now detail how $\Diamond\eta$ terms are detected.

Definition 4.4.1 (*may-contract-to*)**.** A β-normal term s *may-contract-to* a name x if there exists a substitution ρ such that $\rho s =_o x$.

Lemma 4.4.2. *A β-normal term $s = \lambda x_1 \ldots x_n.t$* may-contract-to *$x$ only if one of the following three conditions holds:*

1. *$n = 0$ and $t = x$;*
2. *t is the application of x to a list of terms l and each l_i* may-contract-to *x_i (e.g. $\lambda x_1 \ldots x_n.x\cdot x_1 \ldots x_n =_o x$) ;*

3. *t is a unification variable with scope W, and for any $v \in \{x, x_1 \dots x_n\}$, there exists a $w \in W$, such that w* may-contract-to *v (if $n = 0$ this is equivalent to $x \in W$).*

Proof sketch.

Since our terms are in β-normal form the only rule that can play a role is r_{η_l} in 4.3, and note that that rule uses `beta` to add an argument to an application or to the scope of a variable. If the term s is not exactly x (case 1) it can only be an η-redex of x, or a unification variable that can be assigned to x, or a combination of both. If s begins with a lambda, then the lambda can only disappear by η contraction. In that case, the term t under the spine of binders $x_1 \dots x_n$ can either be x applied to terms that *may-contract-to* these variables (case 2), or a unification variable that can be assigned to the application $x\,x_1 \dots x_n$ (case 3).

□

Definition 4.4.2 (*occurs-rigidly*)**.** A name x *occurs-rigidly* in a β-normal term t, if $\forall \rho, x \in \mathcal{P}(\rho t)$

In other words, x *occurs-rigidly* in t if it occurs in t outside of the scope of a unification variable X; otherwise, an instantiation of X can make x disappears from t. Note that η-contracting t cannot make x disappear, since x is not a locally bound variable inside t.

We can now describe the implementation of $\Diamond\eta$ detection:

Definition 4.4.3 (*maybe-eta*)**.** Given a β-normal term $s = \lambda x_1 \dots x_n.t$, *maybe-eta* s holds if any of the following holds:

1. t is a constant c or a name y applied to the arguments $l_1 \dots l_m$ such that $m \geq n$ and for every i such that $m - n < i \leq m$ the term l_i *may-contract-to* x_i, and no x_i *occurs-rigidly* in $l_1 \dots l_{m-n}\ y$;
2. t is a unification variable with scope W and for each x_i there exists a $w_j \in W$ such that w_j *may-contract-to* x_i.

Lemma 4.4.3 ($\Diamond\eta$ detection)**.** *If t is a β-normal term and $t \in \Diamond\eta$ then* maybe-eta *t holds.*

Proof sketch.

Follows from definition 4.4.2 and lemma 4.4.2

□

Remark that the converse of lemma 4.4.3 does not hold: there exists a term t satisfying the criteria (1) of definition 4.4.3 that is not in $\Diamond\eta$, i.e. there exists no substitution ρ such that ρt is an η-redex. A simple counter example is $\lambda x.f\,(A\,x)\,(A\,x)$ since x does not *occur-rigidly* in the first argument of f, and the second argument of f *may-contract-to* x. In other words $A\,x$ may either use or discard x, but our analysis does not take into account that *the same term* cannot have two contrasting behaviors.

As we will see in the rest of this section, this is not a problem since it does not break proposition 4.1.2 nor proposition 4.1.3.

4.4.2 Compilation and decompilation

Compilation The following rule is inserted just before rule r_λ in 4.5.

```
comp (o-lam F) (uva A Scope) M1 M2 L1 L3 S1 S3 :-
  maybe-eta (o-lam F) [], !,
  alloc S1 A S2,
  comp-lam F F1 M1 M2 L1 L2 S2 S3,
  get-scope (lam F1) Scope,
  L3 = [val (link-eta (uva A Scope) (lam F1)) | L2].
```

Whenever `o-lam F` is detected to be in $\Diamond\eta$ it is compiled to `lam F1` and replaced by the fresh variable `A`. This variable sees all the names free in `lam F1` and is connected to `lam F1` via a η`-link`. Invariant 4.2.2 (LINK LEFT HAND SIDE) holds for this link. Moreover:

Corollary 4.4.4. *The rhs of any* η`-link` *has exactly one lambda abstraction, hence the rule above respects invariant 4.4.1 (*η`-LINK` *RHS).*

Proof sketch.

By contradiction, suppose that the rule above is applied and that the rhs of the link is $\lambda x.\lambda y.t$, where x and y occur in t. If *maybe-eta* $\lambda y.t$ holds then the recursive call to `comp` (made by `comp-lam`) must have put a fresh variable in its place, so this case is impossible. Otherwise, if *maybe-eta* $\lambda y.t$ does not hold, then also *maybe-eta* $\lambda x.\lambda y.t$ does not hold either, contradicting the assumption that the rule was applied.

□

Lemma 4.4.5 ($\mathcal{W}$-partial-enforcement). $\forall s, \langle s \rangle \mapsto (t, m, l)$*, if* $\mathcal{W}_{\beta\eta}(s)$ *then* $\mathcal{W}(t)$*.*

Proof sketch.

By lemma 4.4.3

□

Decompilation The decompilation of a η`-link` is performed by unifying the lhs with the rhs. Note that this unification never fails, since the lhs is a flexible term not appearing in any other η`-link` (by definition 4.4.5).

4.4.3 Progress

η`-links` are meant to delay the unification of "problematic" terms until we know for sure if their head lambdas can be η-contracted.

Definition 4.4.4 (*η-progress-lhs*)**.** A link $\Gamma \vdash X =_\eta t$ is removed from $\mathbb{L}$ when X becomes rigid. Let $y \in \Gamma$. There are two cases:

1. if $X = a$ or $X = y$ or $X = f \cdot a_1 \ldots a_n$ we unify the η-redex of X with t, that is we run $\lambda x.X \cdot x \simeq_m t$

2. if $X = \lambda x.s$ we run $X \simeq_m t$.

Definition 4.4.5 (*η-progress-deduplicate*)**.** A link $\Gamma \vdash X_{\vec{s}} =_\eta T$ is removed from $\mathbb{L}$ when another link $\Delta \vdash X_{\vec{r}} =_\eta T'$ is in $\mathbb{L}$. By invariant 4.2.1 the length of $\vec{s}$ and $\vec{r}$ is the same; hence we can move the term T' from Δ to Γ by renaming its bound variables, i.e. $T'' = T'[\vec{r}/\vec{s}]$. We then run $T \simeq_m T''$ (under the context Γ).

Lemma 4.4.6. *Let $\lambda x.t$ be the rhs of a η*`-link`*, then $\mathcal{W}(t)$, enforcing invariant 4.4.1.*

Proof sketch.

By lemma 4.4.5.

□

Lemma 4.4.7. *Given a η*`-link` *$\Gamma \vdash X =_\eta \lambda x.t$, the unification done by* η-progress-lhs *is between terms in $\mathcal{W}$.*

Proof sketch.

Let σ be a substitution such that $\mathcal{W}(\sigma X)$ (since $\simeq_m$ preserves $\mathcal{W}$). By invariant 4.4.1, we have $\mathcal{W}(t)$. If $\sigma X = \lambda x.r$, then r is unified with t and both terms are in $\mathcal{W}$. Otherwise, $\lambda x.X \cdot x$ is unified with $\lambda x.t$, and $X \cdot x$ and t are both in $\mathcal{W}$.

□

Lemma 4.4.8. *The unification done by* η-progress-deduplicate *is between terms in $\mathcal{W}$.*

Proof.

Given two η`-links` $\Gamma_1 \vdash X =_\eta \lambda x.t$ and $\Gamma_2 \vdash Y =_\eta \lambda x.t'$ the unification is performed between t and t'. By lemma 4.4.5 both terms are in $\mathcal{W}$.

□

Lemma 4.4.9. *The progress of η*`-link` *guarantees proposition 4.1.7 ($\mathcal{W}$-PRESERVATION)*

Proof sketch.

By lemmas 4.4.7 and 4.4.8, every unification performed by the activation of a η`-link` is between terms in $\mathcal{W}$.

□

Lemma 4.4.10. `progress` *terminates.*

Proof sketch.

Rules 4.4.4 and 4.4.5 remove one link from $\mathbb{L}$, hence they cannot be applied indefinitely. Moreover each rule only relies on terminating operations such as $\simeq_m$, η-contraction, η-redex, relocation (a recursive copy of a finite term).

□

Theorem 4.4.11 (Fidelity in $\mathcal{W}_{\beta\eta}$)**.** *Given a list of unification problems $\mathbb{P}$ such that $\mathcal{W}_{\beta\eta}(\mathbb{P})$, if the memory map is bijective then the introduction of η*`-link` *guarantees proposition 4.1.3 (SIMULATION FIDELITY).*

Proof sketch.

We want to prove that step_o succeeds iff step_m succeeds, where step_m is made by a unification step u and a link progression p. Without loss of generality we consider a $\mathcal{O}$ unification problem of the form $s_1 \simeq_o s_2$ where $s_1 \in \Diamond\eta$ and is compiled to a variable X with the accessory link $\Gamma \vdash X =_\eta \lambda x.t_1$. The unification u always succeeds, since X is a fresh variable, we only have to consider the link progression p. Two cases should be analyzed:

$s_2 \in \Diamond\eta$, in this case, the unification problem becomes $X \simeq_m Y$ with the extra link $\vdash Y =_\eta \lambda x.t_2$. After the unification of X and Y, *η-progress-deduplicate* is fired and, once the lambdas are crossed, t_1 and t_2 are unified. This unification succeeds iff $s_1 \simeq_o s_2$ succeeds by theorem 4.3.3.

$s_2 \notin \Diamond\eta$, in this case s_2 is compiled to t_2 and the unification step u assigns t_2 to X and triggers *η-progress-lhs*. If s_1 and s_2 are equal thanks to r_{η_l} in 4.3 then progression p unifies $\lambda x.t_1$ with the η-redex of t_2 namly $\lambda x.t_2 \cdot x$, otherwise they are equal thanks to rule $r_{\lambda\lambda}$ in 4.3 and the link progress p unifies $\lambda x.t_1$ with t_2. Alternatively, if s_1 are different s_2 it cannot be because of the λ constructor in the head of t_1, since rule $r_{\lambda\lambda}$ in 4.3 or rule r_{η_l} in 4.3 move under it, eventually finding two different subterms s_1' and s_2'. By theorem 4.3.3, $\simeq_m$ fails on terms t_1' and t_2' even if *η-progress-lhs* η-expands t_2.

□

Corollary 4.4.12 (Properties of $\simeq_o$ in $\mathcal{W}_{\beta\eta}$)**.** *Given the updated compilation scheme, procedure $\simeq_o$ of section 4.2.6 is a good unification (as per proposition 4.1.1) for $=_o$ in the domain $\mathcal{W}_{\beta\eta}$.*

Example of *η-progress-lhs* The example at the beginning of section 4.4, once $\sigma = \{\ A \mapsto f\ \}$, triggers *$\eta$-progress-lhs* since the link becomes $\vdash f =_\eta \lambda x.B_x$ and the lhs is a constant. This rule runs $\lambda x.f \cdot x \simeq_m \lambda x.B_x$, resulting in $\sigma = \{\ A \mapsto f\ ;\ B_x \mapsto f\ \}$. Since X is mapped to B and B_x is decompiled into $\lambda x.f \cdot x$, by the definition of η`-link` decompilation σ is decompiled to $\rho = \{\ X \mapsto \lambda x.f \cdot x\ \}$.

4.5 Making $\mathbb{M}$ a bijection

We want to allow for partial application of functions. As a consequence the same unification variable X can be used multiple times with different arities. When it is the case the memory map $\mathbb{M}$ generated by the compiler is not a bijection. For example:

$$
\begin{array}{llccccccl}
\mathbb{P} = \{ & \lambda x.(X \cdot x) & \simeq_o & f & & X & \simeq_o & \lambda x.a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & f & & C & \simeq_m & \lambda x.a & \} \\
\mathbb{M} = \{ & X \mapsto C^0 & X \mapsto B^1 & & & & & & \} \\
\mathbb{L} = \{ & x \vdash A =_\eta \lambda y.B_y & & & & & & & \}
\end{array}
$$

in the unification problems $\mathbb{P}$ above, X is used with arity 1 in $\mathbb{P}_1$ and with arity 0 in $\mathbb{P}_2$. In order to preserve invariant 4.2.1 (UNIFICATION-VARIABLE ARITY) the compiler did generate two entries in $\mathbb{M}$ for X, namely B and C. However, there is no connection between these two variables and any incompatible assignements to them would not be detected, in turn breaking proposition 4.1.3. To address this issue, we post-process $\mathbb{M}$ to ensure the following property:

Proposition 4.5.1 ($\mathbb{M}$ is a bijection)**.** *After compilation, for each $\mathcal{O}$-variable X in $\mathbb{P}$ and for each $\mathcal{M}$-variable A in $\mathbb{Q}$ there is exactly one entry $X \mapsto A^n$ in $\mathbb{M}$.*

Note that running $\simeq_m$ may require allocating new variables in $\mathcal{M}$ as explained in section 4.2.5. The property above ignores these variables since they are fresh and have no corresponding variable in $\mathcal{O}$.

The core procedure of this post-processing step is *align-arity* that is iterated by *map-deduplication*:

Definition 4.5.1 (*align-arity*)**.** Given two mappings $m_1 : X \mapsto A^m$ and $m_2 : X \mapsto C^n$ where $m < n$ and $d = n - m$, *align-arity* $m_1\ m_2$ generates the following d links, one for each i such that $0 \le i < d$,

$$x_0 \dots x_{m+i} \vdash B^i_{x_0 \dots x_{m+i}} =_\eta \lambda x_{m+i+1}.B^{i+1}_{x_0 \dots x_{m+i+1}}$$

where B^i is a fresh variable of arity $m + i$, and $B^0 = A$ as well as $B^d = C$.

The intuition is that we η-expand the occurrence of the variable with lower arity to match the higher arity. Since each η`-link` can add exactly one lambda, we need as many links as the difference between the two arities.

Definition 4.5.2 (*map-deduplication*)**.** Forall mappings $m_1, m_2 \in \mathbb{M}$ such that $m_1 : X \mapsto A^m$ and $m_2 : X \mapsto C^n$ and $m < n$ we remove m_1 from $\mathbb{M}$ and add to $\mathbb{L}$ the result of *align-arity* $m_1\ m_2$.

Theorem 4.5.2 (Fidelity with *map-deduplication*)**.** *Given a list of unification problems $\mathbb{P}$, such that $\mathcal{W}_{\beta\eta}(\mathbb{P})$, if $\mathbb{P}$ contains the same $\mathcal{O}$-variable used at different arities, then* map-deduplication *guarantees proposition 4.1.3 (SIMULATION FIDELITY)*

Proof sketch.

Let X be a $\mathcal{O}$-variable appearing in two different subterms s_1 and s_2. In s_1 variable X is applied to the list of terms $x_1 \dots x_m$ while, in s_2, it is applied to $y_1 \dots y_n$. Without loss of generality we take $m < n$. After *map-deduplication* we have two distinct $\mathcal{M}$-variables for X, say A and C, related by a chain $\vec{c}$ of η`-links` of length $n - m$. We prove that if A and C are assigned respectively to the $\mathcal{W}$ terms $\lambda x_1 \dots x_p.t_1$ and $\lambda x_1 \dots x_q.t_2$, then a failure occurs iff these terms do not unify. By *η-progress-lhs*, the assignment of A activates p links of $\vec{c}$. In particular, when $p \ge n$, then the entire chain collapses and unification is performed between $\lambda x_n \dots x_p.t_1$ and $\lambda x_1 \dots x_q.t_2$. By theorem 4.3.3 and since we work with $\mathcal{W}$ terms, this unification succeeds iff it does in $\mathcal{O}$. Otherwise if $p < n$ then we have to consider two cases. 1) If t_1 is a rigid term then t_2 must be the η-redex of t_1, implying $q = 0$. If it is not the case, then *η-progress-lhs* would eventually cause a failure. 2) If t_1 is a unification variable then `scope-check` guarantees that t_2 does not use $x_1 \dots x_p$ as free variables, causing otherwise a unification failure. After this check, $m - n - p$ links of $\vec{c}$ remain suspended, as we expect: decompilation is charged to wake all links in $\mathbb{L}$ and eventually relate t_1 with t_2.

□

Example of *map-deduplication* If we take back the example given at the beginning of this section, *map-deduplication* produces the following result:

$$\begin{array}{llllllll}
\mathbb{P} = \{ & \lambda x.(X\cdot x) & \simeq_o & f & X & \simeq_o & \lambda x.a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & f & C & \simeq_m & \lambda x.a & \} \\
\mathbb{M} = \{ & X \mapsto B^1 & & & & & & \} \\
\mathbb{L} = \{ & \vdash C =_\eta \lambda x.B_x & & x \vdash A =_\eta \lambda y.B_y & & & & \}
\end{array}$$

Note that $X \mapsto C^0$ disappears from $\mathbb{M}$ in favour of the auxiliary η−link $\vdash C =_\eta \lambda x.B_x$. The resolution of $\mathbb{Q}_1$ assignes a to A. This wakes up $\mathbb{L}_2$ by *η-progress-lhs*, assigning «$f\cdot x$» to B_x. The resolution of $\mathbb{Q}_2$ instantiates C to $\lambda x.a$, which, in turn triggers $\mathbb{L}_1$ by *η-progress-lhs*. In turn, this performs $\lambda x.a \simeq_m \lambda x.(f\cdot x)$ that fails as expected.

4.6 Handling of $\Diamond\mathcal{L}$

As observed in [47] it is worth handling terms in $\Diamond\mathcal{L}$ since, in practice, these terms often re-enter $\mathcal{L}$ at runtime.

In the following example problem $\mathbb{P}_2$, namely $X\cdot a \simeq_o a$, admits two different solutions: $\rho_1 = \{ X \mapsto \lambda x.x \}$ and $\rho_2 = \{ X \mapsto \lambda x.a \}$. The unification algorithm alone cannot chose the right one, since no solution is more general than the other, but for the first problem the choice is obvious, and in turn this choice makes $\mathbb{P}_2 \subseteq \mathcal{L}$:

$$\begin{array}{llllllll}
\mathbb{P} = \{ & X & \simeq_o & \lambda x.a & (X\cdot a) & \simeq_o & a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & \lambda x.a & (A\cdot a) & \simeq_m & a & \} \\
\mathbb{M} = \{ & X \mapsto A^0 & & & & & & \}
\end{array}$$

In order to support this scenario we have to improve a little our compiler and progress routines. In particular $\mathbb{Q}_1$ generates $\sigma = \{ A \mapsto \lambda x.a \}$ making $\sigma\mathbb{Q}_2$ equal to $(\lambda x.a)\cdot a \simeq_m a$ that $\simeq_m$ cannot solve since it lacks rules r_{β_l} and r_{β_r} in 4.3.

To address this problem the compiler must recognize $\Diamond\mathcal{L}$ terms and replace them with fresh variables and generate a new kind of links that we call $\mathcal{L}$−`link`.

In addition to invariant 4.2.2 (LINK LEFT HAND SIDE), the term on the rhs of a $\mathcal{L}$−`link` has the following property:

Invariant 4.6.1 ($\mathcal{L}$−`link` rhs)**.** *The rhs of any $\mathcal{L}$−`link` has the shape $X_{s_1\ldots s_n}\cdot t_1 \ldots t_m$ where X is a unification variable in $\mathcal{L}$ (with scope $s_1 \ldots s_n$) and $t_1 \ldots t_m$ is a list of terms such that $m > 0$ and t_1 is either a variable occurring in $s_1 \ldots s_n$ or a term other than a variable.*

Note that the shape of such as rhs is «`app [uva X S | L]`», where `S` is `[s₁, ..., sₙ]` and `L` is `[t₁, ..., tₘ]`.

4.6.1 Compilation and decompilation

We insert this rule just before rule $r_@$ in 4.5

```
comp (o-app [o-uva A|Ag]) (uva B Sc) M1 M3 L1 L3 S1 S4 :- !,
  pattern-fragment-prefix Ag Pf Extra, alloc S1 B S2,
  len Pf Ar, m-alloc (ov A) (mv C (arity Ar)) M1 M2 S2 S3,
  fold6 comp Pf    Pf1    M2 M2 L1 L1 S3 S3,
```

```
fold6 comp Extra Extra1 M2 M3 L1 L2 S3 S4,
Beta = app [uva C Pf1 | Extra1],
get-scope Beta Sc,
L3 = [val (link-llam (uva B Sc) Beta) | L2].
```

The list `Ag` is split into two parts: `Pf` is in $\mathcal{L}$, and can be empty; `Extra` cannot be empty and is such that «`append Pf Extra Ag`». The rhs of the $\mathcal{L}$`-link` is the application of a fresh variable `C` having in scope all names in `Pf1` (the compilation of `Pf`). The variable `B`, returned as the compiled term, is a fresh variable having in scope all the free variables occurring in `Pf1` and `Extra1` (the compilation of `Extra`). This construction enforces invariant 4.6.1 ($\mathcal{L}$-LINK RHS).

Lemma 4.6.2 ($\mathcal{W}$-enforcement)**.** *$\forall s, \langle s \rangle \mapsto (t, m, l)$, then $\mathcal{W}(t)$.*

Proof sketch.

By lemmas 4.3.2 and 4.4.5 and the rule above.

□

Decompilation All $\mathcal{L}$`-link` should be solved before decompilation. If any $\mathcal{L}$`-link` remains in $\mathbb{L}$, decompilation fails.

4.6.2 Progress

We add the followins rules to `progress1`. Let l be a $\mathcal{L}$`-link` of the form $\Gamma \vdash T =_{\mathcal{L}} X_{s_1 \ldots s_n} \cdot t_1 \ldots t_m$

Definition 4.6.1 (*$\mathcal{L}$-progress-refine*)**.** Let σ be a substitution such that σt_1 is a name s not occurring in $s_1 \ldots s_n$. If $m = 1$, then l is removed and the lhs is unified with $X_{s_1 \ldots s_n\ s}$. If $m > 1$, then l is replaced by the link $\Gamma \vdash T =_{\mathcal{L}} Y_{s_1 \ldots s_n\ s} \cdot t_2 \ldots t_m$ (where Y is a fresh variable of arity $n+1$) and link $\Gamma \vdash X_{s_1 \ldots s_n} =_{\eta} \lambda x. Y_{s_1 \ldots s_n\ x}$ is added to $\mathbb{L}$.

Definition 4.6.2 (*$\mathcal{L}$-progress-rhs*)**.** Link l is removed from $\mathbb{L}$ if $X_{s_1 \ldots s_n}$ is instantiated to a term t and $t \cdot t_1 \ldots t_m$ β-reduces to a $t' \in \mathcal{L}$.

Definition 4.6.3 (*$\mathcal{L}$-progress-fail*)**.** progress fails when either

there exists another link $l' \in \mathbb{L}$ with the same lhs as l; or

the lhs of l become rigid.

We relax this condition and accommodate for heuristics in 4.6.3.

Finally we introduce the following η`-link` progress rule.

Definition 4.6.4 (*η-progress-rhs*)**.** A link $\Gamma \vdash X =_{\eta} T$ is removed from $\mathbb{L}$ when either

maybe-eta T does not hold (anymore), in this case X is unified with T; or

T η-contracts to a term T' not starting with the `lam` constructor. In this case X is unified with T'

The idea behind this rule is to instantiate the lhs variable of a η−`link` whenever its rhs is for sure a $\mathcal{W}$ term.

Lemma 4.6.3. `progress` *terminates*

Proof sketch.

Let $l = \Gamma \vdash T =_{\mathcal{L}} X_{s_1 \ldots s_n} \cdot t_1 \ldots t_m$ a $\mathcal{L}$−`link` in the store $\mathbb{L}$. The progression made by $\mathcal{L}$-*progress-rhs* terminates: l is removed from $\mathbb{L}$ after having performed terminating instructions. If l is not activated by $\mathcal{L}$-*progress-rhs*, then X is a variables. The activation of $\mathcal{L}$-*progress-refine* replaces l with the new $\mathcal{L}$−`link` $\Gamma \vdash T =_{\mathcal{L}} Y_{s_1 \ldots s_n\ s} \cdot t_2 \ldots t_m$. This progression can be triggered at most m times, since at each time the number of terms applied to X decreases. In the end l_m will be removed from $\mathbb{L}$ after the unification between lhs with rhs. We also note that each $\mathcal{L}$-*progress-refine* generates a η−`link`, yet, according to lemma 4.4.10, this does not affect termination. $\mathcal{L}$-*progress-fail* terminates since it stop progression with failure. Finally, η-*progress-rhs* performs terminating operations and, if its permises succeed, the considered η−`link` is removed from $\mathbb{L}$, ensuring termination.

□

Theorem 4.6.4 (Fidelity in $\mathcal{O}$)**.** *The introduction of* $\mathcal{L}$−`link` *guarantees proposition 4.1.4 (*FIDELITY RECOVERY*) if* $\mathbb{P} \notin \mathcal{L}$.

Proof sketch.

Let $\mathbb{P}_i$ be the first problem in $\mathbb{P}$ such that it is not in $\mathcal{L}$ and let σ be the substitution obtained solving $\mathbb{Q}_1 \ldots \mathbb{Q}_{i-1}$. After the execution of step$_m$ for the $i-1$ time, each variable X corresponding to a $\diamond\eta$ subterm in the original $\mathcal{O}$ unification problem, is instantiated iff it is known for sure that X is no more in $\diamond\eta$. If $\sigma\mathbb{Q}_i$ is in $\mathcal{L}$, then by definitions 4.6.1 and 4.6.2 the associated $\mathcal{L}$−`link` is solved and removed. In this case, all calls to $\simeq_m$ are between terms in $\mathcal{L}$ and by theorem 4.3.3 fidelity is guaranteed. If $\sigma\mathbb{Q}$ is still in $\diamond\mathcal{L}$, then by definition 4.6.3 unification fails as the corresponding unification in $\mathcal{O}$ would (it is called outside its domain).

□

4.6.3 Relaxing definition 4.6.3 ($\mathcal{L}$-*progress-fail*)

Working with terms in $\mathcal{L}$ is sometime too restrictive [1] and we could find in literature a few strategies to go beyond $\mathcal{L}$ without implementing Huet's algorithm [34]. Some implementations of λProlog [48] such as Teyjus [47] delay the resolution of $\diamond\mathcal{L}$ unification problems until the substitution makes them reenter $\mathcal{L}$. Other systems apply heuristics like preferring projection over mimic, and commit to that solution. This is the case for the unification algorithm Coq uses in its type-class solver [58],

In this section we show how we can implement these strategies by simply adding (or removing) rules to the `progress` predicate. In the example below $\mathbb{P}_1$ is in $\diamond\mathcal{L}$.

$$\begin{array}{lllllllll}
\mathbb{P} = \{ & (X \cdot a) & \simeq_o & a & & X & \simeq_o & \lambda x.Y & \} \\
\mathbb{Q} = \{ & A & \simeq_m & a & & B & \simeq_m & \lambda x.C & \} \\
\mathbb{M} = \{ & Y \mapsto C^0 & & X \mapsto B^0 & & & & & \} \\
\mathbb{L} = \{ & x \vdash A =_{\mathcal{L}} (B \cdot a) & & & & & & & \}
\end{array}$$

If we want the object-language unification to delay the first unification problem (waiting for X to be be instantiated), we can relax definition 4.6.3. Instead of failing when the lhs of $\mathbb{L}_1$ becomes rigid (equal to a), we keep it in $\mathbb{L}$ until the head of its rhs also become rigid. In this case, since both the lhs and rhs have rigid heads, they can be unified. While this relaxed rule does not break proposition 4.1.3 (SIMULATION FIDELITY) per se, the `occur-check-links` procedure becomes incomplete since invariant 4.2.2 (LINK LEFT HAND SIDE) is broken. Also note that delaying unification outside $\mathcal{L}$ can leave $\mathcal{L}$`-link` for the decompilation phase. Therefore `commit-links` should be modified accordingly.

If instead we want $\simeq_o$ to follow the second strategy and pick an arbitrary solution we can modify progress by applying the desired heuristic instead of failing. For instance, in $X \cdot a \cdot b = Y \cdot b$, the last argument of the two terms is the same and unification can succeed by assigning Xa to Y. This heuristic is used by [58].

4.7 Actual implementation in Elpi

In this paper we did study a compiler and a simulation loop on a minimal language. The actual implementation uses the Coq-Elpi meta language to both compile the sequence of problems (the rules) and execute them, that is Elpi plays the role of $\mathcal{M}$ as well.

The main difference is that we cannot implement run_m since it is the runtime of the programming language. In particular the runtime iterates $\simeq_m$, but step_m also needs to check links for progress. Luckily Elpi extends λProlog with constraints (suspended goals) and Constraint Handling Rules (CHR) to operate on them [29, 22]. Similarly to [43] a constraint is a goal suspended on a list of variables that is resumed *as soon as one of these variable is assigned*, before any other existing goal is considered. In turn link activation grants proposition 4.1.3 (SIMULATION FIDELITY). We point out that [43] delays $\Diamond\mathcal{L}$ unification problems while we also delay problems that are in $\Diamond\eta$. For brevity we only provide two pseaudo-code snippets. The first one depicts how η`-link` are suspended or make progress.

```
link-eta L R :- not (var L), !, eta-progress-lhs L R.
link-eta L R :- not (maybe-eta R), !, eta-progress-rhs L R.
link-eta L R :- declare_constraint (link-eta L R) [L,R].
```

The second snippet illustrates the deduplication of η`-link`. The syntax «`N ▷ G ?- P`» denotes a λProlog sequent, that is a goal `P` under a set `G` of hypothetical rules (introduced by `=>`) and where the program context binds (via the `pi` operator) eigenvariables in `N`. Sequents are presented to CHR with their higher order unification variables replaced by "frozen constants", that is A_{xyz} becomes «**uvar** f_A `[x,y,z]`» for a fresh constant f_A. Frozen constants are "defrost" when they are part of a new goal (see [29, section 4.3]).

```
rule (N1 ▷ G1 ?- link-eta (uvar X LX1) T1)    % match
  /  (N2 ▷ G2 ?- link-eta (uvar X LX2) T2)    % remove
  |  (relocate LX1 LX2 T2 T2')               % condition
 <=> (N1 ▷ G1 ?- T1 = T2').                   % new goal
```

The first directive matches a constraint whose lhs is a variable `X`; the second matches and removes a constraint on the same variable; the third relocates `T2` and the latter crafts the new goal that unifies `T1` with `T2'` under `G1` and the set of heigenvariables `N1`.

4.8 Related work and conclusion

Object-language terms can be unified using different strategies.

The first approach that comes to mind consist in implementing $\simeq_o$ as a regular routine, i.e. write rules as follows

```
decision X :- unify X (all A x\ app [P, x]), finite A,
  pi x\ decision (app [P, x]).
```

where `unify` is a regular predicate. This method fails to take advantage of the logic programming engine provided by the meta language since it removes all the data from the rule's heads and makes indexing degrade. Additionally, implementing a unification procedure in the meta language is likely to be significantly slower compared to the built-in one on the common domain of first order terms.

Another possibility is to avoid having application and abstraction nodes in the syntax tree, and use the meta language ones, as in:

```
decision (all A x\ P x) :- finite A, pi x\ decision (P x).
```

However, this encoding has two big limitations. First it is not always feasible to adopt it for Coq due to the fact that the type system of the meta language is too limited to accommodate the one of the object language, e.g. Coq can typecheck variadic functions [8].
Second, the encoding of Coq terms provided by Elpi is primarily used for meta programming, i.e. to extend the Coq system. Consequently, it must be able to manipulate terms that are not known in advance without relying on introspection primitives such as Prolog's `functor/3` and `arg/3`. To avoid that constants cannot be symbols of the meta language, but must rather live in an open world, akin to the `string` data type used in the `con` constructor.

In the literature we could find a related encoding of the Calculus of Constructions (CC) [16]. The goal of that work is to exhibit a logic program performing proof checking for CC and hence relate the proof system of intuitionistic higher-order logic (that animates λProlog programs) with the one of CC. The encoding is hence tailored toward a different goal, for example it utilizes three relations to represent the equational theory of CC, and that choice alone makes things harder for us. Section 6 contains a discussion about the use of the unification procedure of the meta language in presence of non ground goals, but the authors are not interested in exploiting a decidable fragment of higher-order unification but are rather leaning towards an interactive system where the user is given control over the search procedure.

Another work that is, somewhat surprisingly, only superficially related to ours is the type-class engine built in Isabelle's meta language. In [70] classes identify (simple) types, they are not higher order predicates as in Coq, hence the solver does not require a higher-order unification procedure.

The approach presented in this paper addresses all the concerns mentioned earlier. It takes advantage of the unification capabilities of the meta language at the price of handling problematic sub terms on the side. As a result our encoding takes advantage of indexing data structures and mode analysis for clause filtering. It is worth mentioning that we replace terms with variables only when it is strictly needed, leaving the rest of the term structure intact and hence indexable. Some preliminary benchmarks on the Stdpp library [35] show encouraging results: our type-class solver has some overhead on small goals but is consistently faster than the Coq one starting from goals made of 32 nodes. Moreover our approach is flexible enough to accommodate different strategies and heuristics to handle terms outside the pattern fragment. Finally, our encoding is sufficiently shallow to: 1) allow to lift forms of static analysis for the meta language, such as determinacy, also to the object language; 2) take advantage of future improvements to the logic-programming engine of Elpi, such as tabled search.

We considered mechanizing our results with Abella [24], especially in the early phase of this work, but unfortunately we could not. The main reason is that the logic of Abella does not let one quantify over predicates nor use the cut operator. Eliminating cut or higher order combinators like `map` or `forall2` is surely possible, but has a high cost in verbosity given how pervasively they are used. Moreover, and more importantly, it creates a distance between the verified and the actual code, which we found to hinder our exploration. However, we did establish a test suite with about a hundred cases. It is available at https://github.com/FissoreD/ho-unif-for-free.

Chapter 5

Determinacy checking for *Elpi*

5.1 Introduction

Elpi is a higher-order logic programming language, a dialect of λProlog, particularly well suited for manipulating syntax trees with binders (see [44, 45, 15, 29]). It is used as an extension language for the Rocq Prover (formerly the Coq proof assistant) and has become an important piece of infrastructure: several projects and libraries depend on Elpi [36, 7, 9, 66, 19, 20]. Examples include the Hierarchy-Builder library-structuring tool [10], which supports the Mathematical Components ecosystem [41] and mechanizations such as the Odd Order and Four Color theorems [26, 25], and Derive [61, 62, 28], a program-and-proof synthesis framework with industrial applications at BlueRock systems.

Over the years Elpi was put in the hands of quite a number of Rocq users and the most widely cited barrier to entry has been the paradigm shift to a logic programming one. Rocq users are comfortable with typed and functional programming but often lack experience with backtracking and its control. Uncontrolled backtracking can produce slower programs and complicate debugging.

To lower this barrier we design a system of determinacy signatures that allows programmers to declare when a predicate behaves like a function: at most one result is produced and the runtime will not consider alternative results. This notion of determinism is called *semidet* by Henderson et al. [33] and named *operationally deterministic* by Nakamura [49]. Alternative notions exist, for example Warren [13] considers predicates *observably deterministic* when they produce the same result multiple times or diverge. We chose operational determinacy since it matches the behavior of functions in mainstream functional languages.

Our determinacy checker covers Elpi's higher-order features, including higher-order predicates (for example map and fold, see section 2.2.4) and dynamic predicates that extend themselves at runtime (see section 2.2.5).

We apply these signatures to a large portion of publicly available Elpi code in the Rocq ecosystem and report our experience. Our findings indicate that most Elpi programs are intended to be backtracking-free and require only minimal signatures for the static analyzer we implemented to verify determinacy.

Contributions 1) we design a language of signatures to assert the determinacy of predicates that covers higher-order logic programming constructs such as first-class predicates and dynamic predicates; 2) we implement a static analyzer for that language; 3) we apply the static analyzer to most of the Elpi code available in the Rocq ecosystem.

5.2 Elpi and determinacy signatures by examples

As in most logic programming languages, Elpi programs are organized in rules. When rules are not *mutually exclusive*, i.e. multiple rules apply on the same query, the runtime continues the computation using the rule with highest priority. Moreover it stores the other rules as a *choice point* and can backtrack to that point to continue the computation. What makes backtracking tricky to Elpi users is that it is an opt-out feature of logic programming: it is left to the programmer to tell the runtime to commit to one rule and discard the choice point, rather than the other way around. If the user forgets to commit to a rule, his program still works but becomes much harder to debug and typically slower to run in case of failures.

The simplest example is the list-membership *relation* that can succeed more than once if the element we are looking for occurs multiple times in the list:

```
mem X [Y|_ ] :- X = Y.     % the head is the X we are looking for, or
mem X [_|YS] :- mem X YS. % recurse on the tail
```

In the code above each line represents a rule. By convention, unification variables are capitalized, and following the λ-calculus tradition application is written without parentheses, i.e. $f\ x$ rather than $f(x)$. Square brackets are used for lists with pipe separating the head from the tail. The `:-` symbol separates the head of a rule from its premises. Rules are tried in the order of their declaration by unifying their head with the current query.

The first rule for `mem` is the base case: it succeeds if the head of the list is equal to the `X` we are looking for. The second one discards the list's head and continues looking for `X` in the tail. A query such as `mem 2 [2,3,2]` can be solved in two different ways: either using rule 1 immediately, or using rule 2 twice to discard the first two items, and then conclude with rule 1.

Elpi users often have a background in functional programming and simply do expect `mem` to be a *function* testing membership, and not a relation. Moreover one needs to look, carefully, at how `mem` is implemented in order to see the peril. This misconception becomes a problem when they use `mem` in a larger piece of code and observe its execution.

```
main L :- code_before L, mem 1 L, code_after L.
```

For example, in the code above, one may see `code_after` being executed multiple times: each choice point left by `mem` enables the runtime to backtrack to that point in time and continue from there. Even worse, if `code_after` is expensive and fails, one pays its cost multiple times.

It must be said that backtracking can be very useful in some situations and that a programmer with experience in logic programming can take advantage of that. We want the Elpi casual user to be able to write code without mastering all that, and simply declare that they expect *their code* to compute functions.

```
func main list int -> . % the signature for main
```

The signature starts by a keyword declaring the determinacy of the predicate: **`func`** for deterministic predicates, that we also call functions, and **`pred`** for any predicate, including both functions and relations. Inputs are separated from outputs by the arrow symbol, and in the case of `main` there is no output.

```
main L :- code_before L, mem 1 L, code_after L.
%                            ^^^^^^ error: non-deterministic atom
```

Since `main` was declared as a function, our static analysis invites the user to discard the choice points left by `mem`, for example by inserting a `!` (a *cut* directive) after it. Assuming `code_after` is a functions, this code is accepted, since the cut discards all choice points generated by `mem` and `code_before L`.

```
main L :- code_before L, mem 1 L, !, code_after L. % OK
```

Note that the programmer may also write a functional membership test as follows, and use it in functional code with no additional safeguard. In this case the cut discards the second rule as soon as `X = Y` succeeds.

```
func in int, list int -> .
in X [Y|_ ] :- X = Y, !. % commit to this rule, discard the next one
in X [_|YS] :- in X YS.

main L :- code_before L, in 1 L, code_after L. % OK
```

Tracking the use of backtracking code statically is a well known technique in logic programming called determinacy analysis [13, 57]. What the literature lacks is a treatment of Elpi features not available in plain Prolog, see 2.2.4 and 2.2.5.

5.3 Elpi's abstract syntax and rule selection

Any static analysis to rule out non-determinism must depend on the operational semantics of the language. In particular the order in which rules are applied and the order in which rule premises are executed directly impact the scope of the cut directive.

Here we specify the relevant aspects of Elpi. Its abstract syntax is given below.

We assume a set $\mathbb{P}$ of predicate names, a set $\mathbb{K}$ of term constructors, and a set $\mathbb{U}$ of unification variables. We call the union of these three syntactic categories *symbols*. In the syntax of atoms $\mathbb{A}$ we find predicate calls ($\mathbb{C}$), the `Cut` operator (explained in section 5.4) and the `pi` operator.[*] The arguments of predicate calls are in the $\mathbb{T}\mathrm{m}$ nonterminal, for terms, that includes predicate calls along with first order data construction. Because of space constraint we omit from the syntax the construction of data with binders since it plays no interesting role in the static analysis of determinacy. We reserve s for symbols ($\mathbb{P} \cup \mathbb{K} \cup \mathbb{U}$), v for variables, k for terms in $\mathbb{P} \cup \mathbb{K}$, a and b for atoms, c for callables and r for rules. Given a rule $h\ \texttt{:-}\ b_1 \ldots b_n$ we call h the *head*, $b_1 \ldots b_n$ the *body* and each b_i a *body atom*. We use non-terminals to represent the class of objects they identify; for example, $\mathbb{T}\mathrm{m}$identifies (or is the type of) terms. We use $\mathbb{P}$ for the type of programs, being lists of rules in $\mathbb{R}$, and we reserve t for terms and $\mathcal{P}$ for programs.

Rule priority Each rule in a program has a unique identifier called priority; this priority stems from the order in which the rule should be applied at runtime. There exists a unique total order for rules following their priority. When a rule is applied, its body atoms are executed in order.

This selection strategy is called SLD and is the standard one for logic programs. In this strategy non-determinism arises when: 1) at least two rules can be used (successfully) to the *same query* (i.e., lack of mutual exclusion between rules, see section 5.4); 2) the call to a non-deterministic predicate, or the miscall of a deterministic one, is not followed by a cut (see section 5.5).

[*] In reality, in Elpi, `pi` and `=>` are two separate operators, generating a fresh symbol and loading a rule respectively, but for simplicity we only consider their combination.

5.4 Hard Cut and Mutual Exclusion

We focus on how to address the first source of non-determinism.

In order to precisely describe `Cut`, we need to introduce the notion of choice points, or alternatives. The type of an alternative $\mathbb{L}$ is equal to $\mathbb{\Sigma} \times \overrightarrow{goal}$, that is a substitution and a list of goals. The type of a goal goal is equal to $\mathbb{P} \times \mathbb{A}$, that is a program and a query (an atom). Since the program can dynamically change due to `pi`, each (sub)goal must contain the program in which it should be solved.
A substitution σ is a partial mapping from unification variables $\mathbb{U}$ to terms $\mathbb{Tm}$. We call this function space $\mathbb{\Sigma}$ and we write ε for the empty substitution. We write $\sigma \cdot t$ for the application of a substitution σ to a term t.

Definition 5.4.1 (Hard cut)**.** Given a rule for predicate k with priority p and body $b_1 \dots b_i$ `Cut ...`, the effect of reaching `Cut` is that the runtime discards all alternatives generated by $b_1 \dots b_i$ and all rules for k with lower priority than p.

Elpi applies rules by matching the input arguments, like B-Prolog's "matching clauses" [71], and by unifying the output ones using Miller's algorithm [44].

Detto sopra

Definition 5.4.2 (Unification / Matching)**.** Two terms t and u unify if there exists a substitution σ such that $\sigma \cdot t = \sigma \cdot u$. We say that a term t matches (the pattern) u if there exists a substitution σ such that $\sigma \cdot u = t$ and the domain of σ is the free variables of u

Finally, in the next definition we distinguish between global rules, i.e., rules that are part of the program $\mathcal{P}$, and local rules that are loaded via the `pi` operator. For example, the `copy` predicate from section 2.2.5 has two global rules and one local rule.

Definition 5.4.3 (Mutual exclusion (mut_excl $\mathbb{P}$ $\mathbb{R}$))**.** Given a program $\mathcal{P}$ and a rule r, such that $r \notin \mathcal{P}$, we say that r is mutually exclusive with the rules in $\mathcal{P}$ if it has the highest priority and one of the following additional conditions holds:
(a) there is a `Cut` in the body of r, or
(b) r is a gloabl rule and all rules r' in $\mathcal{P}$ have an input argument not unifying with the corresponding input in r, or
(c) r is a local rule introduced via `pi` x r b, x is an input of r and for all rules r' in the program, the corresponding input is not a variable

For example, the rules for predicate `p` below are not mutually exclusive, so choice points may remain after a query resolution, hence introducing non-determinism. For instance, `p 1 X` has two solutions, namely `X = 3` and `X = 2`.

```
pred p int -> int.
p 1 3.
p 1 2.
```

```
func q int -> int.
q 1 3.
q 2 4.
```

```
func r int -> int.
r X R :- p X R, !.
r X R :- q X R.
```

Conversely, `q` satisfies mutual exclusion: no query can trigger both rules. Note that, since inputs are matched, a query like `q X Y` has no solution in Elpi. Standard Prolog implementation would reject the query as not well moded since the input `X` is not ground. Both rules for `r` match any input. If the first rule succeeds, the `Cut` removes the alternatives created by the call to `p` and also discards the other rule `r X R :- q X R`. If the first rule fails, the `Cut` is not reached hence the second rule is not discarded. Thus, at most one rule can succeed for `q`.

The predicates s and s1 below illustrate the role of pi.

```
func s int -> int.                      func s1 int -> int.
s X R :- pi y\ q y 3 => q X R.          s1 X R :- pi y\ q 1 5 => q X R.
                             % error: mutxecl violated  ^^^^^
```

The local rule q y 3 is mutually exclusive with the rules for q above since y is fresh: each time the rule is loaded y is different from 1 and 2 and any other fresh constants generated before. On the contrary the local rule q 1 5 is not mutually exclusive: the query s1 1 R has two results, namely R = 5 and R = 3.

Note that condition *(c)* also rules out the following example, since the pi-quantified symbol y is not used in input position.

```
func s2 int -> int.
s2 1 R :- pi y\ q 3 5 => s2 2 R.
s2 2 R :- pi y\ q 3 6 => q 3 R.
%                ^^^^^ error: mutxecl violated
```

Both rules for s2 load a local rule for q. All these local rules are mutually exclusive with the global rules in the program, but the query s2 2 R has two results namely R = 5 and R = 6.

5.5 Static analysis of determinacy signatures

The abstract syntax of signatures is given below. Predicate signatures $\mathbb{T}\mathrm{y}$ start with input arguments from the $\mathbb{S}_i$ non-terminal, continue with output arguments from the $\mathbb{S}_o$ non-terminal and end with a determinacy marker F: Fun for functions and Rel for relations (respectively func and pred in the concrete syntax).

$$\begin{array}{lll}
\mathbb{S}_i & ::= \mathbb{S}_o \mid \mathbb{S}_i \xrightarrow{\text{i}} \mathbb{S}_i \mid \star\!\!\!\!\!\;\;☆ \xrightarrow{\text{i}} \mathbb{S}_i & \text{sign. start} \\
\mathbb{S}_o & ::= F \mid \mathbb{S}_i \xrightarrow{\text{o}} \mathbb{S}_o \mid ☆ \xrightarrow{\text{o}} \mathbb{S}_o & \text{sign. stop} \\
\mathbb{T}\mathrm{y} & ::= \mathbb{S}_i & \text{signature}
\end{array}
\qquad
\begin{array}{lll}
F & ::= \texttt{Fun} \mid \texttt{Rel} & \text{determinacy} \\
☆ & ::= \star & \text{all data}
\end{array}$$

We collapse all data types into a single type ☆ and we disallow to store predicates into data, i.e., one cannot form a list of predicates. Note that the grammar of $\mathbb{T}\mathrm{y}$ forces all input arguments to come first. We reserve d for instances of type ☆, ρ and τ for signatures and $\mathcal{D}$ for instances of F. For example the signature func map (func A -> B), list A -> list B corresponds to $(\star \xrightarrow{\text{i}} \star \xrightarrow{\text{o}} \texttt{Fun}) \xrightarrow{\text{i}} \star \xrightarrow{\text{i}} \star \xrightarrow{\text{o}} \texttt{Fun}$.

We denote booleans $\mathbb{B} = \{\top, \bot\}$; contexts $\mathbb{\Gamma}$ being partial maps from symbols to signatures. We use $\varnothing$ for the empty list and $x :: xs$ for prepending the element x to a list xs. We reserve Γ for contexts.

5.5.1 Operations on signatures.

We compare signatures using the standard subtyping relation of functional languages. In particular, we compare inputs contravariantly and outputs covariantly. It is similar to [32], except for the absence of the Any terminal. Fun is smaller than Rel since functions generate a "smaller number of results" than relations.

$$\star \subseteq \star \qquad \mathcal{D} \subseteq \mathcal{D} \qquad \texttt{Fun} \subseteq \texttt{Rel}$$
$$\rho \xrightarrow{\text{i}} \tau \subseteq \rho' \xrightarrow{\text{i}} \tau' \ \text{ iff } \ \rho' \subseteq \rho \wedge \tau \subseteq \tau'$$
$$\rho \xrightarrow{\text{o}} \tau \subseteq \rho' \xrightarrow{\text{o}} \tau' \ \text{ iff } \ \rho \subseteq \rho' \wedge \tau \subseteq \tau'$$

The static analysis needs often to keep the stronger or weaker signature among two. We hence derive from $\subseteq$ a `min` and `max` operators defined below.

$$\begin{array}{ll}
\texttt{min}\ \star\ \star = \star \quad \texttt{min}\ \mathcal{D}\ \mathcal{D} = \mathcal{D} & \texttt{max}\ \star\ \star = \star \quad \texttt{max}\ \mathcal{D}\ \mathcal{D} = \mathcal{D} \\
\texttt{min}\ \texttt{Fun}\ \texttt{Rel} = \texttt{min}\ \texttt{Rel}\ \texttt{Fun} = \texttt{Fun} & \texttt{max}\ \texttt{Fun}\ \texttt{Rel} = \texttt{max}\ \texttt{Rel}\ \texttt{Fun} = \texttt{Rel} \\
\texttt{min}\ (\rho \xrightarrow{i} \tau)\ (\rho' \xrightarrow{i} \tau') = \texttt{max}\ \rho\ \rho' \xrightarrow{i} \texttt{min}\ \tau\ \tau' & \texttt{max}\ (\rho \xrightarrow{i} \tau)\ (\rho' \xrightarrow{i} \tau') = \texttt{min}\ \rho\ \rho' \xrightarrow{i} \texttt{max}\ \tau\ \tau' \\
\texttt{min}\ (\rho \xrightarrow{o} \tau)\ (\rho' \xrightarrow{o} \tau') = \texttt{min}\ \rho\ \rho' \xrightarrow{o} \texttt{min}\ \tau\ \tau' & \texttt{max}\ (\rho \xrightarrow{o} \tau)\ (\rho' \xrightarrow{o} \tau') = \texttt{max}\ \rho\ \rho' \xrightarrow{o} \texttt{max}\ \tau\ \tau'
\end{array}$$

When a predicate is miscalled we need to weaken its signature.

We define strong and weak as follows:

$$\begin{array}{ll}
\text{strong}\ \star = \star & \text{weak}\ \star = \star \\
\text{strong}\ \texttt{Rel} = \texttt{Fun} \quad \text{strong}\ \texttt{Fun} = \texttt{Fun} & \text{weak}\ \texttt{Fun} = \texttt{Rel} \quad \text{weak}\ \texttt{Rel} = \texttt{Rel} \\
\text{strong}\ (\rho \xrightarrow{i} \tau) = \text{weak}\ \rho \xrightarrow{i} \text{strong}\ \tau & \text{weak}\ (\rho \xrightarrow{i} \tau) = \text{strong}\ \rho \xrightarrow{i} \text{weak}\ \tau \\
\text{strong}\ (\rho \xrightarrow{o} \tau) = \text{strong}\ \rho \xrightarrow{o} \text{strong}\ \tau & \text{weak}\ (\rho \xrightarrow{o} \tau) = \text{weak}\ \rho \xrightarrow{o} \text{weak}\ \tau
\end{array}$$

The set of signatures that only differ by a leaf in $\mathcal{D}$ form a lattice w.r.t. the order relation $\subseteq$, the join operator `max` and meet operator `min`. The infimum of the lattice is computed by strong and the supremum is computed by weak.

In section 2.2.4 we have weakened the signature of `map` from $\tau = (\star \xrightarrow{i} \star \xrightarrow{o} \texttt{Fun}) \xrightarrow{i} \star \xrightarrow{i} \star \xrightarrow{o} \texttt{Fun}$ to $(\star \xrightarrow{i} \star \xrightarrow{o} \texttt{Fun}) \xrightarrow{i} \star \xrightarrow{i} \star \xrightarrow{o} \texttt{Rel}$.

One could define the supremum of τ as $\tau' = (\star \xrightarrow{i} \star \xrightarrow{o} \texttt{Rel}) \xrightarrow{i} \star \xrightarrow{i} \star \xrightarrow{o} \texttt{Rel}$, which intuitively amounts to considering all `Fun` as `Rel`, i.e., dropping all `Fun` in case of error. However, this would contradict the lattice property: $\texttt{max}\ \tau\ (\text{weak}\ \tau) = \text{weak}\ \tau$, since we would have $\texttt{max}\ \tau\ \tau' = \tau'$, but $\tau \nsubseteq \tau'$. Moreover, since we compare input signatures using $\subseteq$, this alternative definition of weak (resp. strong) would not make the checker accept more programs.

5.5.2 The idea.

The static analyzer accumulates knowledge on variables in a context Γ initially only containing the signature of predicates. The checking algorithm considers one rule at a time. It begins by updating Γ using the preconditions on the *inputs* of the head, then it scans all body atoms and finally checks the postconditions: the determinacy of the body and the determinacy of the *outputs* of the head.

For example, consider the predicates `comp` and `compose` below: `comp` is a function that takes as input two functions `F` and `G`, and calls them in sequence, whereas `compose` builds a new function from the two received in input.

```
func comp (func int -> int), (func int -> int), int -> int.
comp F G X Z :- F X Y, G Y Z.

func compose (func int -> int), (func int -> int) -> (func int -> int).
compose F G (comp F G).
```

After updating Γ w.r.t. the preconditions of `comp`, Γ flags `F` and `G` as functions. Then the algoritms scans all body atoms computing the `max` of their determinacies: since both `F X Y` and `G Y Z` are `Fun`, also their `max` is. Finally the analyzer checks that the determinacy of the body matches the declared one.

The analysis for `compose` needs to check two postconditions: 1) the *output* term `comp F G` is a function, since the two arguments of `comp` match its signature; 2) the body of `compose` is deterministic since empty.

We now move to an example that involes a non-deterministic body atom.

$$\frac{\Gamma \vdash^{i}_{hd} c = \mathcal{D}_1,\ \Gamma_1 \quad \forall i \in 1 \dots n \text{ do } \mathcal{P},\ \Gamma_i,\ \mathcal{D}_i \vdash_{a} b_i = \mathcal{D}_{i+1},\ \Gamma_{i+1} \quad \Gamma_{n+1} \vdash^{o}_{hd} c : \mathcal{D}_{n+1}}{\mathcal{P},\ \Gamma \vdash_{r} c \text{ :- } b_1 \dots b_n} \ (r)$$

Figure 5.1: Check rule

$$\frac{\begin{array}{c} x \text{ fresh in } \Gamma \qquad \mathcal{P},\ \Gamma + \{x \mapsto \star\} \vdash_{r} r \\ \text{mut\_excl } \mathcal{P}\ \Gamma\ x\ r \qquad (r :: \mathcal{P}),\ \Gamma + \{x \mapsto \star\},\ \mathcal{D} \vdash_{a} a = \mathcal{D}',\ \Gamma' \end{array}}{\mathcal{P},\ \Gamma,\ \mathcal{D} \vdash_{a} \texttt{pi}\ x.\ r \Rightarrow a = \mathcal{D}',\ \Gamma'} \ (a_\forall)$$

$$\frac{}{\mathcal{P},\ \Gamma,\ \_ \vdash_{a} \texttt{Cut} = \texttt{Fun},\ \Gamma} \ (a_!) \qquad \frac{\Gamma \vdash^{i}_{tm} c = \texttt{Rel},\ \bot}{\mathcal{P},\ \Gamma,\ \_ \vdash_{a} c = \texttt{Rel},\ \Gamma} \ (a_\bot)$$

$$\frac{\Gamma \vdash^{i}_{tm} c = \mathcal{D}',\ \top \qquad \Gamma \vdash^{o}_{ca} c : \mathcal{D}' = \Gamma'}{\mathcal{P},\ \Gamma,\ \mathcal{D} \vdash_{a} c = \texttt{max}\ \mathcal{D}\ \mathcal{D}',\ \Gamma'} \ (a_c)$$

Figure 5.2: Rules for check atom: its type is $\mathbb{P},\ \mathbb{\Gamma},\ F \vdash_{a} \mathbb{A} = F,\ \mathbb{\Gamma}$

```
func r (func int -> int), (pred int -> int), int -> int.
r F G X Y :- map G [X] [Z], !, compose F F H, H Z Y.
```

After analyzing the head the context asserts that `F` (resp. `G`) is a function (resp. a relation). In the body of `r`, the first atom is non-deterministic, since `G` violates the preconditions of `map`. The signature of `map G [X] [Z]` is hence weakened to `Rel`. Even if this atom could generate multiple alternatives, the `Cut` commits to the first one restoring the determinism of the body of `r` up to that point. The call to `compose` satisfies its preconditions, so its postconditions apply: the entire atom `compose F F H` is considered deterministic *and* `H` is marked as a function in Γ. The final atom `H Z Y` is also deterministic because of the knowledge about `H` accumulated so far in Γ.

5.5.3 The algorithm.

We present the static analyzer using derivation rules. Each procedure has an associated entails-like symbol, e.g., context $\vdash^{\text{flag}}_{\text{name}}$ intake = result. We use the equal sign to "mode" the rule, i.e., separate what is given to the rule from what is generated by it. The static analyzer is composed of seven procedures for a total of 24 rules. Here we list only the interesting ones, leaving the rest to the appendix in [21].

appendix

The main entry point is $\texttt{check}_{:-}$ ($\mathbb{P},\ \mathbb{\Gamma} \vdash_{r} \mathbb{R}$), that verifies if a rule validates the signature ascribed to its predicate. The procedure stores in Γ all the knowledge available on variables in $\mathbb{U}$ and it verifies that the precondition made by the signature entails the postcondition.

In particular the assume head input ($\mathbb{\Gamma} \vdash^{i}_{hd} \mathbb{Tm} = \mathbb{Ty},\ \mathbb{\Gamma}$) procedures loads in Γ_1 the information assumed from the head inputs while check head output ($\mathbb{\Gamma} \vdash^{o}_{hd} \mathbb{Tm} : \mathbb{Ty}$) verifies that Γ_{n+1} entails the postcondition. Each of the n body atoms b_i is processed by check atom (fig. 5.2, $\mathbb{P},\ \mathbb{\Gamma},\ F \vdash_{a} \mathbb{A} = F,\ \mathbb{\Gamma}$) that updates the context and computes the determinacy of the entire body $\mathcal{D}_{n+1}$.

We now focus on the analysis of an atom. The check atom procedure has two rules to handle `Cut` ($a_!$ in 5.2) and `pi` ($a_\forall$ in 5.2), and two more rules to handle predicate calls (a_c in 5.2) and predicate miscalls ($a_\bot$ in 5.2). The `Cut` operator resets the current determinacy to `Fun`, while `pi`

$$\frac{\Gamma\ s = \tau}{\Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} s = \tau,\ \top}\ (\mathrm{tm}^{\mathrm{i}}_{k}) \qquad \frac{\Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} f = \rho \xrightarrow{\mathrm{i}} \tau,\ \top \qquad \Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} t = \rho',\ \top \qquad \rho' \subseteq \rho}{\Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} f\ t = \tau,\ \top}\ (\mathrm{tm}^{\mathrm{i}}_{\top})$$

$$\frac{\Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} f = \rho \xrightarrow{\mathrm{i}} \tau,\ b_1 \qquad \Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} t = \rho',\ b_2 \qquad b_1 = \bot \vee b_2 = \bot \vee \rho' \nsubseteq \rho}{\Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} f\ t = \mathrm{weak}\ \tau,\ \bot}\ (\mathrm{tm}^{\mathrm{i}}_{\bot})$$

Figure 5.3: Rules for check input: its type is $\Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} \mathbb{Tm} = \mathbb{Ty},\ \mathbb{B}$

$$\frac{\tau' = \text{if } X \in \Gamma \text{ then } \Gamma\ X \text{ else } \tau}{\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} X : \tau = \Gamma + \{X \mapsto \mathtt{min}\ \tau\ \tau'\}}\ (\mathrm{tm}^{\mathrm{o}}_{X}) \qquad \frac{\Gamma\ k = \tau' \qquad \tau' \subseteq \tau}{\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} k : \tau = \Gamma}\ (\mathrm{tm}^{\mathrm{o}}_{k})$$

$$\frac{\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} f : \rho \xrightarrow{\mathrm{i}} \tau = \Gamma' \qquad \Gamma' \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} t : \rho = \Gamma''}{\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} f\ t : \tau = \Gamma''}\ (\mathrm{tm}^{\mathrm{o}}_{i})$$

Figure 5.4: Rules for assume output: its type is $\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} \mathbb{Tm} : \mathbb{Ty} = \Gamma$

introduces a fresh symbol, checks the hypothetical rule and proceeds after adding r to $\mathcal{P}$. Rule a_c verifies that the preconditions for c are met, and assumes its postcondition generating an updated Γ'. The determinacy is set to the maximum of current determinacy and the one of the predicate. When the preconditions are not met the context is left untouched and the current determinacy is set to `Rel`.

The most interesting procedure is the one to check if the preconditions of a predicate call are met, that is check input (fig. 5.3, $\Gamma \mathrel{\vdash^{\mathrm{i}}_{\mathrm{tm}}} \mathbb{Tm} = \mathbb{Ty},\ \mathbb{B}$). The procedure ignores data arguments and output arguments and simply fetches from the context the signature of constants and variables (tm^i_k in 5.3). Rule $tm^i_\top$ is the standard typing rule for application: if the input argument matches the expected signature it computes the return type, signaling no miscalls (i.e. $\top$). When the head or the argument contain a miscall, or when the argument does not match the expected signature, the rule $tm^i_\bot$ signals a miscall and computes the return type by weakening the one obtained from the head, i.e., the predicate call is considered as a relation, and the signature of the output arguments is also recursively weakened.

The procedure to update the context when an atom is found to be a good call is assume call output ($\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{ca}}} \mathbb{Tm} : \mathbb{Ty} = \Gamma$). Other than ignoring input arguments it calls assume output on output arguments.

$$\frac{\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{ca}}} f : \rho \xrightarrow{\mathrm{o}} \tau = \Gamma' \qquad \Gamma' \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} t : \rho = \Gamma''}{\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{ca}}} f\ t : \tau = \Gamma''}\ (\mathrm{a}^{\mathrm{o}}_{o})$$

The assume output procedure (fig. 5.4, $\Gamma \mathrel{\vdash^{\mathrm{o}}_{\mathrm{tm}}} \mathbb{Tm} : \mathbb{Ty} = \Gamma$) updates the context when it encounters variables (rule tm^o_X). When variable is already ascribed a signature in the context, we keep the stronger signature. Data and output arguments are skipped, while input arguments are recursively traversed: the signature of variables in their input positions are also updated in the context.

The processing of the head of a rule is akin to the one of body atoms, but symmetric in which arguments are used to strengthen the context and which are checked for conformance. The assume head input procedure ($\mathbb{\Gamma} \mid_{\overline{\text{hd}}}^{\text{i}} \mathbb{Tm} = \mathbb{Ty},\ \mathbb{\Gamma}$) called at the very beginning of 5.1, ignores outputs and calls assume output on inputs.

The check head output ($\mathbb{\Gamma} \mid_{\overline{\text{hd}}}^{\text{o}} \mathbb{Tm} : \mathbb{Ty}$) procedure is called at the very end.

$$\frac{\Gamma \mid_{\overline{\text{hd}}}^{\text{o}} f : \rho \xrightarrow{\text{o}} \tau \qquad \Gamma \mid_{\overline{\text{tm}}}^{\text{i}} t = \rho',\ \top \qquad \rho' \subseteq \rho}{\Gamma \mid_{\overline{\text{hd}}}^{\text{o}} f\ t : \tau}\ (\text{hd}\,^{\text{o}}_{\text{o}})$$

It ignores input arguments but checks that the output ones satisfy the ascribed signature in the current context, that is the result of assuming the rule preconditions and all the postconditions of well called body atoms.

5.6 Determinism guarantees

In order to express *operational determinacy* we need to use an operational semantics for Elpi. That semantics, for its higher-order logic-programming part, is essentially the one given by [53] using a last-in-first-out goal selection. However it must be complemented with a treatment of choice points to describe the behavior of the cut operator following [11]. Moreover Elpi has a peculiar runtime threatment of input arguments, that are matched rather than unified (see definition 2.2.3).

For space constraints we omit the rules defining `runT`, they can be found in the appendix
appendix of [63].

Definition 5.6.1 (Operational semantics ($\texttt{runT} : \mathbb{L} \times \overrightarrow{\mathbb{L}} \to \Sigma \times \overrightarrow{\mathbb{L}}$))**.** Run is a partial function from a non-empty list of alternatives to a solution consisting in a substitution and a list of yet-to-be-explored alternatives.

The properties of `runT` relevant to the current paper are:

1. `Cut` behaves as per definition 5.4.1
2. `pi` x r b generates a fresh constant for x available in r and b
3. `pi` x r b loads r in the program giving it the highest priority
4. input arguments are matched against the query as per definition 2.2.3

Definition 5.6.2 (Operationally deterministic execution ($\text{deterministic}_{\text{a}}\ \mathbb{P}\ \mathbb{A}$))**.** We say that a query atom b is operationally deterministic in a program $\mathcal{P}$ iff

$$\forall \sigma, \texttt{runE}\ ((\sigma,\ (\mathcal{P}, b) :: \varnothing) :: \varnothing) \rightsquigarrow (\_, a)\_ \Rightarrow\ a = \varnothing$$

In other words a deterministic query leaves no alternatives when it succeeds. We now link the static check to the deterministic execution of a query.

Definition 5.6.3 (Checked program ($\text{checked}_{\text{p}}\ \mathbb{\Gamma}\ \mathbb{P}$))**.** A program $\mathcal{P}$ is checked against a context of signatures Γ iff it validates these rules:

$$\frac{\Gamma \mid_{\overline{\text{ca}}}^{\text{o}} f : \rho \xrightarrow{\text{o}} \tau = \Gamma' \qquad \Gamma' \mid_{\overline{\text{tm}}}^{\text{o}} t : \rho = \Gamma''}{\Gamma \mid_{\overline{\text{ca}}}^{\text{o}} f\ t : \tau = \Gamma''}\ (\text{a}\,^{\text{o}}_{\text{o}})$$

The intuition is that we build the program from the end, always adding new rules with highest priority.

Definition 5.6.4 (Checked query atom ($\text{checked}_\text{a}\ \mathbb{\Gamma}\ \mathbb{A}$))**.** An atom b is checked against a context of signatures Γ iff

$$\varnothing,\ \Gamma,\ \texttt{Fun} \vdash_{\text{a}} b = \texttt{Fun},\ \_$$

Theorem 5.6.1 (Static analysis)**.** *Given program $\mathcal{P}$, a context of signatures Γ and a query atom b, $checked_p\ \Gamma\ \mathcal{P} \wedge checked_a\ \Gamma\ b \Rightarrow deterministic_a\ \mathcal{P}\ b$.*

The proof of theorem 5.6.1 has been mechanized in the Rocq proof assistant, formerly known as Coq, for the first-order part [18]. The mechanization for the higher-order part is ongoing.

5.7 Experience report

The static analyzer we describe was integrated in the Elpi language version 3.0. It consists of about 200 lines of OCaml code for the implementation of mut_excl and about 800 lines of code for the implementation of $\texttt{check}_{:-}$, along with all the related auxiliary functions. We have ported some Rocq libraries written using Elpi: Rocq-Elpi, Hierarchy Builder, Algebra Tactics, Trocq, and BRiCk.

We can divide the porting of these libraries into two categories: libraries in which we have traversed all the predicates and carefully changed **`pred`** markers to **`func`** (Elpi, Rocq-Elpi, and Hierarchy Builder), and libraries that have been ported to just pass the static analyzer. In particular, the latter class of libraries measures the minimal cost for adapting code to the changes we made to the Elpi and Rocq-Elpi standard libraries. The typical example of code requiring a fix is code that loads a local rule containing no cut, typically `copy`.

We summarize these results in table 5.1. In the Elpi standard library, we annotated 272 predicates out of 281 as functions, with 37 occurrences of higher-order arguments, as in `map`, `filter`, `fold`, etc. We identified 1 error, i.e., a missing cut. In Rocq-Elpi, we have 928 functions out of 1181, with 62 higher-order predicates. We added 91 cuts to please the static analyzer, 78 of which were clearly bugs in the code. The remaining 13 cuts are due to mut_excl, which is not powerful enough to discriminate among rules. The algorithm could be improved to better distinguish mutual exclusion by extending definition 5.4.3, as in [38], but the problem is, in general, undecidable. An example of this issue is given below:

Software	LOC	% `Fun`	`Cut` added	`Cut` needed (bugs)
Elpi	1448	97%	1	1 (100%)
Rocq-Elpi	13164	79%	91	78 (85%)
Hierarchy Builder	5326	94%	33	30 (90%)
Algebra Tactics	1799	–	1	1 (100%)
Trocq	2918	–	4	4 (100%)
BRiCk (public)	2500	–	11	11 (100%)
BRiCk (private)	7500	–	6	6 (100%)
Total	34655	85%	147	131 (89%)

Table 5.1: Experience report

```
func fact int -> int.
fact 0 1.
fact N R :- N > 0, N' is N - 1, fact N' R', R is N * R'.
```

The two rules for `fact` are in mutual exclusion: the input cannot be *0* and *greater than 0* at the same time. But a complete algorithm identifying all of these scenarios does not exists, and we require an explicit `Cut` in the first rule.

In Hierarchy Builder, we have 460 functions out of 488, with 33 higher-order arguments, and we have corrected 30 bugs in the code.

Overall, we can conclude that 85% of the predicates were functions.

In Algebra Tactics, we added 1 cut over 1799 lines of codes; in Trocq, we added 4 cuts over 2918 lines of code. None of these libraries were using `copy` as a relation, i.e. to generate all possible copies of a term. The BRiCk application has some public code we ported ourselves and some closed source code ported by the BlueRock company, that we thank for sharing these numebrs. In the public code we added 11 cuts over 2500 lines of code, while in the private one they added 11 cuts over 7500 lines.

5.8 Related work and concluding remarks

We summarize several related works in table 5.2. One key difference between our work and [13, 38, 57, 37, 32] is that they focus on *closed* predicates, that do not evolve during execution, while λProlog, hence Elpi, allows rules to be dynamically loaded. For closed predicates, programs can be transformed into *super-homogeneous* form by merging all rules into one with disjunctions. This simplifies determinacy inference and may allow to compile the code to a faster binary. However, this transformation is not feasible in our dynamic setting (section 2.2.5).

Regarding modes, Elpi's matching procedure over input arguments allows us to dispense with explicit mode analysis – an essential component in [13, 38, 57, 37].

We differ from Red Alert [37] in that we pick an operational semantics that explicitly represents choice points as a list of alternatives, and that hence is very close to the actual behavior of Elpi's runtime. They pick a denotational semantics over the super-homogeneous form that in turn lets them formulate the static analyzer as an abstract interpreter. It is unclear if a denotational semantics can be written without transforming the program in super-homogeneous form.

Some of the higher-order features of the Elpi language are available in Mercury [57], although in a more constrained way (‡). For example `once` from section 2.2.4 is considerably more useful if the input relation is allowed to be non-ground, e.g, `once (map get_pdiv [6] L)`. We

Authors	System	Mode analysis	Dynamic	H.O.	Miscalls
[13]	SB-Prolog	required	✗	✗	✗
[38]	Ciao	required	✗	✗	✗
[37]	Red Alert	required	✗	✗	✗
[57]	Mercury	required	✗	✓‡	✗†
[32]	Curry	no need	✗	✓	✓
Fissore and Tassi	Elpi	no need	✓	✓	✓

Table 5.2: Comparison with related work

allow higher-order predicates to be miscalled, while Mercury does not. Instead, it requires higher-order functions to be annotated with explicit mode and determinacy signatures for each possible behavior. As a result, Mercury ascribes 11 signatures to the `foldr` predicate and any call that does not match exactly one of these signatures results in a compilation failure (†). Elpi targets less technically inclined users, more familiar with functional programming languages than logic ones, hence we strive for simplicity, defining only one determinacy class and immediately classifying wrong calls as non-deterministic. As a result the signature for `foldr` is just `func foldr (func L, A -> A), list L, A -> A`, familiar to our audience.

The "determinism types" of Curry [32] are much more similar to our signatures, and Curry supports miscalled functions as well. In the work of Hanus and Prott the `Any` type stands for non-determinism and like a black hole it absorbs the determinism of the surrounding sub-terms. This is a convenient way to capture all miscalls and treat them uniformly. In our presentation, we have no equivalent; instead, we use weak to change the signature of a term when a miscall is detected (such as, $tm^i_\perp$ in 5.3 or atom calls $a_\perp$ in 5.2). The advantage is that weak only changes the leaves (F) of the signature while leaving its structure intact. Although we do not analyze data in this paper, our set of rules can be complemented to perform full type checking (in addition to determinacy checking) in one pass.

In Curry the `allValues` builtin can be used to turn a relation into a function. Prolog, and also Elpi, provide a similar `findall` builtin, but in addition to that they also let one commit to the first result by either using `once`, or by placing a `Cut` after the non-deterministic call. It is this "after the fact" commitment that make the analysis a bit more complex. In fact, in the body of a deterministic predicate we can have calls to relations (or miscalled functions) and still consider the rules valid with respect to determinacy (see $a_!$ in 5.2). The notion of *functional context* in [13] is closely related to this difficulty.

Our experience report confirms most of the Elpi code out there is made of functional predicates and that the static analyzer accepts the vast majority of it requiring no changes. The main improvement we will consider is about *mode subtyping* for higher-order arguments. In the definition of $\subseteq$ we currently accept only arrows that have the same input/output mode. However, it seems possible to relax this condition. For example we could allow passing to `map` a function of type $\star \xrightarrow{\text{o}} \star \xrightarrow{\text{o}}$ `Fun`, since the modes of higher-order arguments seem to play no role in the determinacy of code that receives and calls them.

CHAPTER 6

A Mechanized First-Order Static Determinacy Checker

Elpi is a dialect of λ-*Prolog* (see [44, 45, 15, 29]) used as an extension language for the *Rocq* prover (formerly the *Coq* proof assistant). *Elpi* has become an important infrastructure component: several projects and libraries depend on it [36, 7, 9, 66, 19, 20]. Other remarkable libraries include the Hierarchy-Builder library-structuring tool [10] and Derive [61, 62, 28], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, *Elpi* is equipped with a static analysis for determinacy [21] to help users control backtracking. *Rocq* users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point correspond to an suspended *alternative* in the execution trace. In the following we use *choice point* and *alternative* as synonyms.

This paper reports our first steps towards a mechanization, in the *Rocq* prover, of the determinacy analysis from [21]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for *Prolog* with cut. The first is a stack-based semantics that closely models *Elpi* implementation and is similar to the semantics mechanized by Pusch in *Isabelle* [50] and to the model of Debray and Mishra [12, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard *Prolog* abstract machines [69, 3], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive queries. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1.

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [21]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceding the cut are discarded.

We show that if every rule defining a deterministic predicate passes this analysis, then any call to that predicate leaves no choice points.

6.1 Preliminaries: syntax and informal semantics of cut

In the entire paper we use Figure 2.4 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ-calculus style (no parentheses).

The description of a program is given by the record $\mathbb{P}$ (in fig. 6.1) and is shared by both semantics. A program consists of a list of rules `R` together with a signature environment `sig`. We delay the discussion of signatures to section 6.5 where we describe the static analysis that concerns them.

A rule consists of a head term `Tm` and a list of premises. Each premise is an `Atom`, i.e. either the cut operator or a predicate call. Terms `Tm` include predicate symbols, data symbols, variables, and term application. The syntax tree allows variables to be applied and predicates to be mixed with data. These higher-order features play no role in the current paper that focuses on first order logic programming, but allows future extensions to full λProlog to be based on the same mechanization.

The set of variables `V`, predicates `P` and data symbols `S` are countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the finmap library [42]. The empty substitution is written ε, while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics.

```
Definition backchain : ℙ → 𝓕_𝒱 → Tm → Σ → 𝓕_𝒱 * list (Σ * list Atom)
```

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_\mathcal{V}$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization, we abstract over the unifier and its properties. The function `unify` takes two terms t_1 and t_2, together with a substitution σ, as input, and optionally returns a substitution σ' that extends σ. The expected behavior of `unify` is the standard one (see for example [14, section

```
Inductive Tm := Symb of S | Pred of P | Var of V | App of Tm & Tm.
Inductive Atom := cut | call of Tm.
Record R := mkR { head : Tm; premises : list Atom }.
Record ℙ := { rules : seq R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.
```

Figure 6.1: Definition of term, rule, program and substitution

2.5]), limited to the first order. Therefore, if t_1 and t_2 unify under σ, then $\sigma'\ t_1 = \sigma'\ t_2$, where $\sigma\ t$ denotes substitution application and = denotes syntactic equality.

6.2 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree, defined in Rocq as in fig. 6.2. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. The `Todo` node represents an unexplored branch (an `Atom`). When the atom is a `call`, we use the backchaining procedure from section 6.1 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the `step` procedure described in section 6.2.2.

In addition to the `Todo` node we have two distinguished nodes, `KO` and `OK`, which represent respectively the smallest failed and successful tree.

The `Or` node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

The `And` node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A. We call B_0 the *reset point* for B: it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A, the subtree B is replaced by the atoms in B_0. An example is shown in section 6.2.3.

A graphical representation of a tree is shown in fig. 6.4b. For compactness, we depict `And` and `Or` nodes as n-ary, with children associating to the right. The `KO` node acts as the terminating element of an `Or` list, while `OK` is the terminating element of an `And` list.

6.2.1 The semantics

The tree is constructed incrementally by two recursive functions, `step` and `prune`. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to the `next_tree` (in fig. 6.3). When this node is `OK` we say the entire tree is a *success*, when it is `KO` we say the tree *failed*, otherwise the tree is *incomplete*. In fig. 6.4 we mark with a black arrow the result of `next_tree`.

```
Inductive tag := Expanded | CutBrothers | Failed | Success.

Inductive tree :=
  | KO | OK                              (* failure and success *)
  | Todo of Atom                         (* unexplored branch *)
  | Or   of option tree & Σ & tree       (* A ∨ B_σ *)
  | And  of tree & list Atom & tree.     (* A ∧_B0 B *)
```

Figure 6.2: Definition of And-Or tree

```
Definition step : ℙ → 𝓕ᵥ → Σ → tree → (𝓕ᵥ * tag * tree)
Definition prune : 𝔹 → tree → option tree
Inductive runT : ℙ → 𝓕ᵥ → Σ → tree → option (Σ * option tree) → 𝔹 → 𝓕ᵥ → Prop
```

Since all functions must terminate in Rocq, we define their transitive closure of `step` and `prune` as the inductive relation `runT`. More precisely `runT` iterates calls to `step` on incomplete trees to expand `Todo` nodes. When a tree fails it calls `prune` to find the next alternative and continues from there, if one exists.

6.2.2 The `step` procedure

The `step` procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a `tag`, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The `tag` indicates which kind of step was performed. `Failure` and `Success` are returned for trees that satisfy, respectively, the `failed` and `success` tests. `CutBrothers` indicates the interpretation of a superficial cut: `step` was called on an And-layer and its `next_tree` is the cut atom. Finally, `Expanded` accounts for the interpretation of predicate calls and non-superficial cuts.

The `step` procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

Lemma 6.2.1 (succ_step_iff). $\forall\ p\ v\ \sigma\ t,\ success\ t \leftrightarrow step\ p\ v\ \sigma\ t = (v,\ Success,\ t).$

Lemma 6.2.2 (fail_step_iff). $\forall\ p\ v\ \sigma\ t,\ failed\ t \leftrightarrow step\ p\ v\ \sigma\ t = (v,\ Failed,\ t).$

The `step` procedure expands `Todo` nodes by calling `backchain`, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by `KO`. Otherwise, it is replaced by a right-skewed tree of `Or` nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is `OK`; otherwise, it is a right-skewed tree of `And` nodes.

Figure 6.4 depicts the tree corresponding to the running example (section 2.2.3) as it undergoes calls to `step` and `prune`. We run query `q 0 R` against the program P in fig. 2.4. Let c_0, c_1, and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P. Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in section 2.2.3. Reset points are defines

```
Fixpoint next σ t : Σ * tree :=
  match t with
  | Todo _ | KO | OK ⇒ (σ, t)
  | Or □ σ' B ⇒ next σ' B
  | Or ·A _ _ ⇒ next σ A
  | And A _ B ⇒
    let (σ', pA) := next σ A in
    if pA == OK then next σ' B
    else (σ', pA)
  end.
```

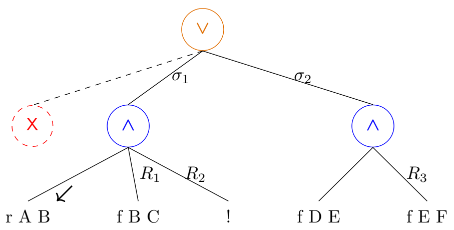
```
Definition next_subst σ t := (next σ t).1.
Definition next_tree t := (next ε t).2.

Definition success t := next_tree t == OK.
Definition failed t := next_tree t == KO.
Definition incomplete t :=
   if next_tree t is Todo _ then ⊤ else ⊥.
```

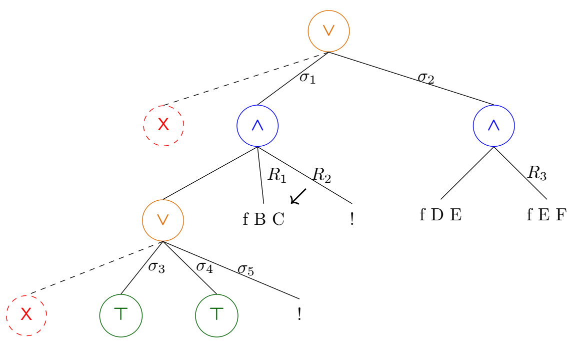
Figure 6.3: `next` and derived functions

g 0 R

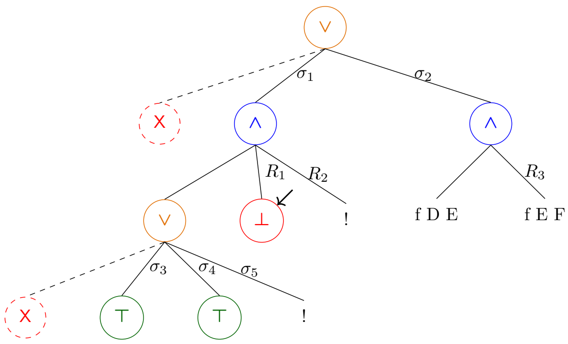
(a) Initial query



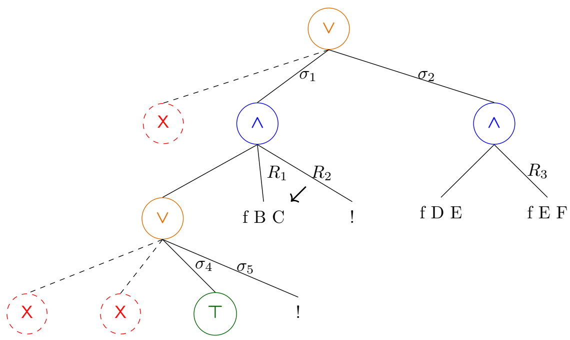
(b) Tree after `step` on `g 0 R`



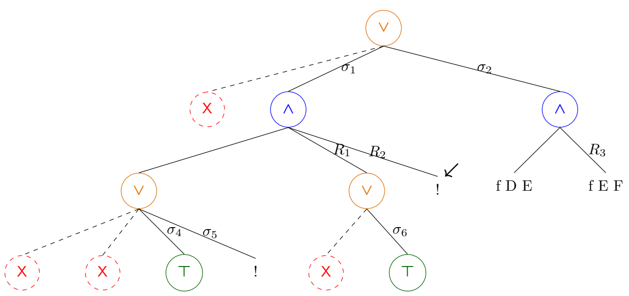
(c) `step` on fig. 6.4b



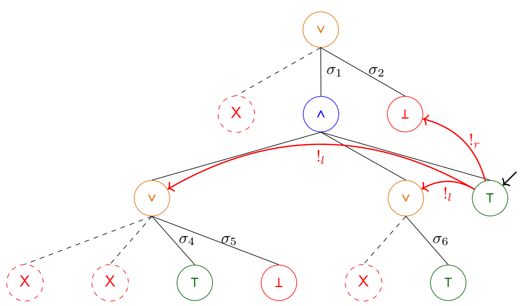
(d.1) `step` on fig. 6.4c



(d.2) `prune` on fig. 6.4d.1



(e) `step` on fig. 6.4d.2



(f) `step` on fig. 6.4e

Figure 6.4: Execution in the tree semantics. The substitutions $\sigma_1 \dots \sigma_6$ are from section 2.2.3. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f Y Z, !`], [`!`], and [`f Y Z`].

as follows: R_1 := [f Y Z, !], R_2 := [!], and R_3 := f Y Z. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of `step` performing a tree expansion via backchain appear in figs. 6.4c to 6.4e.

The interpretation of a cut The step corresponding to a cut performs two actions: (i) the cut node is replaced by `OK`; and (ii) the subtrees in the scope of `Cut` are pruned. More precisely, (ii) prunes the left siblings of the `Cut` node (in the same And-layer, denoted $!_l$) and prunes the right uncles of `Cut` (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in fig. 6.4f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by `KO`. This amounts to discarding the third rule of predicate `r`.

6.2.3 The `prune` procedure

When `backchain` returns an empty list, `step` produces a tree that *failed* and the computation must backtrack. The `prune` procedure is responsible for producing the new tree from which the computation can continue.

In fig. 6.4d.1 we encounter a failure because no rule applies to the atom `f B C` under substitution σ_3, which maps `B` to `0`. The `prune` procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the `OK` node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the `KO` node with the information stored in the reset point R_1. This restores the call `f B C`, which can then be reconsidered under substitution σ_4.

Furthermore, `prune` serves another important purpose. Its behavior depends on the boolean flag it receives as input: `prune` $\perp$ t recursively removes all failures (if any) from t, whereas `prune` $\top$ t removes the first success in t. For example, the tree in fig. 6.4f contains a successful path that maps `R` to `2`. Calling `prune` on this tree returns $\square$, since there are no alternatives in the tree after the success.

Some interesting properties of `prune` are shown below; they illustrate how `prune` complements `step` with respect to failed, successful, and incomplete trees.

Lemma 6.2.3 (incomplete_prune_id). $\forall\ b\ t,\ \mathit{incomplete}\ t \to \mathit{prune}\ b\ t = \cdot t.$

Lemma 6.2.4 (prune_Some). $\forall\ b\ t\ t',\ \mathit{prune}\ b\ t = \cdot t' \to \mathit{failed}\ t' = \perp.$

Lemma 6.2.5 (prune_None). $\forall\ t,\ \mathit{prune}\ \perp\ t = \square \to \mathit{failed}\ t.$

6.2.4 The `runT` relation and its properties

The inductive relation `runT` relates four inputs to three outputs. The inputs are a program, a set of variables, an initial substitution σ, and a tree t. The outputs are an optional result r, a boolean b, and an updated set of variable names.

The main result r is $\square$ when the execution fails; otherwise when $r = \cdot(\sigma', t')$ the execution succeeded and produced the solution σ'; t' represents the remaining, not-yet-explored alternatives. The boolean b records whether a superficial cut was evaluated during execution.

$$\frac{\texttt{success}\ t \qquad \texttt{next\_subst}\ \sigma\ t = \sigma' \qquad \texttt{prune}\ \top\ t = t'}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ \cdot(\sigma', t')\ \bot\ v}\ \textit{StopT}$$

$$\frac{\begin{array}{cc}\texttt{incomplete}\ t & b' = (\mathrm{tg} = \texttt{CutBrothers}) \lor b \\ \texttt{step}\ \mathrm{p}\ v\ \sigma\ t = (v', \mathrm{tg}, t') & \texttt{runT}\ \mathrm{p}\ v'\ \sigma\ t'\ r\ b\ v''\end{array}}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ r\ b'\ v''}\ \textit{StepT}$$

$$\frac{\texttt{failed}\ t \qquad \texttt{prune}\ \bot\ t = \cdot t' \qquad \texttt{runT}\ \mathrm{p}\ v\ \sigma\ t'\ r\ \mathrm{n}\ v'}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ r\ \mathrm{n}\ v'}\ \textit{BackT}$$

$$\frac{\texttt{prune}\ \bot\ t = \square}{\texttt{runT}\ \mathrm{p}\ v\ \sigma\ t\ \square\ \bot\ v}\ \textit{FailT}$$

Figure 6.5: Rule system for `runT`

The `runT` predicate has four cases, depicted as inference rules in fig. 6.5. (*StopT*) If `next_tree` t is a success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned of its successful path. (*StepT*) If `next_tree` t is an atom, then `step` is invoked to evaluate t, and `runT` is called recursively on the updated tree. (*BackT*) If `next_tree` t is a failure, then `prune` is called to clear the failed path; if the resulting cleaned tree exists, `runT` is called recursively on it. Finally, (*FailT*) accounts for trees with no solutions.

The set of rules defining `runT` is deterministic.

Lemma 6.2.6 (runT_det)**.** $\forall\ p\ v_0\ \sigma\ t\ r\ r'\ b\ b'\ v\ v',\ runT\ p\ v_0\ \sigma\ t\ r\ b\ v \to$
$runT\ p\ v_0\ \sigma\ t\ r'\ b'\ v' \to r' = r \land v' = v \land b' = b.$

The proof is by induction on the first `runT` derivation and is immediate, because the rules are selected based on `next_tree` t. In particular, the hypotheses `success` t, `incomplete` t, and `failed` t are mutually exclusive. Moreover, by lemma 6.2.5, the hypothesis of *FailT* implies that t *failed*; hence *FailT* is exclusive with *StopT* and *StepT*, and it is also exclusive with *BackT* since they impose different requirements on the output of `prune`.

The boolean output of `runT` allows us to state interesting properties about the `Or` node in the presence of cut. Here we report only a selection of those we proved.

Lemma 6.2.7 (runSST_or)**.** $\forall\ p\ v\ v'\ \sigma\ \sigma'\ A\ A'\ \sigma_1\ B,\ runT\ p\ v\ \sigma\ A\ \cdot(\sigma',\ \cdot A')\ \top\ v' \to$
$runT\ p\ v\ \sigma\ (\cdot A \lor B_{\sigma_1})\ \cdot(\sigma',\ \cdot(\cdot A' \lor KO_{\sigma_1}))\ \bot\ v'.$

Lemma 6.2.8 (runNT_or)**.** $\forall\ p\ v\ v'\ \sigma\ A\ \sigma_1\ B,\ runT\ p\ v\ \sigma\ A\ \square\ \top\ v' \to$
$runT\ p\ v\ \sigma\ (\cdot A \lor B_{\sigma_1})\ \square\ \bot\ v'.$

Lemma 6.2.9 (runNF_or)**.** $\forall\ p\ v_0\ v_1\ v_2\ \sigma\ A\ \sigma_1\ \sigma_2\ B\ B'\ b,$

```
runT p v0 σ A □ ⊥ v1 → runT p v1 σ1 B ·(σ2, B') b v2 →
  let sR := omap (fun x ⇒ □ ∨ x_σ1) B' in
  runT p v0 σ (·A ∨ B_σ1) ·(σ2, sR) ⊥ v2.
```

Lemmas 6.2.7 to 6.2.9 correspond to the introduction rules for disjunction. If A executes a cut, one *cannot* obtain $A \lor B$ from B: the execution of A, whether it ultimately succeeds or fails, makes the success of B irrelevant (the cut discards it). In the first lemma, B is forced to be `KO`; in the second one, the failure of A absorbs B. The last lemma is connected to the depth-first exploration of the tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been disproved. Depicted as rules:

$$\frac{A \quad (A \text{ cuts})}{A \vee B}(\vee_{il}) \qquad \frac{\neg A \quad (A \text{ cuts})}{\neg(A \vee B)}(\vee_{ir1}) \qquad \frac{\neg A \qquad B \quad (A \text{ does not cut})}{A \vee B}(\vee_{ir2})$$

Lemmas 6.2.10 and 6.2.11 are elimination rules for disjunction.

Lemma 6.2.10 (run_orSST). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ A'\ B\ B'$,

$$\begin{array}{l} \mathit{runT}\ p\ v\ \sigma\ (\cdot A \vee B_{\sigma_1})\ \cdot(\sigma',\ \cdot(\cdot A' \vee B'_{\sigma_1}))\ \bot\ v' \rightarrow \\ \quad \exists\ b,\ \mathit{runT}\ p\ v\ \sigma\ A\ \cdot(\sigma',\ \cdot A')\ b\ v' \wedge B' = \textbf{if}\ b\ \textbf{then}\ KO\ \textbf{else}\ B. \end{array}$$

Lemma 6.2.11 (run_orSNT). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ B\ B'$,

$$\begin{array}{l} \mathit{runT}\ p\ v\ \sigma\ (\cdot A \vee B_{\sigma_1})\ \cdot(\sigma',\ \cdot(\square \vee B'_{\sigma_1}))\ \bot\ v' \rightarrow \\ \quad (\exists\ b,\ \mathit{runT}\ p\ v\ \sigma\ A\ \cdot(\sigma',\ \square)\ b\ v' \wedge \textbf{if}\ b\ \textbf{then}\ B' = KO\ \textbf{else}\ \mathit{prune}\ \bot\ B = \cdot B') \vee \\ \quad (\exists\ v_2\ b,\ \mathit{runT}\ p\ v\ \sigma\ A\ \square\ \bot\ v_2 \wedge \mathit{runT}\ p\ v_2\ \sigma_1\ B\ \cdot(\sigma',\ \cdot B')\ b\ v'). \end{array}$$

From $A \vee B$, when A is not inert (i.e. not already explored), we cannot deduce A or B independently. Instead, we can only derive a weakened conclusion of the form $\top \vee B'$, where B' provides no information in the case where the exploration of A performs a cut. This yields an elimination rule that is stronger than the usual one (depicted on the left). If A does not cut, the elimination rule has the usual two premises, together with the additional constraint that whenever B holds, A must have failed. This again reflects the depth-first traversal of the proof tree.

$$\frac{A \vee B \qquad \begin{array}{c}[A]\\ C\end{array} \qquad (A \text{ cuts})}{C}(\vee_{e1}) \qquad \frac{A \vee B \qquad \begin{array}{c}[A]\\ C\end{array} \qquad \begin{array}{c}[\neg A, B]\\ C\end{array} \quad (A \text{ does not cut})}{\mathrm{C}}(\vee_{e2})$$

It is worth recalling that the "A cuts" side-condition in the rules above is precisely the boolean result returned by `runT`. Without this information, it is impossible to formulate these rules: it is not merely a matter of whether A contains a syntactic occurrence of cut, but whether that cut is actually executed during the (dis)proof of A (in rules $(\vee_{ir1})$ and $(\vee_{ir2})$). Indeed, an atom preceding the cut may fail, so the cut is never reached.

As anticipated in the introduction, these lemmas contradict the commutativity of disjunction.

6.3 Stack-based semantics

The stack-based semantics we present is very close to what is implemented by the *Elpi* interpreter. This semantics is similar to the one proposed by Pusch in [50], which is in turn closely related to the semantics given by Debray and Mishra in [12, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [69, 3].

The execution state is a stack of alternatives. Each alternative is a list of goals. Intuitively it amounts to a disjunctive normal form: each stack item is a self contained computation, coupling a substitution with all the goals to be solved.

Goals pair an atom with a list of the *cut-to* alternatives, making the two datatypes mutually recursive, but these alternatives have a very different role. They have to be understood as pointers to lower levels of the stack itself. This datum is used to implement the cut, i.e. to prune the stack of alternatives.

```
Inductive alts   := nilA | consA of (Σ * goals) & alts
with      goals  := nilG | consG of (Atom * alts)  & goals
```

```
Definition save_gs a g b :=
  [seq (x, a) | x <- b] ++ g.

Definition save_as a g b :=
  [seq (x.1, save_gs a g x.2) | x <- b].

Definition stepS p v t σ a g :=
  let (v', r) := backchain p v t σ in
  (v', save_as a g r).
```

$$\frac{}{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma, \varnothing) :: a)\ \cdot(\sigma, a)}\ \textit{StopS} \qquad \frac{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma, g) :: ca)\ \mathrm{r}}{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma, (\mathrm{cut}, ca) :: g) :: \_)\ \mathrm{r}}\ \textit{CutS}$$

$$\frac{\texttt{stepS}\ \mathrm{p}\ v\ t\ \sigma\ a\ g = (v_1, a') \qquad \texttt{runS}\ \mathrm{p}\ v_1\ (a' +\!\!+\ a)\ \mathrm{r}}{\texttt{runS}\ \mathrm{p}\ v\ ((\sigma, (\mathrm{call}\ t, \_) :: g) :: a)\ \mathrm{r}}\ \textit{CallS} \qquad \frac{}{\texttt{runS}\ \mathrm{p}\ \_\ \varnothing\ \square}\ \textit{FailS}$$

Figure 6.6: Rule system for `runS`

The stack-based semantics is described by the `runS` inductive predicates.

```
Inductive runS (p: ℙ) : 𝓕_𝒱 → alts → option (Σ * alts) → Prop
```

The `runS` predicates relates a set of variables and a list of alternatives to optional result. □ stands for failure, i.e., no solution, whereas $\cdot(\sigma, a)$ represent success with σ begin the solution and a as the list of non-yet-explored alternatives.

The rule system for `runS` is given in fig. 6.6 and is made by 4 cases. We distinguish two main cases. If we run out of alternatives we fail: (*FailS*) stops the execution and return □. If the list of alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is empty, (*StopS*) stops the execution with a success σ_0 leaving a as the unexplored alternatives. If h is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t is a cut rule (*CutS*) continues to evaluate the tail of h under with alternatives ca. Finally, when t is a call, rule (*CallS*) backchains: for each rule it pairs each premise with the current stack of alternatives, and prepends the obtained goals to the tail of h. Note that if backchain returns the empty list, (*CallS*) second premise amounts to exploring the next item in the current stack.

6.3.1 Inadequacy of the stack-based semantics for formal reasoning

The lemmas that we presented in section 6.2.4 are hard to express in the stack-based semantics, due to the lack of structure in the stack to delimit the scope of the cut operator. For example, consider a stack $a_0, a_1, \ldots, a_n$. If a_0 succeeds but executes a cut, it is hard to relate the remaining alternatives $a_1, \ldots, a_n$ to the alternatives $b_1, \ldots, b_m$ that survive the cut, since the only simple invariant is that $b_1, \ldots, b_m$ is a suffix of $a_1, \ldots, a_n$. If $a_1, \ldots, a_n$ were generated before the call whose execution generates the cut in a_0, then they all stay; but if some were generated afterwards, they are discarded. Expressing this precisely would require attaching, at the very least, time-stamps (or depths) to goals, which amounts to a cumbersome encoding of a tree.

6.4 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. We define a translation from trees to stacks, called `t2l`, but we do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [5].

Before describing the translation of trees to stacks (in section 6.4.2) we need to define the notion of *valid tree*. The `tree` datatype indeed contains trees that cannot be generated by `runT` via `step` or `prune`, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

```
Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO ⇒ T
  | Or □ _ B ⇒ valid_tree B
  | Or ·A _ B ⇒ valid_tree A &&
        ((B == KO) || B.base_or B)
  | And A B0 B ⇒ valid_tree A &&
      if success A then valid_tree B...
      else B == big_and B0
  end.
```

```
Fixpoint t2l t σ (cp: alts) : alts :=
match t with
| OK    ⇒ [(σ, ∅)]
| KO    ⇒ ∅
| TA a  ⇒ [(σ, [(a, ∅)])]
```

Figure 6.7: Valid tree and Tree to stack

6.4.1 Valid trees

The class of valid trees is captured by the `valid_tree` function shown in Figure 6.7. While all base cases of `tree` are valid, we need to analyze the `Or` and `And` constructors more carefully.

For the `Or` constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid tree. Otherwise, the left-hand side must be a valid tree, and the right-hand side must either be the `KO` tree or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the latter case we use the `base_or` predicate to check that `B` is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a `call`.

For the `And` constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. `B` must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the `big_and` procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by `backchain` into the And-layer of the resulting tree). When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

6.4.2 From trees to stacks

The `t2l` recursive function translates a tree to a stack. For space constraints fig. 6.7 only gives the base cases, while we describe the recursive cases in the following text.

The function `t2l` takes as input the tree t_0, a substitution σ_0, and a list of choice points cp. These choice points represent alternatives that appear at the same level of the current tree, such as, for example, the right siblings of an `Or` node. The substitution is used to determine under which substitution the subtree being computed lives.

The `OK` node represents one successful alternative; therefore it is translated into a singleton list whose only element has an empty list of goals. The `KO` node represents failure, hence it is mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to list cannot point to cp: atoms are not right-uncle subtrees, whereas cp represents alternatives that will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore determined later, once the trees corresponding to sibling subtrees have been inspected.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B. The alternatives of B are valid choice points for A and should be passed to the translation of A. The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level below cp (i.e. in A or B) should add cp as its cut-to alternatives.

The translation of $A \wedge_{B_0} B$ starts with the translation of A. If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0, since an empty translation for

A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g, whereas B_0 is the continuation for each alternative in a.

Example of `t2l` execution The translations of the trees in fig. 6.4 are shown in fig. 2.5. The letters in the figures relate each tree to the corresponding stack. For example, 6.4b is translated into 2.5b. We also note that 6.4d.1 and 6.4d.2 are both translated into 2.5d, showing that $t2l$ is not injective: different trees can map to the same stack. Consequently, there are transitions in `runT` that are not visible in `runS`. As an example of a call to `t2l`, we take the tree and the stack in fig. 6.8, which are respectively taken from figs. 2.5c and 6.4c. The red arrows point from the head of an alternative in the tree to the head of the corresponding cell in the stack. We focus on the deepest `Or` node in the tree. This subtree is a disjunction with four children. The first is ignored, since it is □. The success node below σ_3 has `f B C` and the cut operator as continuation, corresponding to the first cell in the stack. We note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals `f B C, !`; this is translated into the second cell of the stack. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2, since it is one `Or`-layer above its right uncles.

6.4.3 Equivalence proof

The tree-based semantics exposes more information than needed, hence we hide it behind existentials.

Definition 6.4.1 (runT' `p` v σ t r). `∃` b v'`, runT p` v σ t r b v'.

From figs. 2.5 and 6.4, we observe that, upon backtracking, `runT` may perform more steps than `runS` before returning a solution or a failure. These silent transitions must be skipped when proving the equivalence between the two semantics. Therefore, we prove the following lemma.

Lemma 6.4.1 (pruneF_t2l). $\forall\ t\ t'\ b,\ \mathit{valid\_tree}\ t \to \mathit{failed}\ t \to \mathit{prune}\ b\ t = \cdot t' \to \forall\ l\ \sigma,\ \mathit{t2l}\ t\ \sigma\ l = \mathit{t2l}\ t'\ \sigma\ l.$

Proof.

By induction on the tree t.

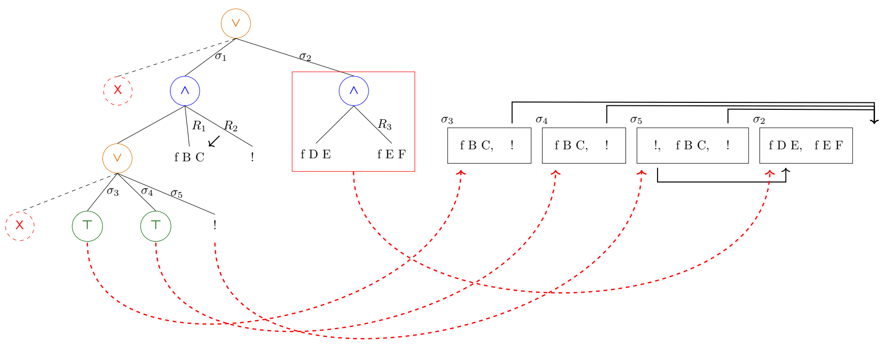


Figure 6.8: Example of `t2l` execution

□

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

Theorem 6.4.2 (tree_to_elpi). ∀ `p t σ r, let v := vars_tree t ∪ vars_sigma σ in`

```
valid_tree t →
  runT' p v σ t r →
    let a := t2l t σ ∅  in
    let r′ :=  if r is ·(σ′, t) then  ·(σ′, t2l (odflt KO t) σ ∅ )
               else □ in
    runS p v a r′.
```

Proof.

We proceed by induction on the execution of `runT`’. There are four cases to consider. The first case arises from *StopT*; it deals with successful trees. A successful tree must start with an empty list, and therefore we conclude using *StopS*. The case for *StepT* requires treating two subcases: `step` evaluates either a cut or a call. Accordingly, we apply *CutS* or *CallS*, followed by the induction hypothesis. The case for *BackT* is silent: it represents the non-injective behavior of `t2l`; however, thanks to lemma 6.4.1 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation *FailT*. It is easily proved using *FailS* and the fact that if `prune` returns □ when trying to erase failure in t, then `t2l` applied to t returns the empty list.

□

The converse direction is stated following [5]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns a unexplored tree that translates to the unexplored stack.

Theorem 6.4.3 (elpi_to_tree). ∀ `p v a r, runS p v a r →`

```
∀ σ t, valid_tree t → t2l t σ ∅  = a →
  if r is ·(σ′, a′) then
    ∃ t′, runT' p v σ t ·(σ′, t′) ∧ t2l (odflt KO t′) σ ∅  = a′
  else runT' p v σ t □.
```

Proof.

$$\begin{tikzcd} t \arrow[r, "\text{xx } t"] \arrow[d, "\text{t2l}"'] & t' \arrow[d, "\text{t2l}"] \\ a \arrow[r, "\text{yy } a"'] \arrow[ur, dash] & a' \end{tikzcd}$$

The proof is based on the idea explained in [5, Section 3.5.1] and depicted in the figure on the right: we have a tree t whose translation is a, we proceed by induction on the execution of `runS`. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to a stack is a'.

□

Stating the converse direction in a symmetric way would require a function `l2t` transforming a stack a into a tree t. Writing this function is far from trivial, since the structure of the tree is lost by `save_as` and `runT` that intuitively keep the state as a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover, one would need to define a notion of valid stack that is equally intricate.

The constructive nature of the logic of Rocq tells us that the proof above provides an effective function translating stacks into trees. However, that function also uses an execution trace of the stack-based semantics, from which it recovers the missing tree structure.

6.5 Case study: determinacy analysis

An interesting application of the tree semantics is determinacy checking, introduced in version 3 of the *Elpi* language [21]. The checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate generate at most one solution; we call this particular kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking.

A predicate behaves deterministically if two conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to any given query, whereas *rule checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Determinacy checking is strongly related to the modes of predicates: we differentiate between input and output arguments. A deterministic call relates to its inputs only one output. In the literature, we find several works on determinacy checking, namely [57, 39, 13]. These works need a mode checker that statically ensures that, in a query with ground inputs, each recursive call is performed with ground inputs.

In our mechanization, and in particular in the *Elpi* language, we adopt a special unification routine for input arguments, called `matching` [3, 71]. Similarly to `unify`, `matching` takes the query term t_1 and the pattern t_2 and returns an optional substitution σ' that does not assign any variable in the query.

The function `matching` has the following property:

Axiom 6.5.1 (matching_disj)**.** $\forall\ \sigma\ \sigma'\ t_1\ t_2,\ \mathtt{vars}\ t_1\ \#\ \mathtt{domf}\ \sigma \rightarrow \mathtt{vars}\ t_1\ \#\ \mathtt{vars}\ t_2 \rightarrow$

$$\mathtt{matching}\ t_1\ t_2\ \sigma = \cdot\sigma' \rightarrow \exists\ e,\ \mathtt{domf}\ \sigma' = \mathtt{domf}\ \sigma \cup e \land e \subseteq \mathtt{vars}\ t_2.$$

If the variables (`vars`) of two terms are disjoint ($\#$) and the variables in the support of σ are disjoint from the variables in t_1, then, if matching succeeds, only the variables in t_2 are assigned; that is, $t_1 = \sigma'\ t_2$, and the variables in t_1 are not assigned in σ'.

The `backchain` function uses `matching` or `unify` on the arguments q depending if the argument is in *input* or *output* mode. We assume two important properties of our unification routines.

Axiom 6.5.2 (unif_trans)**.** $\forall\ t_1\ t_2\ t_3\ \sigma,\ \mathtt{unify}\ t_1\ t_2\ \sigma \rightarrow \mathtt{unify}\ t_2\ t_3\ \sigma \rightarrow \mathtt{unify}\ t_1\ t_3\ \sigma.$

Axiom 6.5.3 (match_unif)**.** $\forall\ t_1\ t_2\ \sigma,\ \mathtt{matching}\ t_1\ t_2\ \sigma \rightarrow \mathtt{unify}\ t_1\ t_2\ \sigma.$

We also assume that refreshing variables does not affect unification. In particular, a refreshing renaming f for a term t is a function from variables to variables that: (i) f is defined on all the variables in t, i.e. all variables get renamed; (ii) f is injective, i.e. does not confuse variables; and (iii) f has non-overlapping domain and codomain, i.e. it maps the variables of the term to a set of completely fresh variables.

Below, we define `refresh_for` to test the validity of a refreshing renaming. Axiom 6.5.4 states the property of term unifiability with respect to term renaming. That is, if the renamings of terms t_1 and t_2 are unifiable under two respective valid refreshing renamings, and they have disjoint variables, then for any pair of renamings z and x that are valid renamings for t_1 and t_2, respectively, and are disjoint from each other, unification also succeeds. In other words, this theorem accounts for α-equivalence of terms and the unification of terms with disjoint variables.

Definition 6.5.1 (refresh_for $\mathtt{x}$ t)**.** $(\mathtt{vars}\ t \subseteq \mathtt{domf}\ \mathtt{x}) \land \mathtt{injective}\ \mathtt{x} \land (\mathtt{domf}\ \mathtt{x}\ \#\ \mathtt{codomf}\ \mathtt{x}).$

Axiom 6.5.4 (unif_ren)**.** $\forall\ x\ y\ z\ w\ t_1\ t_2,$

$$\begin{array}{l}\mathtt{refresh\_for}\ w\ t_1 \rightarrow \mathtt{refresh\_for}\ y\ t_2 \rightarrow \mathtt{refresh\_for}\ z\ t_1 \rightarrow \mathtt{refresh\_for}\ x\ t_2 \rightarrow \\ \mathtt{codomf}\ w\ \#\ \mathtt{vars}\ (\mathbf{rename}\ y\ t_2) \rightarrow \mathtt{codomf}\ z\ \#\ \mathtt{vars}\ (\mathbf{rename}\ x\ t_2) \rightarrow \\ \mathtt{unify}\ (\mathbf{rename}\ w\ t_1)\ (\mathbf{rename}\ y\ t_2)\ \varepsilon \rightarrow \mathtt{unify}\ (\mathbf{rename}\ z\ t_1)\ (\mathbf{rename}\ x\ t_2)\ \varepsilon.\end{array}$$

In fig. 2.4, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are expected to be functions, as specified by the **`func`** keyword, whereas `r` is a relation, as indicated by the **`pred`** keyword. The **`->`**

symbol separates inputs from outputs; therefore, all three predicates are expected to take an `int` as input and return an `int` as output.

The signatures of the program are stored in the `sig` field of the program and are encoded as described in [21]: we distinguish data from predicates, and predicates are classified into deterministic a non-deterministic.

A program execution behaves deterministically if, given a program, a substitution, and an input tree for the `runT` inductive, the execution either fails or, if it succeeds, the resulting tree of alternatives is □.

Definition 6.5.2 (is_det `p σ v t`)**.** `∀ r, runT' p v σ t r → r = □ ∨ ∃ σ, r = ·(σ, □).`

The theorem we want to prove is the following one:

Theorem 6.5.5 (det_check_call)**.** `∀ p σ t v,`

`check_ℙ p → tm_is_det p.(sig) t → is_det p σ v (Todo (call t)).`

Here, `check_ℙ` denotes the function that checks the program with respect to mutual exclusion and rule checking, and `tm_is_det` denotes the function that tests whether the head of the term refers to a rigid constant with a deterministic type.

The proof of this lemma proceeds by induction on the program execution. However, without further work, the induction hypothesis is too weak since it talks about `Todo` trees (i.e. atoms), whereas we need it on any tree.

To address this issue, we introduce the notion of deterministic trees (`det_tree`), show that a tree satisfying this condition enjoys the desired properties, and then prove the following more general theorem:

Lemma 6.5.6 (det_check_tree)**.** `∀ σ v p t, check_ℙ p → det_tree p.(sig) t → is_det p σ v t.`

Our target theorem `det_check_call` is a corollary of `det_check_tree`.

6.6 Related work

We studied a *Prolog* semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the *Elpi* interpreter, which is in turn inspired by B-Prolog's "matching-clauses" [3, 71]. The two semantics we presented (and proved equivalent) also apply to standard *Prolog* if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the proofs only as follows: axiom 6.5.1 does require the query q to be ground. Adapting our results to standard *Prolog* therefore requires the insertion of two additional hypotheses in theorem 6.5.5. First, one has to require that t has ground inputs. Second, that the program p is well-moded. Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. Therefore, the proofs in section 6.5 still apply.

The only mechanization of Prolog's semantics we could find is the one by Pusch in *Isabelle* [50] that is based on the model of Debray and Mishra [12, Sec. 4.3]. Their work starts from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [69, 3]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in section 6.3 that pushed us to develop a more explicit semantics (with respect to cut). In a sense, our work is complementary. By combining both works one can show the tree-based semantics is equivalent to an optimized stack-based one.

Another relevant work is by King [37], but we could not find their Rocq source code. Their semantics is denotational and cannot easily cover all the features that [21] covers, but it would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [5], and we thank Y. Bertot for insightful discussions on the topic. In [5], Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence.

His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of theorem 6.4.3.

6.7 Conclusion

This paper describes two operational semantics for Prolog with cut. The first semantics is based on And–Or trees: it is well suited to graphically illustrating the scope of cut, and to proving lemmas about the impact of its evaluation when reasoning about disjunctions. The second semantics is based on a stack of alternatives: it is computationally more efficient, but it loses the structural expressiveness of the tree-based semantics. Thanks to a dedicated function, we translate trees into stacks and prove the two semantics equivalent. This stack-based semantics corresponds to the first-order fragment of *Elpi*. Using the tree-based semantics, we mechanize the first-order part of the determinacy checker that has been available in *Elpi* since version 3.0, and we prove that a call to a deterministic predicate returns at most one solution.

The formal development consists of approximately 2,500 lines of specifications and about 5,000 lines of *ssreflect* proofs. Roughly 30% of the code is dedicated to the tree semantics (in particular, the proofs in section 6.2.4), 15% to the stack semantics, and 35% to the equivalence proof. The remaining 20% is devoted to the determinacy checker. The mechanization took us approximately 8 months.

To fully model the *Elpi* language and mechanize [21] entirely, we would need to account for higher-order variables, i.e. variables representing predicates. This extension is ongoing and largely orthogonal to the present paper, since cut and higher-order features do not interact in subtle ways.

CHAPTER 7

Conclusion and Perspectives

Bibliography

[1] Andreas Abel and Brigitte Pientka. Extensions to miller's pattern unification for dependent types and records. Preprint, https://www.cs.mcgill.ca/~bpientka/papers/unif_miller60.pdf, 2018.

[2] James H. Andrews. The witness properties and the semantics of the prolog cut. *Theory Pract. Log. Program.*, 3(1):1–59, January 2003. doi:10.1017/S1471068402001540.

[3] Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.

[4] Gilles Barthe. Implicit coercions in type systems. In Stefano Berardi and Mario Coppo, editors, *Types for Proofs and Programs*, pages 1–15, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

[5] Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: https://inria.hal.science/inria-00073199v1.

[6] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development. Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer Verlag, 2004.

[7] Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.

[8] Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013.

[9] Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.

[10] Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34, doi:10.4230/LIPIcs.FSCD.2020.34.

[11] A. de Bruin and E. P. de Vink. Continuation semantics for prolog with cut. In Josep Díaz and Fernando Orejas, editors, *TAPSOFT '89*, pages 178–192. Springer, 1989.

[12] Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.

[13] Saumya K. Debray and David S. Warren. Functional computations in logic programs. *ACM Trans. Program. Lang. Syst.*, 11(3):451–481, July 1989. doi:10.1145/65979.65984.

[14] Gilles Dowek. *Higher-order unification and matching*, page 1009–1062. Elsevier Science Publishers B. V., NLD, 2001.

[15] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: https://inria.hal.science/hal-01176856v1, doi:10.1007/978-3-662-48899-7_32.

[16] Amy Felty. Encoding the calculus of constructions in a higher-order logic. In M. Vardi, editor, *lics93*, pages 233–244. IEEE, June 1993. doi:10.1109/LICS.1993.287584.

[17] Amy Felty and Dale Miller. Specifying theorem provers in a higher-order logic programming language. In Ewing Lusk and Ross Overbeek, editors, *9th International Conference on Automated Deduction*, pages 61–80, Berlin, Heidelberg, 1988. Springer Berlin Heidelberg.

[18] Davide Fissore. Elpi formalization. https://github.com/gares/elpi-formalization, 2025.

[19] Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: https://inria.hal.science/hal-04467855.

[20] Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.

[21] Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.

[22] Thom Frühwirth. Constraint handling rules - what else? In Nick Bassiliades, Georg Gottlob, Fariba Sadri, Adrian Paschke, and Dumitru Roman, editors, *Rule Technologies: Foundations, Tools, and Applications*, pages 13–34, Cham, 2015. Springer International Publishing.

[23] Thom Frühwirth. Theory and practice of constraint handling rules. *The Journal of Logic Programming*, 37(1):95–138, 1998. URL: https://www.sciencedirect.com/science/article/pii/S0743106698100055, doi:10.1016/S0743-1066(98)10005-5.

[24] Andrew Gacek. The abella interactive theorem prover (system description). In *Proceedings of the 4th International Joint Conference on Automated Reasoning*, IJCAR '08, page 154–161, Berlin, Heidelberg, 2008. Springer-Verlag. doi:10.1007/978-3-540-71070-7_13.

[25] Georges Gonthier. A computer-checked proof of the Four Color Theorem. Technical report, Inria, March 2023. URL: https://inria.hal.science/hal-04034866.

[26] Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Pasca, Laurence Rideau, Alexey Solovyev, Enrico Tassi, and Laurent Théry. A machine-checked proof of the odd order theorem. In Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie, editors, *Interactive Theorem Proving - 4th International Conference, ITP 2013, Rennes, France, July 22-26, 2013. Proceedings*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer, 2013. URL: https://inria.hal.science/hal-00816699v1, doi:10.1007/978-3-642-39634-2_14.

[27] Georges Gonthier and Enrico Tassi. A language of patterns for subterm selection. In Lennart Beringer and Amy Felty, editors, *Interactive Theorem Proving*, pages 361–376, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[28] Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.

[29] Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/S0960129518000427.

[30] Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. *Mathematical Structures in Computer Science*, 29:1–26, 03 2019. doi:10.1017/S0960129518000427.

[31] Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip L. Wadler. Type classes in haskell. *ACM Trans. Program. Lang. Syst.*, 18:109–138, March 1996. doi:10.1145/227699.227700.

[32] Michael Hanus and Kai-Oliver Prott. Determinism Types for Functional Logic Programming. In *Proceedings of PPDP*, pages 47–59, 2025.

[33] Fergus Henderson, Zoltan Somogyi, and Thomas Conway. Determinism analysis in the mercury compiler. In *Proceedings of Australian Computer Science Conference*, pages 337–346, 08 1996.

[34] G.P. Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science*, 1(1):27–57, 1975. URL: https://www.sciencedirect.com/science/article/pii/0304397575900110, doi:10.1016/0304-3975(75)90011-0.

[35] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi:10.1017/S0956796818000151.

[36] Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27, doi:10.4230/LIPIcs.ITP.2025.27.

[37] Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. *Theory and Practice of Logic Programming*, 11(4-5):537–553, 2011.

[38] P. López-García, F. Bueno, and M. Hermenegildo. Determinacy analysis for logic programs using mode and type information. In Sandro Etalle, editor, *Logic Based Program Synthesis and Transformation*, pages 19–35, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[39] Lunjin Lu and Andy King. Determinacy inference for logic programs. *Lecture Notes in Computer Science*, 3444:108–123, 04 2005. doi:10.1007/978-3-540-31987-0_9.

[40] Assia Mahboubi and Enrico Tassi. Canonical structures for the working coq user. In Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie, editors, *Interactive Theorem Proving*, pages 19–34, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[41] Assia Mahboubi and Enrico Tassi. *Mathematical Components*. Zenodo, September 2022. doi:10.5281/zenodo.7118596.

[42] math-comp. finmap — finite maps for mathcomp, 2026. URL: https://github.com/math-comp/finmap.

[43] Spiro Michaylov and Frank Pfenning. Higher-order logic programming as constraint logic programming. In *Position Papers for the First Workshop on Principles and Practice of Constraint Programming*, pages 221–229, Newport, Rhode Island, April 1993. Brown University.

[44] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

[45] Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, New York, NY, USA, 2012.

[46] Dale A. Miller. Mechanized metatheory revisited. In *Journal of Automated Reasoning*, volume 63, pages 625–665. Springer, 2018.

[47] Gopalan Nadathur. The metalanguage λprolog and its implementation. In Herbert Kuchen and Kazunori Ueda, editors, *Functional and Logic Programming*, pages 1–20, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

[48] Gopalan Nadathur and Dale A. Miller. An overview of lambda-prolog. In *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, pages 810–827, Seattle, WA, USA, 1988. MIT Press.

[49] Katsuhiko Nakamura. Control of logic program execution based on the functional relations. In *Proceedings of ICLP*, page 505–512. Springer, 1986.

[50] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.

[51] Frank. Pfenning and Conal Elliott. Higher-order abstract syntax. In *Proceedings of PLDI*, page 199–208. Association for Computing Machinery, 1988. doi:10.1145/53990.54010.

[52] Brigitte Pientka. Tabling for higher-order logic programming. In Robert Nieuwenhuis, editor, *Automated Deduction – CADE-20*, pages 54–68, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[53] Xiaochu Qi. *An Implementation of the Language Lambda Prolog Organized around Higher-Order Pattern Unification*. Ph.d. thesis, University of Minnesota, Minneapolis, MN, USA, 2009. Available at the University Digital Conservancy.

[54] Rocq Development Team. Class. https://rocq-prover.org/doc/v9.1/refman/addendum/type-classes.html#coq:cmd.Class. Rocq Prover Reference Manual, version 9.1. Accessed: 2026-03-11.

[55] Rocq Development Team. Hint mode. https://rocq-prover.org/doc/v9.1/refman/proofs/automatic-tactics/auto.html#coq:cmd.Hint-Mode. Rocq Prover Reference Manual, version 9.1. Accessed: 2026-03-11.

[56] Daniel Selsam, Sebastian Ullrich, and Leonardo de Moura. Tabled typeclass resolution. *CoRR*, abs/2001.04301, 2020. URL: https://arxiv.org/abs/2001.04301, arXiv:2001.04301.

[57] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming*, 29(1):17–64, 1996. High-Performance Implementations of Logic Programming Systems. URL: https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684, doi:10.1016/S0743-1066(96)00068-4.

[58] Matthieu Sozeau and Nicolas Oury. First-class type classes. In *Proceedings of TPHOLs*, volume 5170 of *LNCS*, pages 278–293. Springer, 2008.

[59] Christopher Strachey. Fundamental concepts in programming languages. *Higher Order Symbol. Comput.*, 13(1–2):11–49, April 2000. doi:10.1023/A:1010000313106.

[60] SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-Prolog PlDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.

[61] Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: https://inria.hal.science/hal-01637063.

[62] Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019. URL: https://inria.hal.science/hal-01897468, doi:10.4230/LIPIcs.CVIT.2016.23.

[63] Enrico Tassi. Elpi: rule-based extension language. https://github.com/gares/HDR/blob/main/hdr.pdf, 2025.

[64] Enrico Tassi and Freek Wiedijk. ? In ?, editor, *?*, 2026.

[65] The Coq Development Team. The Coq reference manual – release 8.18.0. https://coq.inria.fr/doc/V8.18.0/refman, 2023.

[66] Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.

[67] Philip Wadler. Theorems for free! In *Proceedings of FPCA*, FPCA '89, page 347–359. Association for Computing Machinery, 1989. doi:10.1145/99370.99404.

[68] Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '89, page 60–76, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/75277.75283.

[69] David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983. URL: https://www.sri.com/wp-content/uploads/2021/12/641.pdf.

[70] Markus Wenzel. Type classes and overloading in higher-order logic. In Elsa L. Gunter and Amy Felty, editors, *Theorem Proving in Higher Order Logics*, pages 307–322, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.

[71] Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*, 12(1–2):189–218, January 2012. doi:10.1017/S1471068411000445.

Tritre francais

Davide FISSORE